Université de Bretagne Occidentale

Ecole Doctorale MATHSTIC

Sciences et Technologies de l'Information et de la Communication

Habilitation à Diriger des Recherches

Soutenue par :

Mathieu Léonardon

Sobriety through Co-design in Deep Learning and Digital Communications

Rapporteurs

Bertrand Le Gal	Maître de conférences,	Université de Rennes
Frédéric Pétrot	Professeur des Universités,	Université de Grenoble Alpes
Andrea Pinna	Professeur des Universités,	Sorbonne Université

Examinateurs

Steven Derrien	Professeur des Universités,	Université de Bretagne Occidentale
Catherine Dezan	Maître de conférences,	Université de Bretagne Occidentale

Contents

1 Foreword **4**

2 Digital Communication Algorithms on Programmable Architectures **6**
2.1 Decoding of Polar Codes on General Purpose Processors 7
2.1.1 SC-List and SCAN decoding algorithms 7
2.1.2 A Fast and Flexible Software Implementation of Polar Code Decoders 11
2.1.3 ISTC'25 Challenge 16
2.2 The AFF3CT Project 19
2.2.1 AFF3CT: A Software Suite for Digital Communications 19
2.2.2 A Real-time DVB-S2 Transceiver using Multicore and SIMD CPUs 25
2.2.3 AFF3CT: Next Steps and ISTC 2027 Challenge 28
2.3 ASIP Architectures for Polar Codes 30
2.3.1 Tensilica 30
2.3.2 Transport Triggered Architecture 30

3 Deep Learning on Embedded Targets **33**
3.1 Low-Cost Federated Learning 34
3.1.1 Federated Learning Overview 34
3.1.2 A first contribution: Pruning messages 36
3.1.3 FLOCORA: Federated Learning Compression with LORA 37
3.2 Compression by Pruning or Quantization and Embedded Deployment 40
3.2.1 Pruning Neural Networks and Hardware Acceleration 40
3.2.2 A practical compression case: GoodFloow 47
3.2.3 CNN acceleration with SIMD on ARM and x86 architectures 52
3.2.4 Emerging technologies for deep learning 52
3.3 Deep Learning on FPGA Targets 53
3.3.1 Semantic Segmentation on FPGA for Autonomous Vehicles 53
3.3.2 PEFSL: Few-Shot Learning on FPGA-SoC 57
3.4 Teaching, Supervision, and Research 61
3.4.1 Supervision and Funding Strategies 61
3.4.2 Building a Local Talent Pool 62
3.4.3 Spotlight on Two Courses: PARPING and EFFDL 62

4 Research Perspectives **65**
4.1 New Methods and Architectures for Digital Communications 65
4.1.1 AI agents to assist the development of scientific software 65
4.1.2 Hardware Exploration Facilitated by Evolutionary Algorithms 67

4.2 Algorithm-Architecture Co-design . . . 69
4.2.1 Application of Deep Learning for GNSS Denoising in a Spatial Context . . . 69
4.2.2 Split-Processing and Event-Based Vision for Edge AI . . . 71
4.3 Future Hardware for Deep Learning - Modeling and Exploration . . . 71
4.3.1 Automated Hardware Search . . . 71
4.3.2 Hardware Architectures for AI (PEPR CAMELIA) . . . 73
4.3.3 Edge AI for Health: From Microcontrollers to Wearable ASICs . . . 74

A Curriculum Vitae **76**
A.1 Summary of Professional Career and Academic Activities . . . 76
A.2 Teaching Activities . . . 83
A.3 Publications . . . 84
A.3.1 PhD Thesis . . . 84
A.3.2 Publications in International Journals . . . 84
A.3.3 Publications in International Conferences with Proceedings . . . 84
A.3.4 Publications in National Conferences with Proceedings . . . 86
A.4 Summary of Supervision Activities . . . 88
A.4.1 Doctoral Student Supervision . . . 88
A.4.2 Master's Internship Supervision . . . 89
A.4.3 Postdoctoral Supervision . . . 89
A.4.4 Research Engineers . . . 89
A.5 Research Funding . . . 90
A.5.1 Public Funding . . . 90
A.5.2 Private Funding . . . 90
A.6 References . . . 92

Chapter 1

Foreword

The structure of this manuscript reflects my progression in research. I began my career developing programmable solutions for high-throughput, low-latency, and energy-efficient decoding of error-correcting codes, before shifting toward deep learning compression, its deployment on embedded systems, and the design of dedicated hardware architectures. In the introduction to this manuscript, it is important to identify the common threads linking my various contributions across these two major fields. If I refer to "sobriety" in the title of this manuscript, it is because these different works consistently intersect with this theme.

Regarding this theme of sobriety, I must immediately clarify that I do not interpret it from an ecological or environmental standpoint. While a large portion of my work demonstrably reduces the energy measured per produced result (corrected bits, processed images, etc.), from a higher-level perspective, these gains in energy efficiency do not reduce overall consumption. Instead, they allow us to do more with the same resources. The use of the word "sobriety" here is not a claim to an ecological impact I do not pretend to have, but rather a core design philosophy. Indeed, there are systematically massive, untapped reserves of efficiency in recent innovations. In the field of deep learning, this borders on the caricatural. For example, as seen in Hugo Tessier's work on pruning, certain techniques allow for the removal of over 90% of a neural network's weights without any loss of accuracy.

Another common characteristic of my work, and of my approach to research in general, is the pursuit of a certain pragmatism. I consider it essential to systematically anchor results through demanding benchmarks or by participating in competitions—an approach I also actively apply in my teaching, as discussed in the corresponding chapter. However, these benchmarks and competitions only hold value if they are replicable. This is why I strive to make my work as reproducible as possible by systematically publishing the source codes and datasets used. This is a capital aspect of my research, a culture I built during my PhD at the University of Bordeaux and have continued to cultivate at IMT Atlantique in Brest.

My ambitions are now focused on ensuring the long-term sustainability and integration of my work. Having established a robust foundation in digital communications with the AFF3CT project, I deliberately expanded my research scope toward deep learning to tackle the pressing challenges of hardware-algorithm adequation. I now wish to reinvest in the AFF3CT ecosystem in collaboration with colleagues in France and abroad, presenting avenues in this manuscript that I hope will spark new partnerships. On the deep learning side, my initial contributions were deliberately exploratory, targeting various hardware platforms and compression paradigms. I now aim to consolidate these foundational works into a unified, global vision, structured around the automated co-exploration of neural network topologies and hardware architectures.

To articulate this trajectory, the remainder of this manuscript is organized as follows:

- Chapter 2 details my work in digital communications, focusing on the optimized decoding of polar codes and the development of the AFF3CT software suite for high-performance simulations and software-defined radio.
- Chapter 3 transitions to my research in embedded artificial intelligence, highlighting algorithmic compression techniques—such as pruning and quantization—and their deployment on constrained targets like FPGAs and microcontrollers.
- Chapter 4 outlines my future research perspectives, proposing to merge these experiences through automated hardware-software co-design and AI-assisted engineering frameworks to address the broader challenges of sustainable and sovereign computing.

Chapter 2

Digital Communication Algorithms on Programmable Architectures

This chapter summarizes my research activities. I completed a doctoral thesis titled *Decoding of Polar Codes on Programmable Architectures*. These works, whose part was also done during a temporary lecturer position (ATER), led to the creation of open-source software for simulating complete digital communication chains and their implementation in software-defined radio (SDR). This line of research is detailed in Section 2.1. When I joined IMT Atlantique, I reoriented my research towards the compression of deep learning algorithms and their deployment on embedded targets: FPGAs, embedded processors, microcontrollers, and embedded GPUs. Section 3 presents the main contributions in this area.

Software implementations of error-correcting code decoders play a crucial role in several areas of modern digital communications. Firstly, they are essential for Monte Carlo simulations, enabling the evaluation of communication system performance under various noise and interference scenarios. Secondly, in the context of Cloud-RAN (Cloud Radio Access Networks), software decoders offer the flexibility needed to virtualize physical layer functions, thereby facilitating cooperative multi-point processing. These decoders are indeed the most computationally intensive building blocks of the communication chain. Finally, for software-defined radios (SDR), they allow dynamic reconfiguration of communication parameters, adapting systems to different standards and channel conditions without requiring hardware modifications.

My PhD thesis presents efficient implementation methods (throughput, latency, energy consumption) for polar code decoding algorithms on specific hardware architectures (ASIP: Application Specific Instruction-set Processor) but also on general-purpose processors (GPP: General Purpose Processor). This work, in collaboration with Adrien Cassagne in Bordeaux (Adrien now joined the LIP6 and Sorbonne Université), led to the creation of the open-source software AFF3CT, a complete software toolbox for the simulation and implementation of digital communication chains.

2.1 Decoding of Polar Codes on General Purpose Processors

In the present manuscript, on the subject of polar codes, I will focus the discussion on my key contributions in the field of polar code decoding. I will therefore skip everything concerning code construction and decoding techniques that fall outside the scope of my contributions. I invite the reader to refer to my PhD manuscript for these topics [1]. I will, however, attempt to clearly explain the structure of the SCL and SCAN decoding algorithms, their scheduling, and their potential for parallelization. I will also describe their refinements (FAST-SCL and Adaptive simplification) in Section 2.1.1 before clearly presenting the contributions I proposed and the results obtained in Section 2.1.2. More recent developments and new results, obtained during my participation in the ISTC 2025 (International Symposium on Topics in Coding) conference, are presented in Section 2.1.3.

2.1.1 SC-List and SCAN decoding algorithms

Polar codes, introduced by Arıkan in 2009, are the first error-correcting codes to provably achieve Shannon capacity for infinite code lengths [29]. The Successive Cancellation (SC) decoding algorithm was initially proposed for polar codes, but its performance in terms of bit error rate (BER) and frame error rate (FER) is insufficient for short code lengths. To address this, the Successive Cancellation List (SCL) algorithm was proposed in the literature, significantly improving decoding performance by maintaining a list of candidate paths. An optimized version, called Fast SCL, was subsequently developed to reduce computational complexity while maintaining good decoding performance. These algorithms allow for trade-offs between complexity, latency, and decoding performance, making polar codes suitable for modern communication standards like 5G.

The polarization phenomenon and polar codes were developed assuming the use of the Successive Cancellation (SC) decoding algorithm. This is a hard-output algorithm, meaning the output data consists of bits. However, a second decoding algorithm, Belief Propagation (BP), was also proposed. This one is a soft-output algorithm. This means that the output data are estimates typically represented as LLRs (Log-Likelihood Ratios). As a reminder, in the LLR representation, the sign corresponds to the hard decision: a bit equal to 0 for a positive LLR, and a bit equal to 1 for a negative LLR. The absolute value indicates the reliability of this decision.

Although the SC algorithm is asymptotically optimal as N tends to infinity, its performance is limited for finite-sized codes. Numerous variants of these algorithms have therefore been proposed to improve their error correction performance. The Successive Cancellation List (SCL) algorithm is an evolution of the SC algorithm. Furthermore, the algorithm called Soft Cancellation (SCAN) can be seen as a combination of the SC and BP algorithms. Each of these algorithms is described in this section.

$$\begin{array}{lcl} f(L_a, L_b) & = & \text{sign}(L_a.L_b).\min(|L_a|, |L_b|) \\ g(L_a, L_b, \hat{s}_a) & = & (1-2\hat{s}_a)L_a + L_b \\ h(\hat{s}_a, \hat{s}_b) & = & (\hat{s}_a \oplus \hat{s}_b, \hat{s}_b) \\ \texttt{R0}(L_a) & = & 0 \\ \texttt{R1}(L_a) & = & \begin{cases} 0 \text{ si } L_a \geq 0 \\ 1 \text{ si } L_a < 0 \end{cases} \end{array} \tag{2.1}$$

Successive Cancellation decoding can be represented as a traversal of a binary tree. The tree representation, as shown in Figure 2.1, is an alternative representation that highlights certain algorithmic simplifications. The data are organized in a binary tree with $log_2(N)+1$ **levels**. In our

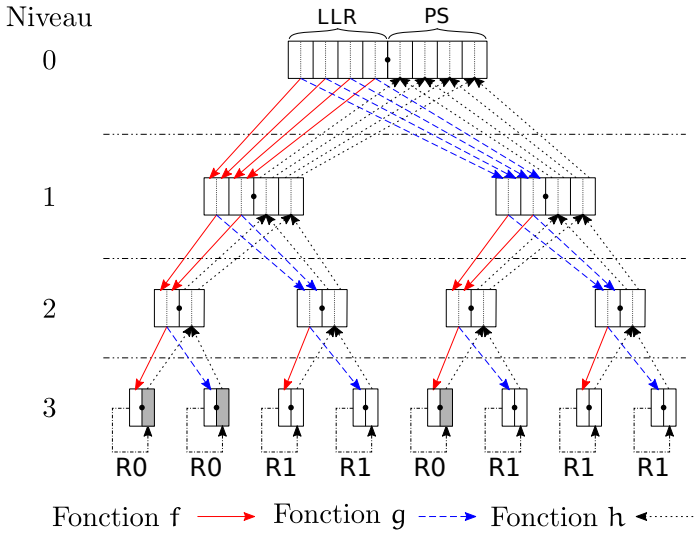


Figure 2.1: SC decoding tree.

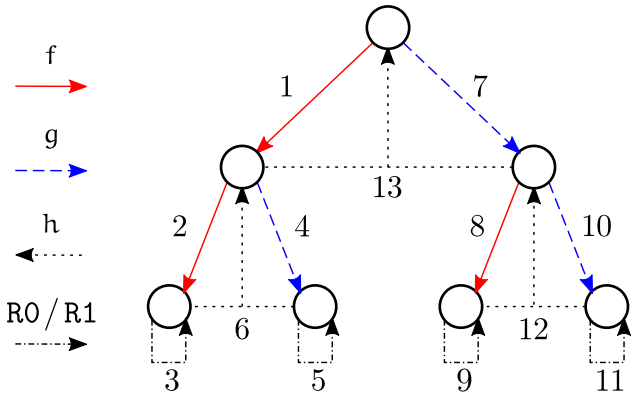


Figure 2.2: Scheduling of SC decoding for a polar code of size $N = 4$.

pedagogical example, the tree has four levels. At each level, a certain number of **nodes** are assigned. The **root** of the tree, numbered 0, consists of a single node. The root contains N LLRs and N partial sums. As the tree is traversed, at each level, the number of nodes doubles. Conversely, the number of LLRs and partial sums in each node is halved. The **leaves** are the nodes of the last level of the tree (level 3 in Figure 2.1). Processing a leaf corresponds to applying the `R0` and `R1` operations. Each level d contains 2^d nodes, each consisting of 2^{n-d} LLRs and 2^{n-d} partial sums, where $n = \log_2(N)$. The partial sums are propagated by the function h. They are propagated up to a level that depends on the index of the leaf that has just been processed. In the case of a non-systematic polar code, the decoded sequence $\hat{u}$ is composed of the partial sums contained in the leaves of the decoding tree. During non-systematic decoding, the partial sums contained in the leaves correspond to the decoded sequence $\hat{u}$.

Figure 2.2 is a more compact representation of the decoding tree. Operations 12 and 13 in Figure 2.2 are therefore redundant since they only serve to compute the partial sums of the root node. In the case of a systematic code, it is the partial sums of the root node that correspond to the decoded codeword. The traversal of the tree is a depth-first traversal. The scheduling of the different functions is detailed in Figure 2.2, where each arrow representing a function is numbered according to the execution order.

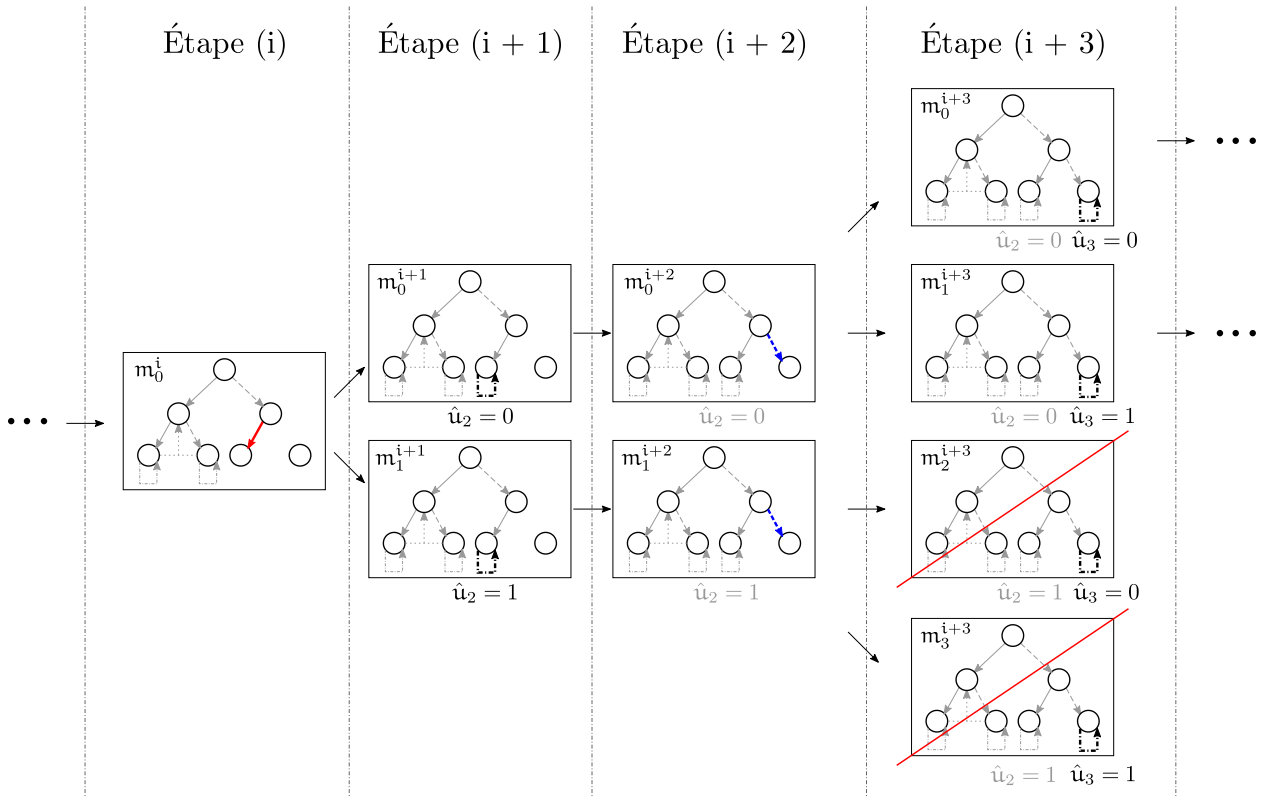


Figure 2.3: Duplication de l'arbre de décodage dans l'algorithme SCL.

Successive Cancellation List (SCL) decoding algorithm extends the SC algorithm [31]. The SC algorithm suffers from limited error-correction performance for short and moderate-length polar codes ($N < 8192$), whereas SCL decoding significantly improves these results.

In the previously described SC decoder, hard decisions are applied whenever a leaf corresponding to an information bit is processed (operation `R1`). If an error is made at this stage, it cannot be corrected, and the decoding of the codeword fails. The key idea of list decoding is to postpone hard decisions. Instead of applying a threshold directly to the LLR, both possible decoding outcomes are explored. To do so, the decoding tree, together with all its LLRs and partial sums, is duplicated. This process is illustrated in Figure 2.3.

The algorithm is shown at step (i), where the function f is applied. At the next step (i+1), the decoding tree is duplicated to create two paths: one assuming $\hat{u}_2 = 0$, and the other assuming $\hat{u}_2 = 1$. Decoding then proceeds in parallel along both paths. At step (i+2), the function g is applied. These two trees are referred to as decoding **paths**. At each leaf of rate 1, the same procedure is applied, doubling the number of paths to be decoded in parallel, as shown at step (i+3).

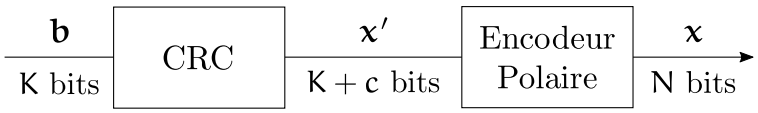


Figure 2.4: Concatenation with a CRC.

Concatenation with a CRC The authors of [31] proposed concatenating a Cyclic Redundancy Check (CRC) to discriminate between the different candidates. The corresponding encoding is illustrated in Figure 2.4. A CRC is an error-detecting code. It associates with any binary sequence, here $\boldsymbol{b}$ of length K, a sequence of length $K + c$. After decoding, the CRC verification allows the detection of potential errors. The CRC is designed so that the probability of an undetected error is extremely small. Thus, at the end of the decoding algorithm, when the L candidate codewords at the output of the SCL algorithm are available, a CRC verification is performed. If one of them passes the CRC check, it is then highly probable that it is the correct candidate. In the rare case where several candidates pass the CRC check, the one with the smallest metric among them is selected. When this candidate selection mechanism based on CRC verification is applied, the algorithm is referred to as CRC-Aided SCL (CASCL).

Adaptive SCL Decoding Algorithms The CASCL algorithm significantly improves decoding performance compared to the SC algorithm. However, this improvement comes at the cost of increased computational complexity. Nevertheless, it is possible to combine the two algorithms, SC and CASCL, in order to approach the low computational complexity of the former while preserving the decoding performance of the latter [32]. The principle is as follows: a first decoding attempt is performed with the SC algorithm. The CASCL algorithm is then triggered only if a decoding error is detected through CRC verification.

Two variants of the adaptive algorithm are possible. In the case of the Partially Adaptive SCL (PASCL) algorithm presented in [33], when SC decoding fails, the CASCL algorithm is executed only once with a list size of L_{max}. In the case of the Fully Adaptive SCL (FASCL) algorithm originally proposed in [32], the CASCL algorithm may be applied several times. The first CASCL decoding attempt is performed with $\mathrm{L} = 2$. If, after decoding, the CRC is still not satisfied, the algorithm is relaunched as many times as necessary, iteratively doubling L until reaching L_{max}.

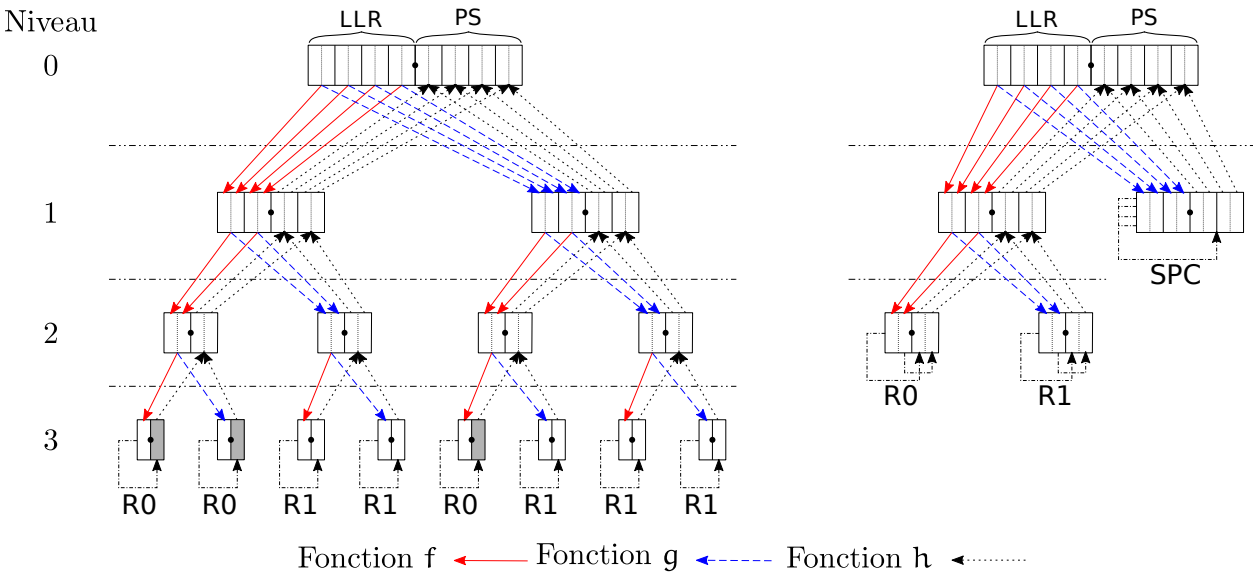


Figure 2.5: Pruning of the SC decoding tree.

Pruning of the polar decoding tree To reduce the computational complexity of the SC decoding algorithm, the decoding tree can be pruned as described in [34]. For instance, a subtree

whose leaves are all rate-0 (frozen bits) does not need to be traversed, since all its partial sums are necessarily zero. Similarly, a subtree whose leaves are all rate-1 can be replaced by directly applying the thresholding function `R1` at its root, which yields the same result as a full traversal.

Beyond these two simple cases, other subtree patterns have also been identified, such as repetition (`REP`) and single parity check (`SPC`) nodes [35]. For these structures, dedicated decoding techniques can be applied to derive the partial sums directly from the LLRs at the root, again avoiding traversal of the entire subtree.

The advantage of such pruning, often referred to as Fast-SC, is a drastic reduction of both computational and control complexity, which is especially relevant for software implementations where tree traversal is costly. Importantly, these techniques are not limited to SC decoding: they can be extended to list decoding as well, leading to the Fast-SCL algorithm [33]. The complexity reduction becomes particularly significant when the code rate is close to 0 or close to 1.

2.1.2 A Fast and Flexible Software Implementation of Polar Code Decoders

The main contribution of my thesis was the proposition of a fast and flexible software implementation of polar code decoders [4]. Software implementation of channel codes has emerged as a compelling alternative to dedicated hardware. It enables rapid prototyping and validation of new algorithms, while offering high flexibility to support a wide variety of codes and decoding strategies. Unlike hardware, software decoders can be easily configured to trade off correction capability, latency, and throughput, making them well suited for adaptive schemes. They also facilitate reproducibility and portability across platforms. At the network level, software implementations align with the trend toward Cloud-RAN, where virtualized signal processing allows dynamic allocation of resources, improved inter-cell cooperation, reduced energy consumption, and lower communication latency.

The software implementation on General Purpose Processors (GPP) is a complete discipline, and it is not possible to present here the full set of methods, best practices, and techniques required to achieve good latency and throughput performance. On a side note, this expertise is the one I teach in the course named "PARPING", which is described in Section 3.4.3. Let us just mention a few of the most important techniques that I applied in my implementations.

First, vectorization. Every modern GPP includes vector processing units, which are hardware modules within the processor capable of performing several identical computations in parallel on different (but contiguous) data. This is referred to as SIMD – Single Instruction Multiple Data. In polar decoding trees, these instructions are particularly useful to accelerate processing, as they allow several of the previously listed functions to be computed simultaneously when traversing from one node of the tree to another. Vectorization can be applied within a single decoding tree (intra-frame) or by processing several frames in parallel (inter-frame). The first case is more challenging since the number of processed data varies with the depth of the decoding tree. This number may even fall below the size of the vector registers at the lowest nodes, resulting in incomplete utilization of parallelism. In contrast, inter-frame parallelization does not suffer from this limitation, since the number of frames can be chosen to fully occupy the vector registers. However, its drawback lies in frame size: for large codewords, the cache capacity of the processors may be exceeded, leading to a collapse of performance.

A second technique, closely related to vectorization, is data quantization. It is possible to represent data in SC and SCL decoding trees with fewer than 32 bits, typically 16 or 8. This brings two main advantages. First, it increases the parallelism of vector instructions. Second, it reduces the memory footprint of the decoding trees, thereby decreasing cache usage and lowering the memory bandwidth requirements.

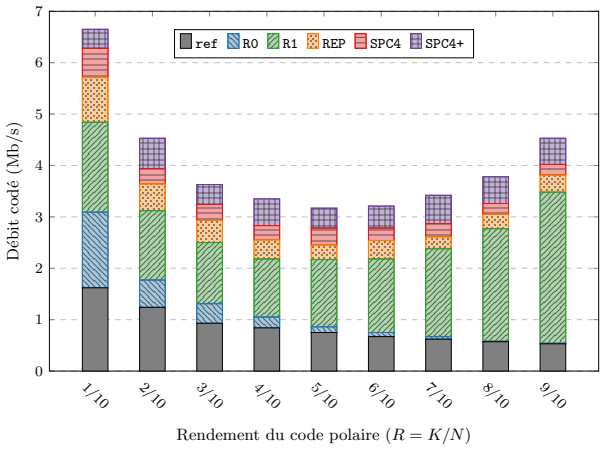


Figure 2.6: Impact of activating pruning nodes in the CASCL algorithm, $N = 2048$, L $= 32$, $c = 32$.

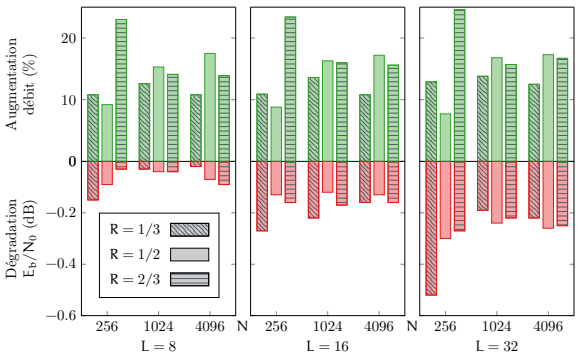


Figure 2.7: Effects of using SPC4+ nodes in the CASCL algorithm for a FER of 10^{-5}.

Finally, another notable technique is *unrolling*. This consists in explicitly expanding the loops of the decoding algorithm in the source code, instead of relying on iterative loop execution. By doing so, the control overhead associated with loop counters and branching is eliminated, and the compiler can optimize the resulting straight-line code more aggressively. In the context of polar decoders, unrolling allows to reduce the latency of critical functions by removing conditional branches and improving instruction-level parallelism. However, the main drawbacks are, first, the increase in code size, which can become significant for large code lengths, and second, the lack of flexibility: for implementations supporting different code sizes or rates, dedicated source codes would need to be generated, which is not practical.

I proposed several major contributions for the software implementation of SCL decoders [4].

Fine-grained Pruning Configuration I allowed fine control over tree pruning in our polar decoder implementations, enabling or disabling different node types (R0, R1, REP, SPC) independently. Figures 2.6 and 2.7 illustrate how enabling certain nodes affects coded throughput and decoding performance. High-rate codes benefit more from R1 pruning, while low-rate codes gain more from R0 and REP pruning; SPC nodes show a less clear trend. Allowing SPC nodes larger than 4 (SPC4+) increases throughput by 8–20% but may slightly degrade FER depending on the code and list depth. The pruning is dynamically activated, suggesting potential for a real-time adaptive pruning software decoder.

Fixed-Point Representation I implemented SCL decoders using 8- and 16-bit fixed-point representations for LLRs and path metrics to increase parallelism and improve throughput. Figure 2.8 shows that 16-bit fixed-point representation matches floating-point performance, while 8-bit representation can cause overflow in large repetition nodes (REP). Disabling large repetition nodes (REP8-) eliminates performance loss for 8-bit representation. Table 2.1 demonstrates that fixed-point representations reduce latency and increase throughput, e.g., FASCL achieves 425.9 Mb/s with 8-bit LLRs. This allows efficient, high-throughput software implementations without significant loss in decoding performance.

Adaptive SCL Algorithm Variants Two adaptive SCL algorithms were implemented: Partially Adaptive (PASCL) and Fully Adaptive (FASCL), shown in Figure 2.9. PASCL is faster at low

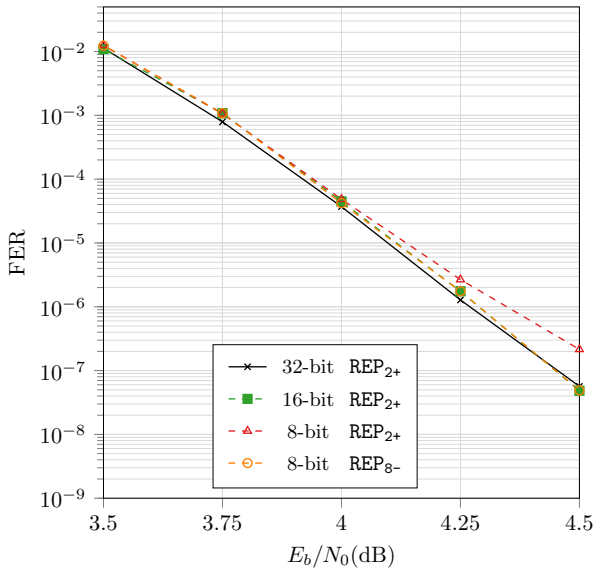


Figure 2.8: Impact of using repetition nodes on quantized implementations.

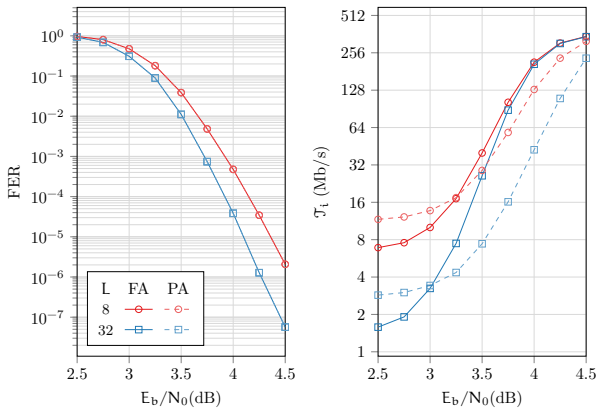


Figure 2.9: Decoding performance and throughput of the PASCL and FASCL decoding algorithms for a polar code (2048,1723), L = 32, and CRC $c = 32$ GZip.

SNRs due to fewer intermediate decodings, while FASCL is superior at intermediate SNRs because it executes the algorithm with multiple intermediate list sizes. At high SNRs, both algorithms achieve similar throughput as SC decoding often suffices. Worst-case latency is higher for FASCL, as verified in Table 2.1. These implementations highlight the flexibility of our decoder, including dynamic pruning and support for different list-adaptive strategies.

Partial Sum Management We store partial sums in fixed-size integers matching LLR representation to exploit SIMD instructions. Memory footprint is reduced from L$(2N-1)$ to LN for SCL by reusing memory when possible. We compare two assignment methods during duplication: copying (SCL_{cpy}) and pointer-based (SCL_{ptr}). SCL_{cpy} is faster for small and medium codes ($N < 8192$), while SCL_{ptr} is more efficient for large codes, especially with 8-bit representation (Figure 2.10). This optimization reduces memory overhead and improves throughput depending on code size and data representation.

Table 2.1: Comparison of throughputs and latencies of adaptive SCL algorithms for floating-point (32-bit) and fixed-point (16 and 8-bit) representations. Polar code (2048,1723), L = 32, CRC $c = 32$ (GZip).

Décodeur	**Quantif.**	$\mathcal{L}_{PC}$	**3.5 dB**		**4.0 dB**		**4.5 dB**	
			$\mathcal{L}_{moy}$	$\mathcal{T}_i$	$\mathcal{L}_{moy}$	$\mathcal{T}_i$	$\mathcal{L}_{moy}$	$\mathcal{T}_i$
PA-SSCL	32-bit	635	232.3	7.6	41.7	42.1	7.4	237.6
	16-bit	622	219.6	8.0	40.1	43.8	6.6	267.5
	8-bit	651	232.4	7.6	41.2	42.6	6.5	268.3
FA-SSCL	32-bit	1201	67.2	26.1	8.5	207.8	5.1	345.5
	16-bit	1198	68.7	25.6	7.7	225.7	4.3	408.7
	8-bit	1259	71.8	24.4	7.7	227.3	4.1	425.9

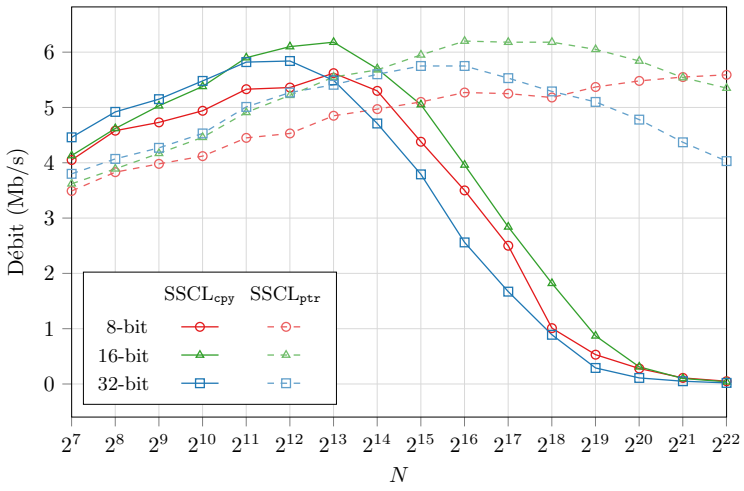


Figure 2.10: Comparison of throughputs associated with the SCL_{cpy} and SCL_{ptr} methods for different values of N.

Partial Sorting Algorithm A new sorting method based on Schreier's tournament algorithm was proposed to select the smallest LLRs or path metrics during SCL execution [36]. Compared to bubble sort, quick-sort, heap-sort, PSN networks, Schreier's method achieves the fastest execution in all tested scenarios, including `SPC` nodes. This simplifies the software design by using a single, efficient algorithm for all partial sorting operations. Although PSNs could theoretically improve throughput, we do not implement them due to the complexity of creating generic networks for any list size and pruning configuration, which would reduce flexibility and increase code size.

Table 2.2: Comparison of throughputs and latencies of the proposed decoders with those from the state of the art. LLRs and partial sums represented on 32 bits. Polar code (2048,1723), L = 32, CRC GZip $c = 32$.

Target	**Algorithm**	**Version**	$\mathcal{L}_{PC}$ (μs)	$\mathcal{T}_i$ (Mb/s)		
				3.5 dB	**4.0 dB**	**4.5 dB**
i7-4790K	CASCL [1]	[37]	1572	1.10	1.10	1.10
i7-2600	CASCL[1]	[35]	23000	0.07	0.07	0.07
i7-2600	CASCL[1]	[33]	2294	0.76	0.76	0.76
i7-2600	CASCL[1]	This work	4819	0.37	0.37	0.37
i5-6600K	CASCL[1]	This work	3635	0.48	0.48	0.48
i7-2600	CASCL	[35]	3300	0.52	0.52	0.52
i7-2600	CASCL	[33]	433	4.0	4.0	4.0
i7-2600	CASCL	This work	770	2.3	2.3	2.3
i5-6600K	CASCL	This work	577	3.0	3.0	3.0
i7-2600	PASCL	[35]	$\approx$ 3300	0.9	4.90	54.0
i7-2600	PASCL	[33]	$\approx$ 433	8.6	33.0	196.0
i7-2600	PASCL	This work	847	5.5	31.1	168.4
i5-6600K	PASCL	This work	635	7.6	42.1	237.6
i7-2600	FASCL	This work	1602	19.4	149.0	244.3
i5-6600K	FASCL	This work	1201	26.1	207.8	345.5

[1] Non-pruned version of the CASCL algorithm.

Accelerated CRC Checking Profiling shows that CRC verification consumes significant execution time, as it is performed once per SC decoding and L times per SCL decoding. We accelerate CRC by packing 32 bits together, using precomputed CRC sequences, and exploiting pruning nodes for parallel extraction. For FASCL decoding of a polar code (2048,1723) with CRC-32 at $E_b/N_0 = 4.5$ dB, throughput increases from 262 Mb/s to 426 Mb/s (63% gain). This approach reduces the computational complexity of CRC extraction and verification while integrating efficiently with SIMD operations.

Comparison with the State of the Art The throughputs and latencies of the proposed implementations are detailed in this section. All reported results were obtained using the AFF3CT tool [6]. The source code of this software is open and available online. Therefore, all results are easily reproducible. The hardware target is an Intel i5-6600K CPU with Skylake architecture. The AVX2 SIMD instruction set is exploited. Its frequency for the reported results is 3.9 GHz. Compilation was performed on a Linux OS with the GNU C++ compiler (version 5.4.0). The applied compilation options are: `-Ofast -march=native -funroll-loops`.

Table 2.3: Comparison of throughputs, number of clock cycles required to decode one frame, and energy per decoded bit, for the proposed decoders on different hardware targets. Fixed-point representation on 8 bits, $E_b/N_0 = 4.5dB$, CRC GZip $c = 32$.

Target	Freq. (MHz)	Algo.	$\mathcal{T}_i$ (Mb/s)			# Cycles per frame			$\mathcal{E}_b$ (nJ/bit)		
			3.5 dB	4.0 dB	4.5 dB	3.5 dB	4.0 dB	4.5 dB	3.5 dB	4.0 dB	4.5 dB
i7 4790	3592	CASCL	2.34	2.38	2.24	2645	2600	2763	6325	6218	6607
		PASCL	5.80	33.88	207.96	1067	183	30	2552	437	71
		FASCL	19.43	194.66	372.30	319	32	17	762	76	40
Ryzen 2700X	4230	CASCL	2.68	2.72	2.56	2675	2635	2800	5149	5074	5391
		PASCL	6.55	38.76	237.78	1094	185	30	2107	356	58
		FASCL	20.93	209.37	406.15	342	34	18	659	66	34
Cortex A73	2360	CASCL	1.23	1.28	1.20	3310	3175	3382	1475	1415	1507
		PASCL	3.08	18.90	100.75	1319	215	40	588	96	18
		FASCL	10.64	83.58	137.37	382	49	30	170	22	13
Cortex A15	2000	CASCL	0.74	0.74	0.70	4639	4639	4923	3365	3365	3571
		PASCL	1.81	11.06	61.86	1899	312	56	1378	226	40
		FASCL	6.23	52.77	88.31	553	65	39	401	47	28
Cortex A53	1840	CASCL	0.48	0.48	0.45	6560	6560	7064	1353	1353	1457
		PASCL	1.17	7.32	42.77	2701	433	74	557	89	15
		FASCL	4.45	39.70	68.32	712	80	46	147	16	10
Cortex A7	1400	CASCL	0.28	0.29	0.27	8615	8318	8934	1071	1034	1111
		PASCL	0.71	4.26	21.89	3397	566	110	423	70	14
		FASCL	2.41	19.40	30.93	1001	124	78	124	15	10

Our work demonstrates a significant advantage over fully unrolled decoders from the literature by offering a high degree of flexibility. The proposed implementations are generic and easily portable, allowing the use of quantized representations, configurable puncturing patterns, and adaptive decoding algorithms—features not available in prior works. The throughput and latency results of the proposed decoders are detailed in Table 2.2, showing that FASCL consistently achieves the highest performance. With an 8-bit representation of LLRs and partial sums, FASCL reaches 425 Mb/s for a polar code (2048,1723) with a 32-bit GZip CRC at $E_b/N_0 = 4.5$ dB, nearly twice the throughput of PASCL. In the FER region of 10^{-3} to 10^{-5}—relevant for wireless communication standards such as LTE and future 5G networks—FASCL decodes at approximately 230 Mb/s, whereas PASCL achieves only 40 Mb/s. This confirms that FASCL remains state-of-the-art, as will be further illustrated in Section 2.1.3.

Importantly, Table 2.3 presents the first software implementations on ARM architectures, demonstrating excellent energy efficiency. ARM cores such as the Cortex-A73 achieve compet-

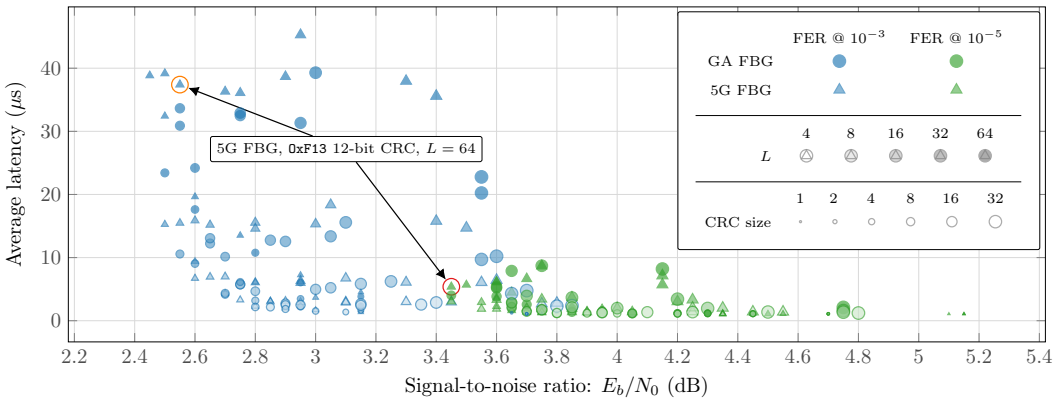


Figure 2.11: Space exploration of the ASCL decoder performance for $K = 128$ and $R = 1/2$. Measured latencies from the AFF3CT generic ASCL decoder running on one core of an Intel Xeon Gold 6140 CPU. The selected code is highlighted in orange for FER @ 10^{-3} and in red for FER @ 10^{-5}.

itive cycle counts compared to desktop CPUs while consuming only around 2 W versus 15 W, resulting in a 3-4x improvement in energy efficiency. Lower-power cores like the Cortex-A7 and A53 maintain good energy efficiency, whereas the Cortex-A15 shows slightly lower performance per watt. These results highlight the flexibility, portability, and energy-performance benefits of our implementations, making them suitable for both high-performance x86 workstations and embedded ARM platforms.

2.1.3 ISTC'25 Challenge

The ISTC'25 Challenge on "High-Performance Software Implementations of Polar Codes" was organized to evaluate and compare the performance of software implementations of polar code decoders. The challenge focused on achieving high throughput and low latency while maintaining competitive error-correction performance. Participants were required to implement decoders for specific polar code configurations and report their results in terms of throughput, latency, and error rates. We participated with a student of mine, Mohammed EL Houcine Ayoubi and my colleague from Bordeaux and Sorbonne Université. We reused the implementations developed in the previous section, and added some features, detailed in the following.

Bit-packed Encoder The encoder implementation relies on bit-packing and proceeds in four stages. First, the input vector U_N, containing both frozen and information bits, is generated. This is done by statically identifying constituent nodes at compile time: rate-0 nodes are skipped, and large rate-1 nodes can be vectorized during memory copy. Second, U_N is bit-packed using SIMD instructions, specifically `_mm256_cmpgt_epi8` followed by `_mm256_movemask_epi8`, which extracts the MSBs of each 8-bit element to form packed output. Third, systematic Polar encoding is performed using vectorized XOR and shift operations, applied intra- or inter-register depending on the node position in the encoding graph. Finally, unpacking is achieved using a 256-entry lookup table of 256-bit values.

Unrolled ASCL Decoder The decoder implemented in this work follows the ASCL decoding principle. This approach aims to reduce the average latency of Polar decoders without compromising error-correction performance. As described before, decoding begins with a fast SC decoder.

Table 2.4: Proposed codes for the contest with the corresponding achieved performance (fully unrolled ASCL decoders).

						Decoder				
		Code specifications			Encoder		FER @ 10^{-3}		FER @ 10^{-5}	
Rate	K	CRC size	CRC poly.	Fb. gen.	$\mathcal{L}_{\text{worst}}$ (μs)	L	E_b/N_0	$\mathcal{L}_{\text{avg}}$ (μs)	E_b/N_0	$\mathcal{L}_{\text{avg}}$ (μs)
1/4	64	11	`0x385`	5G	0.20	64	2.15 dB	2.53	3.35 dB	0.55
	128	12	`0xF13`	5G	0.28	64	1.60 dB	6.44	2.60 dB	1.07
	256	12	`0xF13`	5G	0.60	64	1.25 dB	10.58	2.25 dB	1.66
	512	12	`0xF13`	GA	1.01	32	1.10 dB	14.92	1.85 dB	3.53
1/2	64	7	`0x65`	5G	0.14	32	3.15 dB	0.70	4.60 dB	0.33
	128	12	`0xF13`	5G	0.21	64	2.55 dB	3.31	3.45 dB	0.69
	256	12	`0xF13`	5G	0.34	64	2.15 dB	6.55	3.10 dB	1.18
	512	16	`0x8005`	5G	0.68	64	1.90 dB	11.95	2.70 dB	2.12
4/5	64	8	`0x9B`	GA	0.15	32	5.05 dB	0.48	6.45 dB	0.16
	128	10	`0x3D9`	5G	0.20	32	4.40 dB	1.21	5.35 dB	0.41
	256	10	`0x3D9`	5G	0.34	32	4.10 dB	2.20	5.00 dB	0.67
	512	12	`0xF13`	5G	0.66	32	3.75 dB	4.92	4.55 dB	1.36

If the decoded codeword passes the CRC check, it is accepted immediately. Otherwise, the decoder falls back to more powerful SCL decoders with increasing list sizes L. This adaptive strategy significantly lowers the average decoding time in high-SNR regimes, where SC decoding is often sufficient, while preserving the robustness of SCL decoding in more challenging conditions. The ASCL mechanism is well suited for short blocklength codes, where latency is critical. Short blocklenghts were the focus of the ISTC 2025 contest, with blocklengths ranging from 80 to 2048 bits.

To achieve low latency, we adopt a fully unrolled decoder generation strategy. For each Polar code configuration used in the contest, a dedicated and specialized C++ source file is generated. This eliminates control-flow overhead such as loops and recursive function calls, making the approach particularly effective for short blocklengths where latency is critical. We provide an open-source decoder generator tool[1], which supports SC, SCL, and ASCL decoding modes. The tool produces optimized C++ code tailored to each frozen set, enabling reproducibility and integration into larger systems.

Search space and exploration To identify optimal trade-offs for the ISTC 2025 contest, we explored various code and decoder configurations, including different code constructions (GA, 5G) [38, 39], CRC lengths and polynomials – including some polynomials that do not correspond to standard CRCs – and list sizes L. This search helped uncover "sweet spots" balancing latency and error-correction performance. An example of the results obtained for $K = 128$ and $N = 256$ is shown in Figure 2.11.

Performance results The testbed was a Minisforum AtomMan X7 Ti PC (Intel Ultra 9 185H, 16 x86-64 cores with 6 p-cores @ 5.1 GHz (one core used), 8 e-cores @ 3.8 GHz (unused), and 2 LPe-cores (unused), 32 GB DDR5 @ 5600 MT/s) running Linux (Ubuntu 25.04 kernel 6.14, g++ 14.2.0) has been used. Table 2.4 summarizes the proposed Polar code configurations submitted for the contest, along with their measured performance metrics. The table provides details on the code rate, information block length K, CRC size and polynomial, code construction method, worst-case encoder latency, list size L, and decoder performance in terms of Frame Error Rate (FER) at 10^{-3} and 10^{-5} thresholds, together with corresponding average decoding latencies.

[1]Polar decoder generator: https://github.com/aff3ct/polar_decoder_gen

1st place and future directions The results obtained in the ISTC 2025 contest were very promising, with our submission achieving 1st place in the software implementation category ([2]). This success highlights the effectiveness of our approach and the optimizations implemented in the AFF3CT software suite. The next section is about AFF3CT, its features, and its impact on the field of digital communications. In the end I will also develop our plan to participate in the organization for the next edition of the ISTC contest.

[2] https://2025.istc-conf.org/program/implementation-competition-awards/

2.2 The AFF3CT Project

During my PhD years, followed by a year as an ATER at the University of Bordeaux in the IMS laboratory, a dynamic emerged around the development of a complete software simulation chain, initiated in particular by Adrien Cassagne, then a PhD student in the team. This momentum led to the creation of a software suite named AFF3CT[3][6]. This long-term project (soon to celebrate its 10th anniversary!) has had a strong impact both nationally and internationally, resulting in numerous publications ranging from contributions to software decoders, to the definition of a dedicated programming language for software design, and its use in a variety of application contexts.

For my part, in this document I will focus on the projects in which I was most involved, namely the creation of the chain itself, followed by the implementation of a software radio chain based on the DVB-S2 standard.

2.2.1 AFF3CT: A Software Suite for Digital Communications

AFF3CT is an open source software suite designed for the simulation and implementation of communication systems. At the time of its creation, it focused on Monte Carlo simulations of the whole communication chain: some data is generated, encoded, modulated, noised, decoded, and the performance is then estimated by measuring the Bit Error Rate (BER) and the Frame Error Rate (FER). The main motivation behind the development of AFF3CT was to address several challenges commonly encountered in the field of channel coding simulations:

1. **Simulation time:** In order to accurately estimate the FER/BER, at least 100 erroneous frames must be simulated. For example, measuring a FER of 10^{-7} requires simulating the transmission of approximately $100 \times 10^7 = 10^9$ frames. Assuming a frame length of 1000 bits, the simulator must emulate the transmission of 10^{12} bits. Considering that decoding algorithms may be computationally demanding, several weeks or even months of simulation may be required to obtain accurate FER/BER estimates for a given coding scheme.

2. **Algorithmic heterogeneity:** A wide variety of channel codes have been developed over time. For each type of code, several decoding algorithms may be available. While describing a single coding scheme is relatively straightforward, providing a unified software framework that supports multiple coding schemes and their associated algorithms is much more challenging. This difficulty stems from the heterogeneity of the data structures required to represent codes and decoders: turbo codes rely on trellises, LDPC codes are naturally expressed on factor graphs, and polar codes are efficiently decoded using binary trees. An this extends to channel models, modulation schemes, and other components of the communication chain.

3. **Reproducibility:** Reproducing published results is often a tedious task. This is partly due to the large number of empirical parameters required to specify a communication system, not all of which are systematically reported in publications. Moreover, simulator source codes are rarely made publicly available. As a consequence, significant time is frequently spent "reinventing the wheel" merely to enable comparison with state-of-the-art results.

The long simulation times make it desirable to have **high throughput implementations**. The algorithmic heterogeneity requires **flexible, modular software**. The reproducibility issue advocates for a **portable** and **open-source software**. These are the purposes of AFF3CT.

[3] https://aff3ct.github.io

2.2.1.1 Related Works

Table 2.5: C/C++ open source channel coding simulators/libraries.

Name	Contributors	Lines	Commit		License	Polar	LDPC	Turbo	Turbo P.	BCH	RS	Conv.	RA	Rep.	Erasure
			First	Last											
AFF3CT	11	76k	2016	2019	MIT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗
eccpage	20	-	1989	2006	-	✗	✓	✓	✗	✓	✓	✓	✗	✗	✓
EZPWD	2	6k	2014	2018	GPLv3	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗
FastECC	2	1k	2015	2018	Apache 2.0	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗
FEC-AL	1	3k	2018	2018	BSD 3-Clause	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓
FECpp	1	2k	2009	2018	-	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
GNURadio	192	270k	2006	2019	GPLv3	✓	✓	✗	✗	✗	✗	✓	✗	✓	✗
IT++	20	109k	2005	2018	GPLv3	✗	✓	✓	✗	✓	✓	✓	✗	✗	✗
LeGal	1	83k	2015	2019	-	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Leopard	4	5k	2017	2018	BSD 3-Clause	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗
libcorrect	6	5k	2016	2018	BSD 3-Clause	✗	✗	✗	✗	✗	✓	✓	✗	✗	✗
OpenAir	148	740k	2013	2019	OAI Public	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
OpenFEC	8	55k	2009	2014	CeCCIL-C	✗	✓	✗	✗	✗	✓	✗	✗	✗	✗
Schifra	1	7k	2010	2018	GPLv3	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗
Siamese	1	11k	2018	2018	BSD 3-Clause	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓
Tavildar-Polar	1	2k	2016	2017	-	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Tavildar-LDPC	1	1k	2016	2016	-	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
the-art-of-ecc	1	-	2006	2009	Copyright	✗	✓	✓	✓	✓	✓	✓	✗	✗	✗
TurboFEC	2	4k	2015	2018	GPLv3	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗

In the digital communications community, many scientists implement their own simulation chain to validate their works. Table 2.5 presents, to the best of our knowledge, a list of currently available C/C++ open source channel coding simulators/libraries. We choose to compare with projects compiled as binaries, since they aim at high throughput and low latency, as AFF3CT. Many open source projects in Python or in MATLAB exist as well, but these tools are usually slower than compiled binaries, and rather aim at prototyping.

Table 2.5 shows that, generally, the C/C++ FEC libraries target a single family or a small subset of channel codes. As a consequence, a large effort is spent to re-develop similar features, since all those libraries and tools share many characteristics (except the channel code itself). AFF3CT attempts to lower this redundancy by releasing a full simulator/library that consistently supports a wide range of channel codes to the community. AFF3CT also tries to homogenize usage (command line, C++ interfaces, etc.) for all code families. Table 2.5 does not aim at comparing channel code implementation performances. Instead, on the AFF3CT website, an overview of software channel decoders state-of-the-art is provided for turbo, LDPC and polar codes[4].

2.2.1.2 Software description

AFF3CT is a Forward Error Correction (FEC) toolbox. The most important tools it includes are the *simulator* and the *library*. Additionally, the toolbox comes with a bank of *simulated references*, a *GUI software* to browse those references, and a set of predefined *configuration files* for common communication standards.

Simulator As a standalone simulator, AFF3CT proposes to simulate various communication chains with a broad range of codes. The simulator is a command line program; Figure 2.12 depicts

[4]FEC Software Decoders Hall of Fame: `http://aff3ct.github.io/hof_turbo.html`

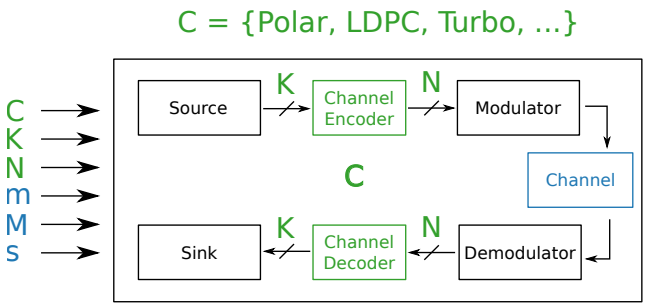
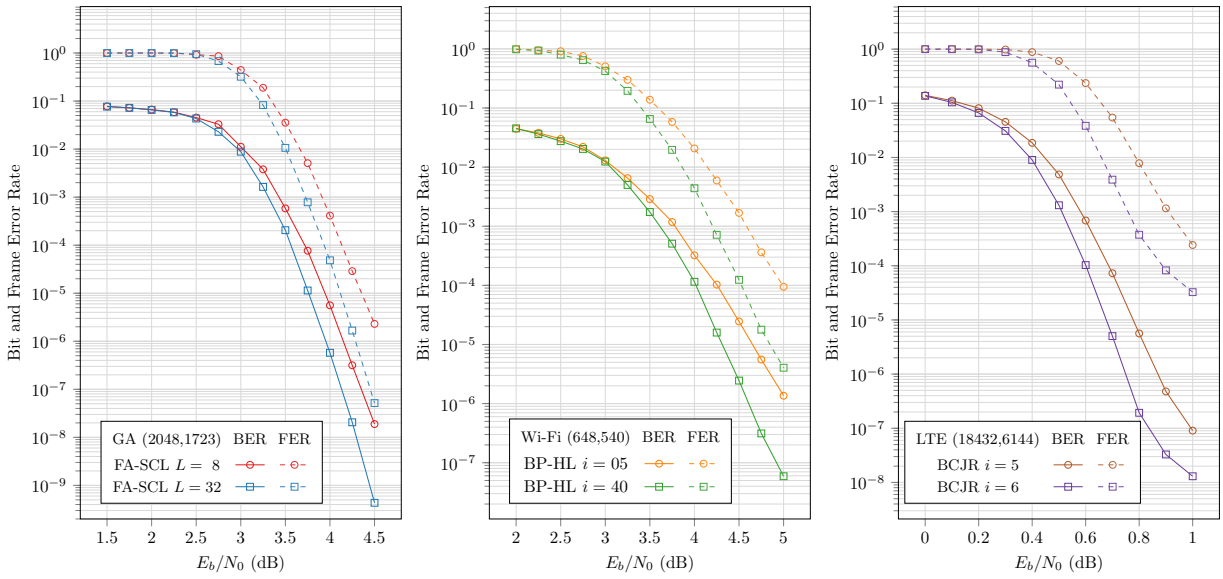
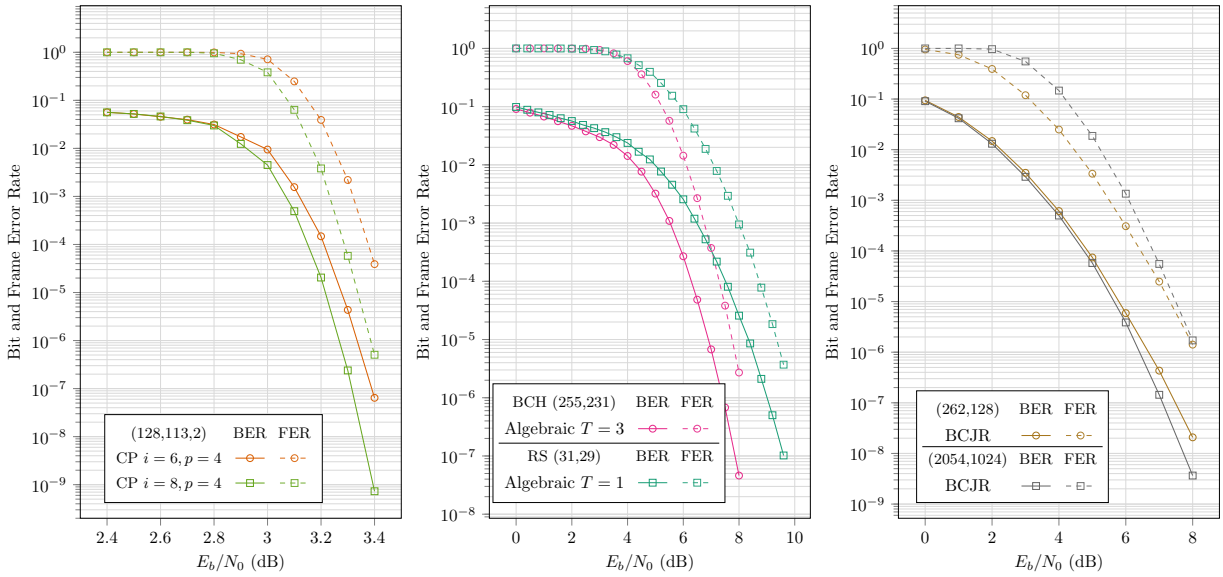


Figure 2.12: Main simulation parameters.



(a) Polar codes. (b) LDPC codes. (c) Turbo codes.



(d) Turbo product codes. (e) BCH & RS codes. (f) Convolutional codes.

Figure 2.13: AFF3CT simulation of various code families.

its main arguments: `-m` and `-M` set the minimum and the maximum Signal Noise Ratio (SNR, E_b/N_0) to simulate, `-s` specifies the iteration step inbetween. Parameter `-C` selects the channel code type, `-K` sets the length in bits of the initial information message and `-N` sets the codeword size, which is the encoder output size[5]. The command line interface is designed to be easily used from scripts. Figure 2.13 shows simulations ran with AFF3CT on various code types (BPSK modulation and AWGN channel).

2.2.1.3 Library

As a FEC library, AFF3CT can be used programmatically, for instance in *Software Defined Radio* (SDR) contexts or in simulations. AFF3CT blocks can be used in external projects without restriction. Compute intensive blocks are optimized and vectorized to run fast on a single core. The library is thread-safe; however, it is not multi-threaded by itself, in contrast to the simulator. Instead, it is the responsibility of the user to manage multi-threading.

2.2.1.4 Software Architecture

AFF3CT is developed in `C++` in an object-oriented programming style. It provides the fundamental blocks involved in building communication chains (sources, modems, codecs, channels, ...). Those blocks are organized as *modules* and *tasks*.

Module: A set of related tasks sharing some characteristics. For instance, the *modem* module contains the *modulate* and *demodulate* tasks.

Task: An elementary processing performed on some data. For instance, *decode* or *modulate* are tasks. The tasks are characterized by their *sockets*. A socket of a task defines an entry point through which the task will consume and/or produce data. There are three kinds of sockets: *input*, *output* and *input/output*, following a philosophy close to *ports* in component-based development approaches.

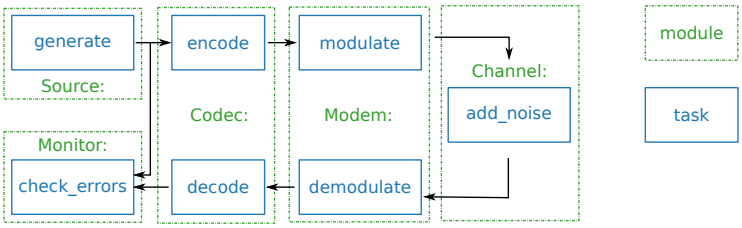


Figure 2.14: Modules and tasks.

As a rule, a task is always a *verb* and a module is always a *noun*. Modules are implemented as C++ classes, and tasks as class methods. AFF3CT defines several abstract modules, for sources, codecs, modems, channels, etc. It readily provides many implementations of those abstract classes; it is also straightforward to add new ones. Figure 2.14 presents common modules and tasks typically found in a basic communication chain. It shows that the number of tasks per module can vary depending on the module type.

2.2.1.5 Software Functionalities

The AFF3CT software functionalities can be decomposed in three main parts: the *codecs*, the *modems* and the *channels*. The codecs are the main part of the toolbox. There is a broad range of

[5]The AFF3CT simulator comes with many more options, omitted here for concision.

supported codes listed in Table 2.6.

Table 2.6: List of supported channel codes (codecs).

Channel Code	Standards	Decoders
LDPC	5G (data), WiFi, WiMAX, WRAN, 10 Gigabit Eth., DVB-S2, CCSDS etc.	Scheduling: Flooding and H./V. Layered Sum-Product Algorithm (SPA) Min-Sum (NMS, OMS) Approximate Min-Star (AMS) Bit Flipping: GallagerA/B/E, PPBF, WBF
Polar	5G (control channel)	Successive Cancellation (SC) Successive Cancellation List (SCL) CRC Aided SCL (CA-SCL, Adaptive SCL) Soft Cancellation (SCAN)
Turbo Parallel (single and double binary)	LTE (3G, 4G), DVB-RCS, CCSDS, etc.	Turbo BCJR Turbo BCJR + Early Termination (CRC) Post proc.: Flip aNd Check (FNC)
Turbo Product	WiMAX (opt.)	Turbo Chase-Pyndiah
BCH	CD, DVD, SSD, DVB-S2, Bitcoin, etc.	Berlekamp-Massey + Chien search
Reed-Solomon	CD, DVD, SSD, DVB-T, ADSL, etc.	Berlekamp-Massey + Chien search
Convolutional (single and double binary)	NASA	BCJR - Maximum A Posteriori (MAP) BCJR - Linear Approximation BCJR - Max Approximation

The codecs naturally encompass the encoders and decoders, but they can also include puncturing patterns to shorten frames length according to some communication standards. Most of the codec algorithms come from the literature, while the others have been designed under AFF3CT [41, 42, 40, 4]. In channel coding, the decoder is the most time-consuming process, compared to the puncturing and the encoding processes. This is why a specific effort is put on ensuring the high computing performance of the decoders. Most of the decoding algorithms have thus been optimized to satisfy high throughput and low latency constraints [43, 44, 46, 45]. Those optimizations generally involve a vectorized implementation, a tailored data quantization and the use of fixed-point arithmetic.

In typical communication chains, it is necessary to adapt the digital signal to the physical support. This operation is performed by the modulator and conversely by the demodulator. AFF3CT comes with a rich set of modems to this purpose. Table 2.7 lists all the supported modems.

AFF3CT supports several coded modulation/demodulation schemes like the Continuous Phase Modulation (CPM) [59, 60] and the Sparse Code Multiple Access (SCMA) modulation [58, 5, 24]. It allows to easily combine and evaluate the channel codes with several types of modulations. In the case of the CPM, analogical wave shapes are also simulated. The other modulation schemes are at the digital level.

For simulation purposes, it is crucial to emulate the behavior of the physical layer. This is the role of the channel. There are many possible configurations depending on the physics phenomena to simulate. Table 2.8 reports all the supported channels.

The channels involve complex floating-point computations. It is frequent to use exponential and trigonometric operations. Those types of operations cost a large amount of CPU cycles to be computed. As for the decoders, the channels have been carefully optimized based on branch instructions reduction and massive vectorization.

Table 2.7: List of supported modulations/demodulations (modems).

Modem	Standard	Characteristics
N-PSK	IEEE 802.16 (WiMAX) UMTS (2G, 2G+) EDGE (8-PSK), ...	Phase-Shift Keying
N-QAM	IEEE 802.16 (WiMAX) UMTS (2G, 2G+) 3G, 4G, 5G, ...	Quadrature Amplitude Modulation
N-PAM	IEEE 802.16 (WiMAX) UMTS (2G, 2G+) 3G, 4G, 5G, ...	Pulse Amplitude Modulation
CPM	GMSK, Bluetooth IEEE 802.11 FHSS	Continuous Phase Modulation Coded (convolutional-based) modulation
OOK	IrDA (Infrared) ISM bands	On-Off Keying Used in optical communication systems
SCMA	Considered for 5G	Sparse Code Multiple Access Multi-user modulation
User defined	-	Constellation and order can be defined from an external file

Table 2.8: List of supported channels.

Channel	Multi-user	Characteristics
AWGN	Yes	Additive White Gaussian Noise
BEC	No	Binary Erasure Channel
BSC	No	Binary Symmetric Channel
Rayleigh	Yes	Flat Rayleigh fading channel
User defined	Not yet	User can import noise samples from an external file

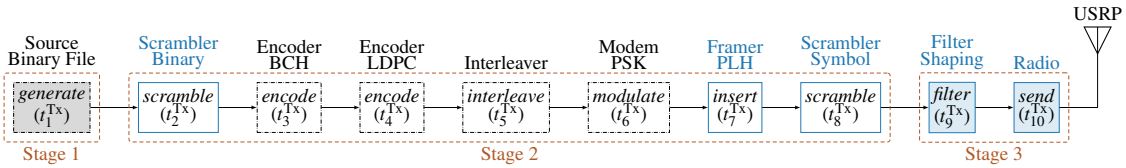


Figure 2.15: DVB-S2 transmitter software implementation.

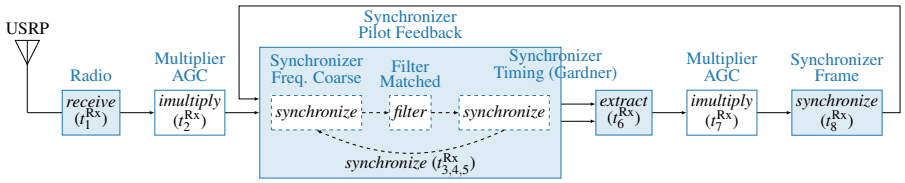


Figure 2.16: DVB-S2 software receiver: waiting phase and acquisition phase 1 & 2.

All these features are available from the simulator and in the library. Many additional functional functionalities available are skipped here for concision.

2.2.1.6 Software Continuous Integration

AFF3CT's development leverages streamlined Continuous Integration (CI) process. Each new commit to the version control repository triggers a comprehensive sequence of tests to catch potential regressions. Regression tests are based on past simulation results, validated from the state-of-the-art.

The CI process enables us to safely and confidently integrate contributed features and improvements from the community to AFF3CT. It also helps to keep the code review time by the core development team low-enough to swiftly integrate such contributions into the master branch.

2.2.2 A Real-time DVB-S2 Transceiver using Multicore and SIMD CPUs

During my ATER position at the University of Bordeaux, our research group was contacted by Airbus Defense and Space to develop a software-defined radio (SDR) implementation of the DVB-S2 standard. The goal was to create a real-time demonstrator capable of streaming video over Universal Software Radio Peripheral (USRP) platforms. This demonstrator serves to showcase the AFF3CT toolbox and its capabilities in practical SDR applications.

The collaboration led to the development of a complete end-to-end DVB-S2 transceiver, including both the transmission and reception chains. In addition to implementing the standard coding and modulation schemes, a key contribution of this work was the integration of robust synchronization functionalities, which were not available in AFF3CT at the time. This enabled reliable frame, timing, and frequency recovery under real-world transmission conditions. The developed system is still used today to demonstrate AFF3CT toolsuite features and to illustrate high-performance SDR processing on general-purpose multicore CPUs.

Software implementations of digital communication standards such as DVB-S2 are particularly relevant in satellite ground stations. They offer flexibility, hardware mutualization, and maintainability advantages over dedicated hardware solutions, especially when the production volume is low or when the system must adapt to evolving standards. However, achieving real-time performance in software is challenging, as the system must process tens of megabits per second with minimal

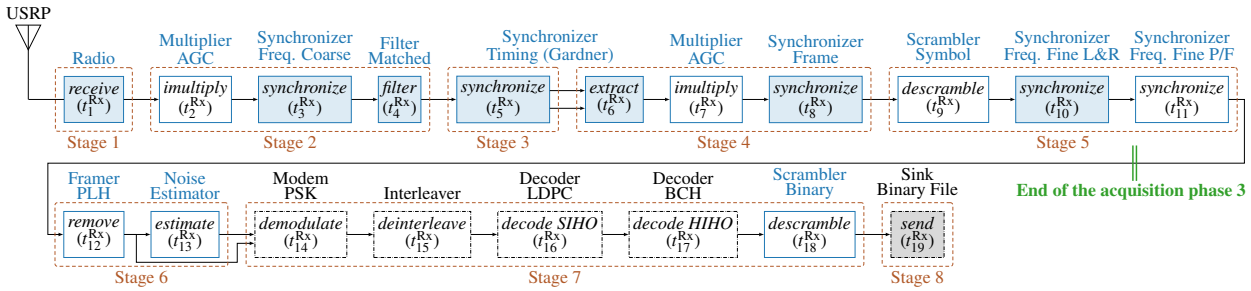


Figure 2.17: DVB-S2 software receiver: acquisition phase 3 & transmission phase.

latency while compensating for imperfections in the transmission chain, such as frequency offsets and timing drifts.

In this context, the proposed SDR implementation combines the flexibility of software with the performance required for real-time operations. It relies on the parallelism of multicore CPUs and Single Instruction Multiple Data (SIMD) vectorization to accelerate processing tasks. The demonstrator supports multiple DVB-S2 MODulation and CODing schemes (MODCODs) and provides precise synchronization through dedicated software modules.

The remainder of this section presents the overall DVB-S2 transceiver architecture, detailing the transmission and reception chains as illustrated in Figs. 2.15, 2.16 and 2.17. This description emphasizes both the processing steps inherited from AFF3CT and the newly implemented modules for synchronization, filtering, and real-time sample handling.

2.2.2.1 DVB-S2 Transceiver Overview

The developed DVB-S2 transceiver is composed of two main parts: the transmitter and the receiver, both implemented in software and interfaced with USRP hardware for analog signal transmission and reception. The design follows a modular approach, where each processing task is clearly separated and can be optimized or parallelized independently.

Transmitter Architecture The transmitter chain, illustrated in Fig. 2.15, reads the source data and applies the DVB-S2 processing steps, including encoding, modulation, and optional interleaving. Additional tasks such as scrambling, pilot insertion, and shaping filtering ensure that the transmitted signal meets the standard requirements and facilitates synchronization at the receiver. Some tasks rely directly on existing AFF3CT modules, while others have been implemented specifically for the SDR context, allowing real-time transmission through the USRP.

Receiver Architecture The receiver chain, depicted in Figs. 2.16 and 2.17, follows a phased processing approach. Initially, the receiver waits for an incoming signal and performs acquisition in several stages to estimate timing, frequency, and frame alignment. Once synchronization is achieved, the transmission phase begins, in which the incoming frames are processed in real time through the full DVB-S2 decoding chain.

Dedicated synchronization tasks ensure resilience to frequency offsets, timing drifts, and phase variations in the received signal. These tasks, along with filters and frame extractors, were newly implemented for this SDR demonstrator and are integrated with the AFF3CT interfaces, providing a modular and instrumented software architecture.

Key Features The SDR implementation highlights several important aspects:

- **Real-time processing:** The receiver pipeline is carefully parallelized and optimized to handle the high data rates imposed by the USRP modules.
- **Flexible and modular design:** The transceiver architecture separates tasks in a way that facilitates adaptation to different MODCODs and transmission scenarios.
- **Synchronization:** Custom software modules compensate for timing and frequency drifts, enabling robust reception under realistic conditions.
- **Integration with AFF3CT:** The design leverages the toolbox for core signal processing and decoding tasks, while extending it with SDR specific functionality.

This high-level architecture demonstrates that a software-only DVB-S2 transceiver can meet real-time constraints using general-purpose CPUs, multicore parallelism, and SIMD acceleration, while remaining fully modular and extensible.

2.2.2.2 Real-Time Implementation Features

Achieving real-time performance for a software DVB-S2 transceiver requires careful management of data flow, processing parallelism, and hardware capabilities. In our SDR demonstrator, the following key strategies were applied:

Reliable Sample Collection and Delivery The USRP modules impose strict constraints on sample throughput and jitter. To avoid underflows and overflows, the radio threads are pinned to dedicated CPU cores and cores are isolated using the Linux `isolcpus` option. Large host-side buffers and a copy-less, lock-free producer/consumer pipeline ensure continuous delivery and collection of samples at high rates.

Vectorized Processing with SIMD Tasks such as synchronization and filtering are heavily vectorized using SIMD instructions. Complex numbers are represented in an array-of-structures format, and the MIPP library is employed to maintain portability across CPU architectures (x86 AVX-512 and ARM NEON). Fused Multiply-Add (FMA) instructions are used in filters to accelerate linear algebra computations.

High-Throughput Pipeline and Parallelization The receiver is decomposed into sequential and parallel pipeline stages, allowing multi-threaded execution of computationally intensive tasks. The pipeline design ensures that slower sequential stages, such as the *Synchronizer Timing* and BCH decoder, do not block the overall throughput more than necessary. Stage duplication is applied for highly parallelizable tasks to maximize CPU utilization, with careful memory placement for NUMA systems.

Task Throughputs and Latencies Table 2.9 summarizes the throughput and latency of each sequential task measured for a typical MODCOD in the transmission phase on an x86 platform. The table highlights the sequential stages (blue), the slowest sequential stage (red), and the slowest stage overall (orange). This overview allows identifying pipeline bottlenecks and guiding parallelization strategies.

Table 2.9: Receiver sequential task characteristics on the x86 CPU. Sequential tasks are represented by blue rows. The slowest seq. stage is in red while the slowest of all is in orange.

Stages and Tasks	Throughput (Mb/s)	Latency (μs)	Time (%)
Radio - *receive* (t_1^{Rx})	431.83	527.32	0.94
Stage 1	431.83	527.32	0.94
Multiplier AGC - *imultiply* (t_2^{Rx})	367.45	619.71	1.11
Synch. Freq. Coarse - *synchronize* (t_3^{Rx})	841.32	270.66	0.48
Filter Matched - *filter* (t_4^{Rx})	116.41	1956.08	3.49
Stage 2	80.00	2846.45	5.08
Synch. Timing - *synchronize* (t_5^{Rx})	55.42	4108.52	7.34
Stage 3	55.42	4108.52	7.34
Synch. Timing - *extract* (t_6^{Rx})	281.83	807.97	1.44
Multiplier AGC - *imultiply* (t_7^{Rx})	685.51	332.18	0.59
Synch. Frame - *synchronize* (t_8^{Rx})	159.41	1428.51	2.55
Stage 4	88.65	2568.66	4.58
Scrambler Symbol - *descramble* (t_9^{Rx})	1682.89	135.31	0.24
Synch. Freq. Fine L&R - *synchronize* (t_{10}^{Rx})	1246.85	182.63	0.33
Synch. Freq. Fine P/F - *synchronize* (t_{11}^{Rx})	112.56	2022.98	3.61
Stage 5	97.27	2340.92	4.18
Framer PLH - *remove* (t_{12}^{Rx})	1008.60	225.77	0.40
Noise Estimator - *estimate* (t_{13}^{Rx})	550.06	413.98	0.74
Stage 6	355.94	639.75	1.14
Modem PSK - *demodulate* (t_{14}^{Rx})	40.47	5626.34	10.05
Interleaver - *deinterleave* (t_{15}^{Rx})	1347.25	169.02	0.30
Decoder LDPC - *decode SIHO* (t_{16}^{Rx})	164.21	1386.74	2.48
Decoder BCH - *decode HIHO* (t_{17}^{Rx})	6.92	32905.37	58.79
Scrambler Binary - *descramble* (t_{18}^{Rx})	91.11	2499.41	4.47
Stage 7	5.35	42586.88	76.09
Sink Binary File - *send* (t_{19}^{Rx})	1838.31	123.87	0.22
Stage 8	1838.31	123.87	0.22
Total	4.09	55742.37	99.57

2.2.3 AFF3CT: Next Steps and ISTC 2027 Challenge

AFF3CT is a powerful tool for performing efficient and optimized Monte Carlo simulations. It effectively leverages multicore architectures and SIMD (Single Instruction Multiple Data) instructions. The main building blocks of the baseband communication chain, such as source, channel coding, modulation, multiple access [5, 24], demodulation, channel decoding, and error monitoring, are all available in AFF3CT. The software suite supports a large number of codes, modulations, and channels, accompanied by extensive and detailed documentation[6]. Continuous integration scripts are used so that with each new software evolution, a complete database of BER and FER curves is replayed to avoid any regression in the implemented modules.

AFF3CT's channel decoders are designed to be state-of-the-art or close to it for polar codes, turbo codes, and LDPC codes. A "hall of fame" is available on the website for each code type, comparing the performance of implementations available in AFF3CT with those reported in the

[6] `https://aff3ct.readthedocs.io`

literature. This allows researchers and engineers to easily compare their results with the best-known performances, thus facilitating innovation and continuous improvement of decoding algorithms.

Following these initial developments, during my ATER year, I participated in transforming AFF3CT to adapt it to an SDR implementation context using USRP modules. This transformation made AFF3CT compatible with software-defined radio systems, thus paving the way for practical and real-world applications in digital communications. The main results of this transformation include a significant improvement in system flexibility and portability, allowing easy integration with various hardware equipment [20].

In order to promote the use of AFF3CT and to encourage contributions from the research community, we are planning with my colleagues from Sorbonne Université and the University of Bordeaux to organize a challenge for the ISTC 2027 conference. This challenge will focus on optimizing the performance of channel decoders implemented in AFF3CT. The advantage over last competition is a much more flexible and modular software architecture, which allows for more reproducible and reusable contributions. The challenge will be open to all researchers and engineers interested in channel coding and decoding, with a particular emphasis on the usage of parallel computing and SIMD optimizations.

2.3 ASIP Architectures for Polar Codes

Following the work on software implementations of polar code decoding on general-purpose processors (GPP), two contributions explored performance optimization through specialized programmable processors (ASIP - Application Specific Instruction-set Processor). This paradigm shift is in line with the Cloud-RAN logic, where the virtualization of physical functions allows great flexibility in resource allocation. Nevertheless, to achieve high throughput and energy efficiency while maintaining a certain level of programmability, the use of ASIPs proves particularly relevant. Unlike GPPs, which are often oversized and energy-intensive, ASIPs allow for fine-tuning of hardware resources to the target algorithm, while integrating vector processing units or specific instructions to improve efficiency.

2.3.1 Tensilica

The first work relied on the Cadence Tensilica ecosystem to design a processor based on the Xtensa LX7 core, adapted for SC and SCAN (Soft CANcellation) polar code decoding [21]. This processor was configured to include a VLIW pipeline, SIMD instructions on 512-bit registers, and a reduced set of general-purpose registers optimized for LLR and partial sum access. The approach combined architectural optimizations (custom instructions, adapted memory management, quantized arithmetic) and software optimizations (pseudo-unrolling of decoding, use of specialized functions like R0/R1/REP/SPC). The results showed that throughput was comparable to that obtained on an ARM Cortex-A57, with a 10-fold reduction in energy consumption, thus demonstrating the relevance of the approach for embedded applications or those deployed on Cloud-RAN servers requiring strong energy efficiency constraints.

2.3.2 Transport Triggered Architecture

The second article proposes a significant evolution of ASIP-based polar decoding through the use of a *Transport Triggered Architecture* (TTA), developed in collaboration with the University of Tampere, which also motivated the project visit with Pekka Jääskeläinen via Maupertuis funding [22]. The Transport Triggered Architecture (TTA) is a modular processor architecture template [48], inspired by the MOVE architecture [50]. To facilitate the design and implementation of TTA-based systems, a dedicated hardware/software development toolchain, known as the TTA Co-Design Environment (TCE), has been developed [49].

A TTA consists of elementary components, called function units (FUs), interconnected through a configurable number of buses. A Base TTA comprises a control unit (GCU) handling fetch and decode operations, an arithmetic logic unit (ALU), a load/store unit (LSU), and a register file (RF). The modularity of the architecture allows designers to add custom FUs and to expand the interconnection network with additional buses. Unlike conventional processor architectures, a TTA does not rely on a fixed instruction set. Instead, computation is triggered by data movements: writing data to a particular input port, known as the *trigger port*, initiates the corresponding operation within the FU. The executed "instruction" is thus a side effect of the data transfer. The architecture inherently supports parallelism, allowing multiple data transfers to occur simultaneously. Consequently, programming a TTA entails specifying, for each clock cycle, the source and destination of all concurrent data movements, effectively exposing the processor's datapath to the programmer.

While this exposed datapath provides flexibility, manual programming at the assembly level would be extremely tedious. TCE addresses this challenge by providing a retargetable compiler

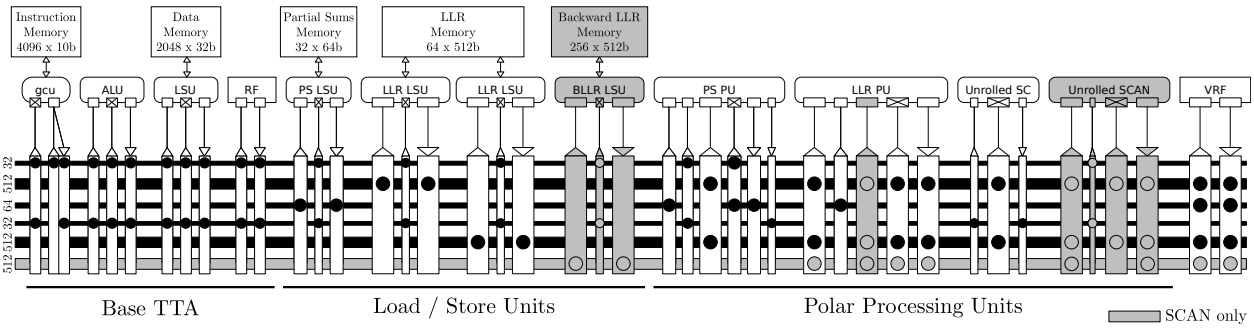


Figure 2.18: Transport Triggered Polar Decoder Architecture.

Table 2.10: Comparison of programmable processors running the SC decoding algorithm for R=0.5 polar codes

Architecture	N	Latency [μs]	Throughput [Mb/s]	E_b [nJ/bit]
i7-3.3GHz (GPP)	1024	2.0	257	41
	512	1.2	210	49
	256	0.7	179	59
	128	0.4	143	73
A57-1.1GHz (GPP)	1024	10.7	48	17
	512	5.3	48	17
	256	2.8	46	17
	128	1.6	41	20
LX7-835MHz (ASIP)	1024	7.2	71	1.6
	512	3.9	66	1.7
	256	1.9	65	1.7
	128	1.0	62	1.8
TTPD-800MHz (ASIP)	1024	1.4	352	0.14
	512	0.8	313	0.15
	256	0.4	304	0.16
	128	0.2	284	0.17

that, given an architecture description, translates high-level code (C, C++, OpenCL) into executable TTA programs [49]. Program efficiency is enhanced by leveraging the modular LLVM infrastructure [51], which handles frontend and middle-end optimizations, while TTA-specific low-level optimizations are applied through a custom code generation library integrated into TCE. A fully customized TTA processor, including its bespoke function units, can be automatically generated as either generic or optimized RTL (VHDL or Verilog). The resulting processor can then be directly synthesized and implemented on ASIC or FPGA platforms. Thanks to their modularity and flexibility, TTA-based designs have demonstrated elegant and efficient solutions across a wide range of applications.

2.3.2.1 Transport Triggered Polar Decoders

The proposed architecture, illustrated in Figure 2.18, builds upon a Base TTA and extends it with vector custom function units (FUs) to enable highly parallel processing. A vector size of $P = 64$ was selected, allowing the FUs to process up to 64 data elements (LLRs or PSs) simultaneously. LLRs are represented as $Q = 8$-bit values, ensuring negligible decoding performance loss. Core components of the architecture include parallel Load/Store Units (LSUs) for efficient memory access, Polar Processing Units (PPUs) that implement the specific f and g polar functions, and an

unrolled SC decoder designed to accelerate the most sequential parts of polar decoding. A Vector Register File is included to buffer vector data and facilitate smooth data transfers between FUs.

The LSUs are tailored for high throughput, supporting the maximum polar code length of $N_{\max} = 1024$ as defined in the 5G standard [39]. PS memory is organized as 32x64 bits to allow parallel access to $P = 64$ data, while LLR memory is mapped into a 64x512-bit RAM. The LSUs are optimized to reduce load/store latency from three to one clock cycle by removing internal registers, and they also support unaligned memory accesses without affecting the critical path, which remains in the 32-bit scalar ALU.

The PPUs, comprising the PS PU and LLR PU, act as SIMD units specialized for polar decoding, allowing multiple f and g operations to be performed in parallel. To further accelerate computation, unrolled subtree decoders are integrated as FUs for the lower stages of the decoding tree. These unrolled SC decoders can process 8 LLRs in 6 clock cycles, using a multi-cycle timing strategy instead of additional registers. This approach removes unnecessary hardware, keeps the critical path in the general-purpose ALU, and maintains full flexibility for varying frozen bit patterns. Overall, the architecture leverages vectorized FUs, efficient memory access, and unrolled subtree processing to achieve high-performance polar decoding while preserving the programmability and modularity of the Base TTA.

2.3.2.2 Results

A fine-grain comparison of polar decoders is challenging due to differences in technology, algorithms, pruning strategies, quantization, and frozen bit locations. Nevertheless, the Transport Triggered Polar Decoder (TT-SC), customized for SC decoding, demonstrates that TTA provides a natural compromise between programmable and dedicated architectures. Compared to previous work, the proposed TT-SC processor achieves significantly higher throughput and energy efficiency.

The TT-SC decoder was implemented using a 28 nm FD-SOI library, with memories realized in SRAM blocks. For a (1024, 512) polar code, it achieves a throughput of 352 Mb/s, which is 37% higher than an Intel i7 CPU implementation, while consuming only 0.14 nJ/bit, two orders of magnitude lower than the general-purpose solution. Although more clock cycles are required than in dedicated architectures due to pipeline registration and more complete tree pruning, the TT-SC maintains comparable logic utilization and competitive hardware complexity while preserving programmability. Overall, the TT-SC processor illustrates that a TTA-based approach can deliver high-performance polar decoding while retaining flexibility.

Table 2.11: ASIC implementations of dedicated SC decoders for a (1024,512) polar code

	Proposed TTPD	[47]	[52]	[52][2]
Target	28nm	28nm	180nm	28nm
Clock cycles	1161	1833	1568	1568
IT/P [Mb/s]	352	94	49	436
Freq [MHz]	800	336	150	1335
Power [mW]	48	18	67	5
E_b [nJ/bit]	0.14	0.19	1.4	0.011
Area [mm^2]	0.16	0.44	1.71	0.04
Pruning	R0 & R1	First R0	None	None

[2] Scaling factors from 180nm to 28nm of [52] are taken from [47].

Chapter 3

Deep Learning on Embedded Targets

My work in deep learning, which began with my arrival at IMT Atlantique in 2020, has been shaped by a focus on optimizing models for resource-constrained environments, drawing parallels with my previous experience in error-correcting codes. This research has been structured around three main axes: federated learning, algorithmic compression, and hardware implementation on FPGAs.

The study on federated learning in Section 3.1 was initiated through an IMT "Futur et Ruptures" project, a collaboration between IMT Atlantique and Telecom Saint-Étienne and Mines Saint-Étienne. The partnership was a natural fit, combining the expertise in federated learning from Saint-Étienne with our skills in embedded systems at IMT Atlantique. This project allowed us to address the classic challenges of federated learning—privacy and communication overhead—from the perspective of algorithmic complexity and communication cost. Our goal was to explore the feasibility of deploying federated learning on low-cost embedded targets, such as embedded GPUs or FPGAs.

Then a whole line of work concerns Deep Learning Compression, detailed in Section 3.2. My approach to deep learning mirrors the lessons learned from working with error-correcting codes, emphasizing the principle of achieving more with less. This led to a focus on algorithmic compression, specifically through methodological contributions to pruning and the exploration of neural network architecture search spaces. This work included an industrial project with GoodFloow, addressing the crucial trade-off between model accuracy and memory footprint for microcontrollers. We also developed highly optimized CPU implementations using SIMD instructions in collaboration with the company Deeplite.

The third major axis, detailed in Section 3.3, focuses on the implementation of deep learning systems on FPGAs. This work was notably advanced through the PEFSL project, which began as a student initiative and won the 2023 OpenHardware AMD Xilinx competition. Hugo Le Blevec's doctoral thesis further enabled us to achieve state-of-the-art results for semantic segmentation networks on FPGAs. A key insight from this thesis is that, contrary to much of the literature which emphasizes complex and exotic network architectures, the training routine and data preprocessing often have a greater impact on performance than the network's complexity or the specific operations employed.

3.1 Low-Cost Federated Learning

3.1.1 Federated Learning Overview

Data privacy has become a critical concern in domains like health, marketing, and machine learning in general. Centralized data collection can expose sensitive information to interception or unauthorized access. Federated learning (FL) [53] addresses these concerns by enabling machine learning models to be trained on decentralized data stored across multiple participant devices, called **clients**, without sharing raw data. The main objective is to mitigate privacy violations while leveraging distributed data.

In a typical FL setup, a central server orchestrates training by selecting clients, sending them the current model, and aggregating their local updates. This process is repeated in multiple **communication rounds**. The aggregation fuses knowledge from all participating clients into a global model.

Federated learning can be characterized along several dimensions [54]:

- **Orchestration:** Centralized (server coordinates updates) or decentralized (peer-to-peer).
- **Synchronization:** Synchronous (waits for all clients) or asynchronous (updates as received).
- **Client type:** Cross-device (many mobile/edge clients, limited resources) or cross-silo (few institutions, reliable and resourceful).
- **Client statefulness:** Stateful (retains inter-round information) or stateless (no retained info).
- **Data distribution:** IID (similar across clients) or non-IID (highly heterogeneous). Non-IID distributions create challenges like client drift.
- **Data partition:** Horizontal (same features, different samples) or vertical (different features, same samples).

This work focuses on deep learning image classification using CNNs, with centralized orchestration, synchronous updates, stateless cross-device clients, IID and non-IID data, and horizontal partitions.

3.1.1.1 Training Loop and Federated Averaging

A federated learning round consists of multiple coordinated steps involving the server and a set of participating clients. The overall loop is illustrated in Figure 3.1. The key steps are as follows:

1. **Client Sampling:** At the start of each round t, the server selects a subset $M \subset U$ of clients from the total pool U to participate. This sampling is critical for scalability, especially in cross-device settings with thousands or millions of potential clients, and also reduces the likelihood of stragglers slowing down the round.
2. **Model Broadcast:** The server sends the current global model $\mathcal{W}^{(t)}$ to each selected client. This model can include weights, biases, or other relevant parameters depending on the architecture.

3. **Local Training:** Each client $m \in M$ trains the received model locally on its private dataset d_m. The objective is to minimize the client-specific loss function:

$$\mathcal{L}_m(\mathcal{W}_m) = \mathbb{E}_{(x,y)\sim d_m}[\mathcal{L}(x, y, \mathcal{W}_m)]. \tag{3.1}$$

Clients perform training over multiple local epochs with a specified batch size. This local training allows each client to adapt the global model to its specific data distribution while keeping raw data private.

4. **Update Upload:** After local training, clients compute their model updates, which can be the updated weights $\Delta\mathcal{W}_m$, gradients, or other proxy information. These updates are sent back to the server, not the raw data. This ensures that sensitive information remains local.

5. **Aggregation:** The server aggregates all client updates to produce the next global model. The most widely used method is **Federated Averaging (FedAvg)** [55], which computes a weighted average based on dataset sizes:

$$\mathcal{W}^{(t+1)} = \sum_{m=1}^{M} \frac{|d_m|}{|D|} \Delta\mathcal{W}_m, \tag{3.2}$$

where $|D| = \sum_{m=1}^{M} |d_m|$. This aggregation fuses knowledge from heterogeneous client distributions into a coherent global model.

6. **Iteration:** The updated global model is then used in the next round. This process repeats for R rounds until the global model converges to a desired performance level.

$$\min_{\mathcal{W}} V(\mathcal{W}) = \sum_{m=1}^{M} \frac{|d_m|}{|D|} \mathcal{L}_m(\mathcal{W}_m) \tag{3.3}$$

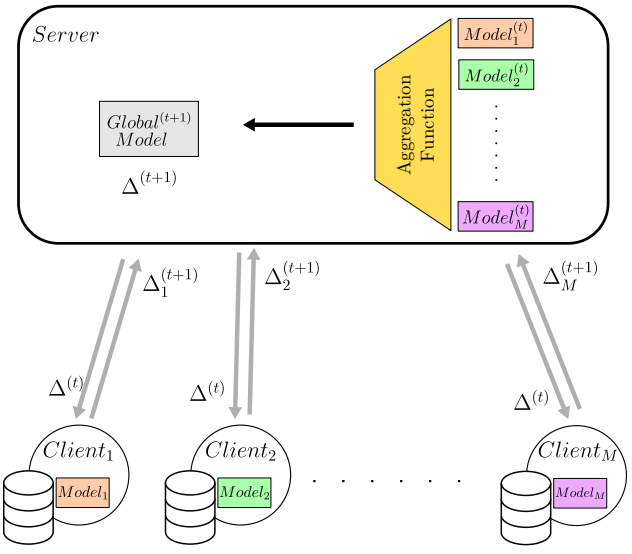


Figure 3.1: Training loop for one round of federated learning.

Experiments with 20 clients training ResNet-20 on CIFAR-10 over 100 rounds demonstrate that IID distributions achieve $\sim 92\%$ accuracy, whereas non-IID achieve $\sim 81\%$.

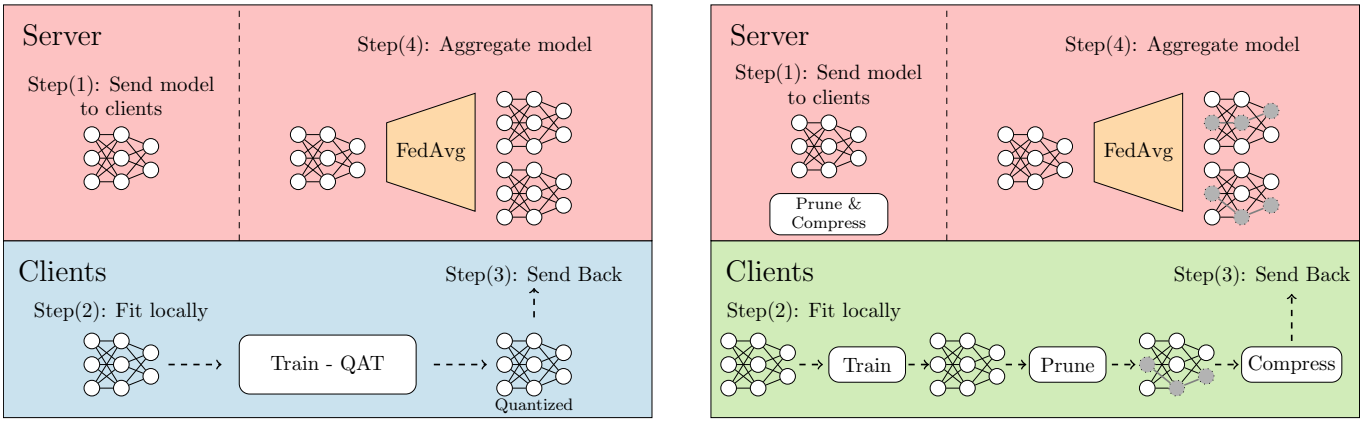


Figure 3.2: The pipeline of our study. Quantization is applied on the client side (left), while pruning is applied on both clients and server (right).

3.1.1.2 Challenges in Cross-Device Federated Learning

Cross-device FL introduces challenges in computation and communication. Devices often have limited processing power, memory, and energy, making deep learning training resource-intensive. Frequent model exchanges increase network load and energy consumption. Reducing communication rounds improves convergence and mitigates straggling clients but may increase local computation.

A study [56] evaluated energy costs for CIFAR-10 and ImageNet using ResNet-18:

- **CIFAR-10:** Centralized training consumed 2.7 Wh; federated training across 500 clients totaled 30.4 Wh (3.04 Wh per client). Communication accounted for 96% of energy.
- **ImageNet:** Centralized 971 Wh; federated across 100 clients totaled 2697.5 Wh (269.5 Wh per client). Communication was only 0.7% due to larger dataset size.

This highlights that dataset size and task complexity significantly affect communication costs in federated learning. Model compression and convergence optimization are potential strategies to reduce energy and communication overhead.

3.1.2 A first contribution: Pruning messages

In a first work, we investigate how model compression through quantization and pruning can reduce communication costs in Federated Learning while maintaining high accuracy [16]. Our experimental pipeline, illustrated in Fig. 3.2, applies quantization-aware training on clients and pruning on both clients and server to ensure coherent aggregation. This setup allows us to study different compression levels and local training epochs, measuring their effects on message size and accuracy under non-IID data distributions.

Compared to uncompressed training or approaches like ZeroFL, our compression techniques provide substantial reductions in communication overhead—up to 32× with 1-bit quantization—without a significant loss in accuracy. Table 3.1 summarizes results for CIFAR-10, showing that moderate pruning (30-60%) achieves 2-4× bandwidth savings while maintaining accuracy close to baseline FP32 models. In contrast, ZeroFL methods often require complex optimization or introduce higher training instability. Our approach is straightforward, compatible with standard aggregation methods, and provides a practical trade-off between communication efficiency and computational cost on embedded devices.

Table 3.1: Summary of message size and accuracy for the CIFAR-10 dataset

Compression Technique	**Accuracy (%)**		**Message Size (MiB)**
	1 Local Epoch	10 Local Epochs	
Baseline	78.94	78.18	2.97
Pruning			
10 %	74.79	78.18	2.57
20 %	76.01	78.12	2.34
30 %	78.20	77.83	2.10
40 %	77.50	77.81	1.85
50 %	72.74	77.65	1.57
60 %	76.00	77.65	1.29
70 %	73.43	78.11	1.01
80 %	75.18	77.89	0.70
90 %	73.37	76.63	0.37
95 %	71.05	74.81	0.19
99 %	52.77	66.82	0.04
Quantization			
8-bits	78.80	78.58	0.75
4-bits	79.74	77.04	0.38
1-bit	48.93	70.89	0.10

3.1.3 FLOCORA: Federated Learning Compression with LORA

Low-Rank Adaptation (LoRA) A recent approach to reduce training and compression costs relies on *Low-Rank Adaptation* (LoRA). The main idea is to avoid updating all parameters of a neural network during training, and instead restrict updates to *low-rank matrices* that approximate the necessary parameter changes.

Consider a weight matrix of a layer:

$$W \in \mathbb{R}^{d_1 \times d_2}.$$

In standard training, this matrix is updated directly via gradient descent. In LoRA, however, the matrix W is *frozen*, and the update is represented as a low-rank decomposition:

$$W^* = W + \frac{\alpha}{r} BA,$$

where

$$A \in \mathbb{R}^{d_1 \times r}, \quad B \in \mathbb{R}^{r \times d_2},$$

are trainable matrices with rank at most r, and α is a scaling factor. Since $r \ll \min(d_1, d_2)$, the number of additional parameters to train is reduced.

This approach was originally motivated by *transfer learning* on very large models, in particular *large language models* (LLMs) with hundreds of billions of parameters. Rather than fine-tuning the entire model, LoRA allows adapting a pretrained backbone to new tasks with a fraction of the memory and compute cost. An additional advantage is that the low-rank updates can be merged back into the original weights without adding inference latency.

In summary, LoRA addresses several key challenges:

- **Training efficiency**: only the low-rank matrices are optimized, reducing memory and computation requirements.

- **Parameter efficiency**: the pretrained backbone can be reused, while task-specific information is stored in lightweight adapters.
- **Communication efficiency**: since the backbone remains frozen, only low-rank updates need to be transmitted, which is especially relevant in distributed or federated settings.

Although originally developed for *natural language processing (NLP)* and *foundation models*, LoRA has rapidly become a standard tool in efficient deep learning, and its extension to other domains (computer vision, multimodal learning, federated learning) is an active area of research.

Our Method: FLoCoRA In this work, we propose **FLoCoRA** (**F**ederated **L**earning **Co**mpression with **Lo**w-**R**ank **A**daptation), a method that integrates LoRA into the federated learning (FL) framework to reduce both communication and memory requirements.

The key idea of FLoCoRA is to *never update the original neural network weights* shared across clients at the start of training. Instead, only the LoRA adapter parameters, A and B, are trained, exchanged, and aggregated. Because the original weights remain frozen, communicating only the low-rank adapters is sufficient to represent each client's updates. This design allows FLoCoRA to be seamlessly integrated with any FL optimization algorithm.

Figure 3.3 illustrates the FLoCoRA training loop. All clients start with identical weights W_{initial}, which remain fixed throughout training. During a communication round, the server first sends the global LoRA adapter parameters $\bar{\Delta}_t L$ to the selected subset of clients. Each client then trains its LoRA adapters locally and computes the updates $\Delta^k_{t+1} L$, which are uploaded back to the server. The server aggregates the updates using a weighted average, as in FedAvg, to obtain the new global adapters $\bar{\Delta}_{t+1} L$ for the next round.

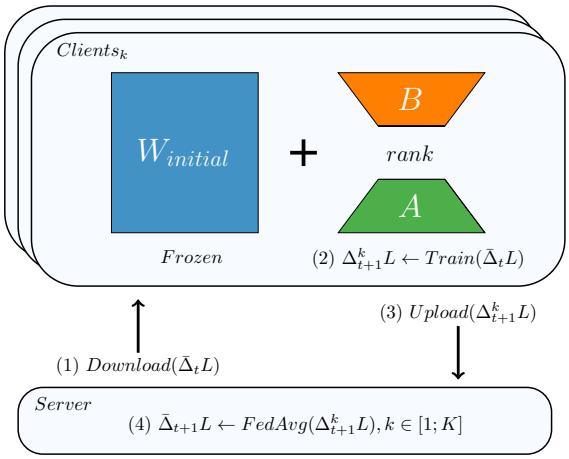


Figure 3.3: FLoCoRA training loop.

By training only the low-rank adapters, FLoCoRA reduces memory usage since fewer gradients need to be stored during backpropagation. Communication costs are lowered because only the adapter parameters are transmitted between clients and the server, and computational cost per round is reduced because the majority of the model weights remain frozen. FLoCoRA can also be combined with quantization techniques to further decrease communication overhead without significant loss in accuracy.

Experiments and Results We evaluated FLoCoRA on image classification tasks using small vision models in a federated learning setting. The basic setup consists of 100 clients, with 10%

sampled in each communication round. Models are trained for 100 rounds with a local batch size of 32, learning rate of 0.01, 5 local epochs, and momentum of 0.9. Stochastic Gradient Descent with momentum is used as the client optimizer, and FedAvg is used as the aggregation algorithm. All experiments are repeated three times with different seeds.

To determine which layers should be trained with LoRA adapters, we conducted an ablation study using a ResNet-8 model on the CIFAR-10 dataset, with LoRA hyperparameters $r = 32$ and $\alpha = 512$. The results are summarized in Table 3.2.

Table 3.2: Effect of training different layers with or without LoRA adapters on ResNet-8 for CIFAR-10.

Method	**Number of Parameters to Update**	**Accuracy (%)**
FedAvg	1.23 M	76.14 ± 0.74
FLoCoRA Vanilla	0.26 M	22.14 ± 3.99
+ Normalization Layers	0.26 M	39.80 ± 12.05
+ Final FC Layer	0.26 M	75.51 ± 1.34

From Table 3.2, several conclusions can be drawn. Freezing all model weights and training only LoRA adapters for convolution layers ("FLoCoRA Vanilla") results in very low accuracy. Including normalization layers improves performance but remains insufficient. Training the final fully connected layer in addition to LoRA adapters and normalization layers recovers almost all the accuracy of FedAvg, demonstrating that some layers (especially highly specialized or normalization layers) cannot be fully approximated by low-rank adapters. This informs the FLoCoRA configuration used in all subsequent experiments.

We compared FLoCoRA, including quantized versions, with standard compression techniques such as ZeroFL [57] and magnitude pruning [16]. Experiments used a ResNet-18 model, 100 clients, and 700 communication rounds. Results are summarized in Table 3.3.

Table 3.3: Comparing FLoCoRA and its quantized versions with ZeroFL and magnitude pruning. Communication cost is measured in total message size and TCC.

Method	**Configuration**	**Message Size (MB)**	**TCC (GB)**	**Accuracy (%)**
FedAvg	Full Model	44.7 (÷1.0)	62.6	84.43 ± 0.36
ZeroFL [57]	90% SP + 0.2 MR	27.3 (÷1.6)	38.2	81.04 ± 0.28
	90% SP + 0.0 MR	10.1 (÷4.4)	14.1	73.87 ± 0.50
Magnitude Pruning [16]	40% prune	27.1 (÷1.6)	38.0	85.20 ± 0.20
	80% prune	9.8 (÷4.6)	13.7	80.70 ± 0.24
FLoCoRA	r=64	9.2 (÷4.9)	12.9	85.17 ± 0.44
	r=32	4.6 (÷9.7)	6.5	83.90 ± 0.20
	r=16	2.4 (÷18.6)	3.3	82.33 ± 0.35
	r=64, Q=8	2.4 (÷18.6)	3.3	85.24 ± 0.23
	r=32, Q=8	1.2 (÷37.3)	1.7	83.95 ± 0.32
	r=16, Q=8	0.7 (÷63.9)	1.0	81.89 ± 1.01

Table 3.3 shows that FLoCoRA alone already achieves strong compression ratios with minimal loss of accuracy. For instance, using $r = 16$ reduces the communication cost by 18.6 times with only a 2% drop in accuracy. Combining FLoCoRA with quantization further reduces message sizes, achieving reductions up to 63.9 times while maintaining reasonable accuracy. In comparison, ZeroFL and magnitude pruning either achieve smaller compression ratios or suffer larger accuracy degradation. These results highlight the effectiveness of LoRA-based low-rank updates as a strong baseline for communication-efficient federated learning.

3.2 Compression by Pruning or Quantization and Embedded Deployment

Another set of contributions focused on neural network pruning and was carried out as part of Hugo Tessier's thesis, in collaboration with Matthieu Arzel and Vincent Gripon from IMT Atlantique, thanks to CIFRE funding from the company Stellantis, with Thomas Hannagan and David Bertrand as supervisors.

3.2.1 Pruning Neural Networks and Hardware Acceleration

Principle The basic idea of pruning is to reduce the cost of a neural network by removing part of its structure. Hugo has introduced in a popular blog post a comprehensive overview of this technique[1]. The intuitive principle of pruning naturally raises three questions:

- Which elements of the network can be removed, and what impact does this removal have on its cost?
- How can one identify the elements whose removal affects performance the least?
- What is the most effective way to remove these elements?

These questions respectively define three complementary dimensions of pruning: the pruning structure, the pruning criterion, and the pruning method.

Structures The pruning structure determines:

- which type of cost pruning will reduce;
- how easily pruning can be exploited in practice;
- the granularity of pruning, and thus the ability to prune significantly without degrading performance.

The most basic form of pruning is non-structured pruning, where individual weights are removed according to a given rule. This approach provides an excellent trade-off between performance and number of parameters, and is relatively simple to implement. However, it also presents two major drawbacks: it generates sparse tensors that are difficult to leverage efficiently on hardware, and it does not reduce tensor sizes, so intermediate representations remain unaffected.

To address these limitations, many approaches prune entire convolutional filters, a strategy usually referred to as structured pruning. Although this coarser granularity leads to a less favorable performance-to-parameters trade-off, it has two significant advantages: it actually reduces the size of weight tensors (making it compatible with most hardware and frameworks), and it decreases the size of intermediate representations, thereby lowering runtime memory requirements.

Criteria Once the pruning structure is chosen, the pruning criterion determines which elements are most relevant to remove. The guiding principle is to discard those elements that contribute the least to the accuracy of the model. Since direct evaluation of each parameter's importance is infeasible in large networks, surrogate metrics are employed. The two most common types are magnitude-based criteria (absolute value of weights) and gradient-based criteria.

[1] `https://towardsdatascience.com/neural-network-pruning-101-af816aaea61/`

When criteria are applied to groups of weights, two main tactics are used: either computing the norm of the metric across the group, or relying on proxies such as batch-normalization scaling factors, which are assumed to reflect the relative importance of upstream filters.

Another important dimension concerns how pruning is distributed across layers. Strategies include uniform pruning across all layers, manually defined layer-wise pruning rates, or a global target applied to the network as a whole. Moreover, since layers differ in size, the definition of the pruning rate may vary: some works consider the fraction of pruned filters, while others focus on the resulting number of remaining parameters.

Methods The pruning method specifies how elements selected by the criterion are actually removed. Two intertwined aspects are considered: (i) the removal strategy and (ii) the pruning schedule.

The removal strategy can be definitive (setting weights permanently to zero) or dynamic (allowing regrowth, reparameterization via masks, or continuous penalization). The schedule defines when pruning occurs relative to training: after training with alternating fine-tuning, progressively during training, or through re-training strategies with or without weight reinitialization.

Contributions Hugo proposed three contributions in the field of pruning.

- **Selective Weight Decay (SWD)**: a method that prunes weights continuously during training, using an increasing penalty until their values reach zero. SWD is compatible with any pruning structure and criterion, and its adaptive nature allows efficient pruning even at very high compression rates [2].
- **Dimensional Clear-Out (DCO)**: a framework addressing the structural inconsistencies induced by structured pruning. DCO identifies dependencies between layers and operations, removes disconnected weights, and adapts broken operations (e.g., mismatched tensor dimensions in residual connections), ensuring that inference remains possible under arbitrary structured pruning distributions [17].
- **Evaluation of structured pruning on GPU**: a study showing how cost metrics influence the perceived efficiency of pruning and highlighting the risks of imbalance in pruning criteria when deployed on hardware [18].

Selective Weight Decay (SWD) SWD aims at pruning as smoothly as possible by applying an additional $\mathcal{L}_2$ penalty on the subset $\mathbf{W}^*$ of weights to prune from $\mathbf{W}$. The standard optimization problem with weight decay can be written as:

$$\min_{\mathbf{W}} \mathcal{L}(\mathcal{N}(\mathbf{X}), \mathbf{y}) \quad \text{with} \quad \mathcal{L}(\mathcal{N}(\mathbf{X}), \mathbf{y}) = \mathcal{F}(\mathcal{N}(\mathbf{X}), \mathbf{y}) + \mu \left\|\mathbf{W}\right\|_2, \tag{3.4}$$

where $\mathcal{F}$ is the supervised training loss and μ the weight decay factor.

We extend this formulation by introducing Selective Weight Decay in Equation 3.5:

$$\min_{\mathbf{W}} \mathcal{L}(\mathcal{N}(\mathbf{X}), \mathbf{y}) \quad \text{with} \quad \mathcal{L}(\mathcal{N}(\mathbf{X}), \mathbf{y}) = \mathcal{F}(\mathcal{N}(\mathbf{X}), \mathbf{y}) + \mu \left\|\mathbf{W}\right\|_2 + a\mu \left\|\mathbf{W}^*\right\|_2. \tag{3.5}$$

This additional $\mathcal{L}_2$ penalty, weighted by the multiplier a, can be seen as a relaxation of an $\mathcal{L}_0$ constraint, progressively forcing the targeted weights toward zero. This principle is directly inspired by Lagrangian smoothing [61]. Figure 3.4 illustrates the landscape of the combined penalty of weight decay and SWD.

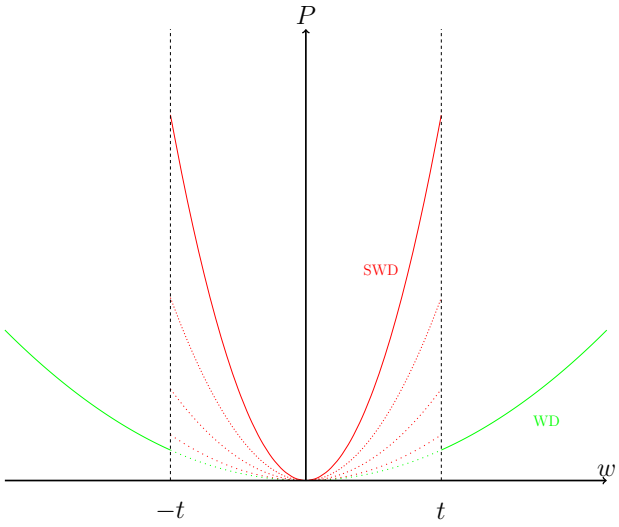


Figure 3.4: In the case of non-structured magnitude pruning, SWD behaves as such: all weights w of $\mathbf{W}$ are applied a penalty P, that combines weight decay and SWD. Since SWD is only applied to weights below a given threshold t, it is like P applies a stronger penalty on weights below t and a smaller one above t. Another way to present this is to consider SWD as a trap that progressively subjects weights below a certain threshold to an increasingly strong penalty, that drives them to zero, thus pruning them.

The subset $\mathbf{W}^*$ represents the weights to prune. Its definition depends on the chosen pruning structure and criterion, which are orthogonal to SWD itself, as SWD is only a removal method. A key property is that $\mathbf{W}^*$ can be updated dynamically during training. In practice, we chose to update $\mathbf{W}^*$ at each training step, although less frequent updates are theoretically possible.

The multiplier a controls the strength of the selective penalty. Its value must grow during training so that the targeted weights are eventually forced toward zero, while avoiding divergence of the optimization dynamics. We interpolate a between a starting value a_{min} and a final value a_{max} using an exponential schedule:

$$a(s) = a_{min} \left(\frac{a_{max}}{a_{min}} \right)^{\frac{s}{s_{final}}}, \tag{3.6}$$

with s the current training step and s_{final} the total number of pruning steps. Ablation studies on the influence of a have been conducted in our previous work [2], and we therefore do not reproduce them in detail here.

In this manuscript I focus on the most significant results which are on ImageNet, but further results on CIFAR10, City Scapes on various network architectures are available in Hugo's thesis [62]. CIFAR-10 allows finer-grained exploration: it shows extensive trade-off curves on two variants of ResNet-20, one over-parameterized and easy to prune (initial width of 64 channels) and one much more parsimonious and sensitive to pruning (initial width of 16 channels). Finally, Cora serves to evaluate SWD on a graph-based task, namely node classification. Indeed, a great advantage of SWD is that it applies easily on any network architecture, pruning method, pruning structure.

ImageNet provides large-scale evaluation, albeit at high computational cost. Table 3.4 reports top-1 accuracy for both non-structured and structured pruning. SWD consistently achieves the best performance, particularly under aggressive pruning. For example, at 80% non-structured sparsity, SWD reaches 72.4% top-1 accuracy, compared to 68.7% for the baseline and 70.1% for LRR; at

Experiments on ImageNet ILSVRC2012						
Non-structured pruning						
Target (%)	Han et al. [63]		+LRR [66]		SWD	
	Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
50	74.9	92.2	58.4	82.1	**75.0**	**92.2**
10	71.1	90.5	54.6	79.6	**73.1**	**91.3**
2.5	47.2	73.2	34.8	61.54	**67.8**	**88.4**
Structured pruning						
Target (%)	Liu et al. [64]		+LRR [66]		SWD (+LRR)	
	Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
90	**74.7**	**92.2**	56.1	80.7	74.2	91.9
75	73.4	91.6	51.1	77.1	**73.5**	**91.5**
50	63.6	85.7	40.0	66.2	**69.0**	**88.8**
20	0.1	0.5	0.1	0.5	**69.0**	**88.7**

Table 3.4: Non-structured and structured pruning on ResNet-50 with ImageNet ILSVRC2012. Comparison between a baseline method (Han *et al.* [63] or Liu *et al.* [64]), its LRR variant [65], and SWD.

90% sparsity, SWD maintains 68.2%, while the baseline drops to 60.5% and LRR to 63.0%. For structured pruning of filters, removing 50% of filters results in 75.1% accuracy for SWD, versus 73.2% for the baseline and 74.0% for LRR.

Two main tendencies are observed. First, SWD preserves substantial accuracy under high pruning rates where other methods fail. Second, the LRR variant [66] occasionally degrades the baseline's performance while improving SWD, a behavior not seen in other experiments. The cause of this discrepancy remains unclear, suggesting that SWD and the baseline produce pruned architectures with differing retraining dynamics, although this does not fully explain why pruned and fine-tuned networks outperform warm-restarted counterparts.

Dimensional Clear-Out (DCO) From an implementation perspective, it is clear that structured pruning techniques can be effectively applied to reduce the dimensionality of neural networks and therefore increase their throughput, decrease their latency, improve their power efficiency. Removing filters reduces the size of layers, which any framework can leverage. Even though it is perfectly true, there are some problems that we did not mention and that have a significant impact on how to perform structured pruning.

Filter pruning is complicated by interdependencies between layers. In a network, each filter produces a feature map, and the filters of subsequent layers must match these input dimensions. Pruning a filter therefore requires removing the corresponding kernels in downstream layers. This explains why filter pruning is often referred to as "channel pruning". Residual connections further complicate pruning, since tensors to be summed must have matching dimensions. In ResNet-like architectures, these dimensions depend on complex combinations of upstream layers, making it difficult to predict the exact size of intermediate representations.

Figure 3.5 illustrates typical issues: pruned filters alter output dimensions, requiring adjustments in subsequent layers. Residual additions may need to be replaced by concatenations to maintain correct tensor shapes. The input dimensions of layers, and thus the number of parameters, depend on the global pruning pattern. Ignoring these dependencies may cause runtime errors

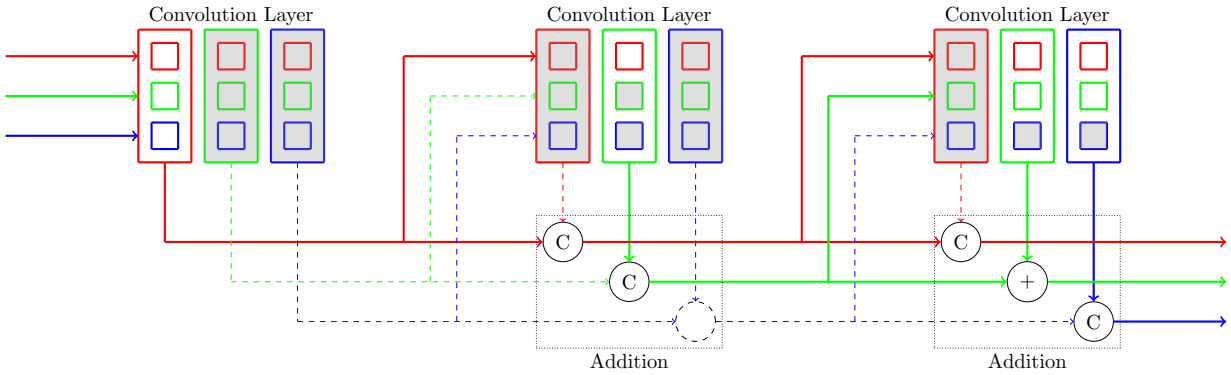


Figure 3.5: Example of problems introduced by pruning in the presence of residual connections.

or force pruning to be simulated by zeroing weights rather than actually removing them.

A related issue is *orphan biases*. When a filter is removed but its bias remains, it produces a constant output. This can prevent downstream kernels from being removed, reducing the effective compression and potentially altering performance if fine-tuning is not applied. Biases are often stored in batch-normalization layers, so pruning only the convolution layer may leave the bias intact. Furthermore, secondary removals (following layers' kernels) can be overlooked, leading to incomplete pruning.

Together, layer interdependencies and orphan biases make proper filter pruning subtle and challenging. In complex networks such as HRNet, these issues motivate the need for automatic, generic methods to identify dependencies and adjust operators, ensuring correct and effective pruning.

We developed a method to automatically identify all dependencies, remove disconnected parameters, and adapt operators so that unconstrained filter-pruned networks can be leveraged on embedded devices. This method is generic, tested on various network types, implemented in PyTorch, and freely available (`https://github.com/HugoTessier-lab/Neural-Network-Shrinking`). Pruned networks can also be converted to ONNX for multi-hardware deployment.

To reliably identify disconnected weights while avoiding issues such as orphan biases and complex non-linear interactions, we introduced *Dimensional Clear-Out* (DCO). DCO operates on a modified copy of the network in which biases and non-linearities are removed and weights replaced by pruning masks. In this linearized network, any weight with a zero gradient is structurally disconnected due to pruning, allowing identification of all pruned parameters with a single input.

Residual block additions still pose a challenge when tensors have different channel dimensions. To address this, we define an *indexation-addition* operation, which sums only the matching channels from two tensors using configurable index lists. Identity 1×1 convolutions are inserted before each addition to automatically generate these indices from the DCO masks. Figure 3.6 illustrates this crossbar trick.

In combination, DCO, indexation-addition, and automatic index generation enable any distribution of unconstrained filter pruning to be functional, guaranteeing consistent input/output dimensions and correct residual additions. This framework provides a practical way to study and deploy aggressively pruned networks on hardware.

The existence of dependencies between layers and the necessity of acknowledging them for a reliable measurement of the compression rate is something that is often left ambiguous in many papers. Blalock *et al.* [101] even cited this lack of consistency, in the metrics reported to characterize the compression of a pruned network, among a large proportion of papers in the field.

This is why, when presenting our work on DCO [17], we illustrated its purpose with a comparison

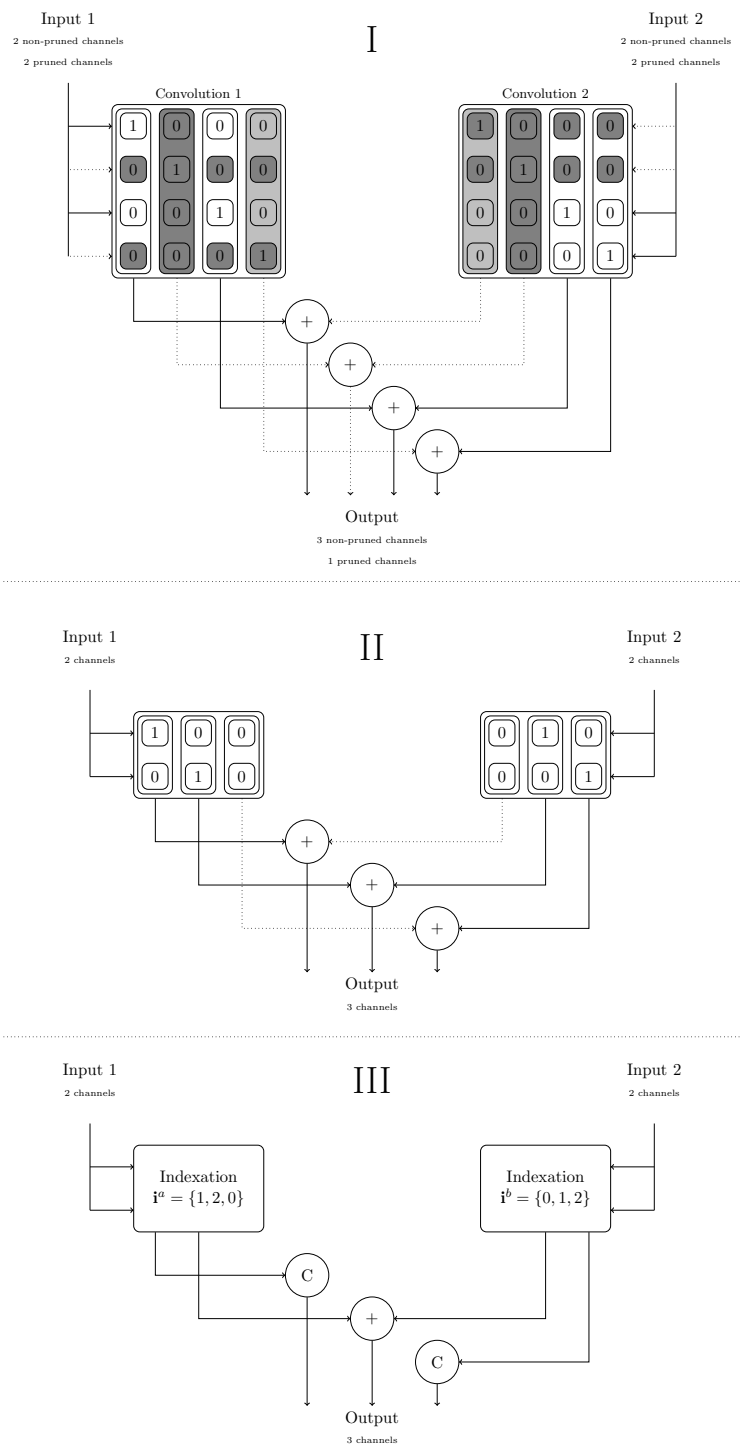


Figure 3.6: Creation of the indexation-addition operation. I) Identity convolutions are applied to each input. Grey-out parameters can be removed; the light-grey filters would be removed by DCO, but we keep them to for the next step (because the same filter in the other layer is not pruned). Dotted arrows represent the empty feature maps. II) The shrunken convolutions now are able to insert each of the two inputs into larger tensors that can be properly summed together. III) The indices $\mathbf{i}^a$ and $\mathbf{i}^b$ can be directly deduced from the reduced convolutions from step II.

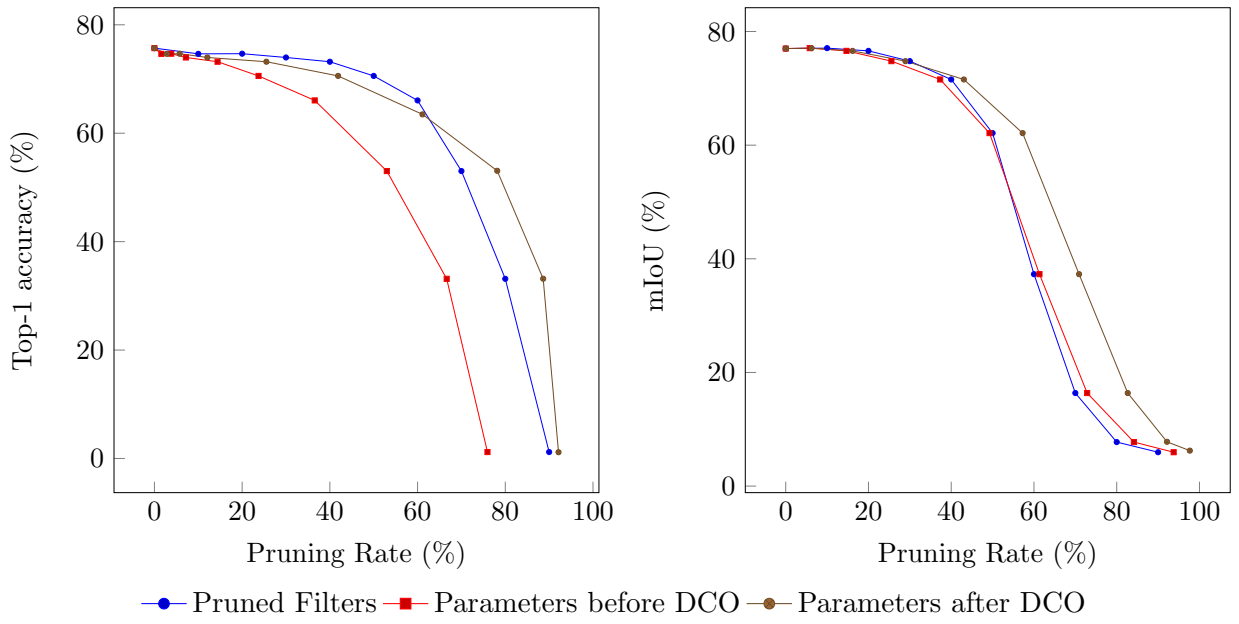


Figure 3.7: Trade-off between the pruning rate and the accuracy of ResNet-50 on ImageNet (left) or the mIoU of HRNet-48 on Cityscapes (right), depending on three different ways of estimating the pruning rate: 1) the proportion of pruned filters, 2) the remaining parameters after pruning the filters and 3) the remaining parameters once the pruned network is applied DCO.

of the effect of different ways of calculating the pruning rate on the apparent trade-off between accuracy and pruning rate. We compare three different calculation methods:

- A very naive method, that only counts the proportion of pruned filters.
- A more relevant method, that counts the remaining parameters after pruning, but without taking into account any dependencies between networks.
- The actual proportion of remaining parameters after DCO.

Figure 3.7 shows precisely this comparison. These experiments did not use LR-rewinding [66] and networks post-DCO were not fine-tuned, so some of them encountered a small drop in accuracy because of the perturbation of the distribution of data in intermediate representations, due to the removal of orphan biases. Also, the pruning target, for each pruning iteration, was each time defined according to the proportion of pruned filters.

This figure allows to make some interesting observations:

- For low pruning rate, the proportion of filters overestimates the actual count of parameters; but for high pruning rates it underestimates it.
- The relationship between the proportion of filters and the pre-DCO parameters count are completely different between ResNet and HRNet.
- The gap between before and after DCO is significant, but fairly less for HRNet than for ResNet.

Evaluation of structured pruning on GPU This contribution [18] focuses on a detailed analysis of the energy consumption of HRNet segmentation networks compressed by filter pruning using DCO, compared to smaller non-pruned networks (HRNet-18 and HRNet-32). The goal was to assess to what extent pruning effectively reduces energy consumption, and to identify the methodological and practical limitations of this approach.

The results highlight that energy consumption is primarily correlated with the number of operations rather than the number of parameters. Two pruned networks with the same number of parameters can exhibit significantly different energy profiles, since the distribution of sparsity directly influences the density of operations. Consequently, the pruning methods tested (Slimming [64], SWD [2], LR-Rewinding [66]) yield architectures that do not all lie on the same efficiency–energy trade-off curve, even when they achieve similar compression rates. This finding underlines the complexity of the relationship between pruning criteria, the distribution of retained weights, and the final energy efficiency.

Another key observation is that smaller non-pruned networks often appear more energy-efficient than pruned versions of HRNet-48 with a comparable number of parameters or operations. This is largely due to the overhead introduced by indexation-addition operations, and in particular by the *ScatterND* operator. Although conceptually simple, this operator is poorly optimized in TensorRT: it is treated as a "Foreign Node," executed by Myelin (cuBLAS) rather than cuDNN, leading to disproportionately high latency relative to its theoretical complexity. As a result, even when the number of convolution operations decreases, the overall latency of pruned networks remains higher than that of smaller compact models.

These results lead to several general conclusions. First, the benefits of pruning in terms of energy efficiency are not guaranteed: when compared to native lightweight architectures, compressing a large model does not necessarily bring an advantage. Second, the actual efficiency of a network depends not only on its theoretical architecture but also on the implementation of its operators within inference frameworks. An operator that seems optimal in theory may in practice become a bottleneck if not efficiently supported by the runtime library. Finally, the choice of pruning criterion and method plays a central role, as they determine not only the accuracy (mIoU) but also the distribution of sparsity, which directly affects the number of operations and, consequently, energy consumption.

In summary, this study emphasizes the need to jointly consider architecture, pruning criteria, and software implementation when assessing the energy relevance of compressed networks. It also opens up research perspectives, particularly on the design of pruning criteria better suited to energy optimization, and on the exploration of alternative implementations for critical operators such as *ScatterND*.

3.2.2 A practical compression case: GoodFloow

In the era of Industry 4.0, efficient management and tracking of industrial packages has become a key concern for many companies. GoodFloow, a company specializing in sustainable logistics solutions, focuses on replacing disposable cardboard packaging with Reusable Plastic Crates (RPCs) to reduce environmental impact and enhance operational efficiency. Ensuring the real-time monitoring of these packages is essential to prevent loss, damage, or theft, and to guarantee accountability throughout the supply chain.

Traditional methods for tracking packages, such as GPS, Time Difference of Arrival (TDoA), or Bluetooth Low Energy (BLE), suffer from limitations in industrial environments. GPS loses accuracy indoors, TDoA requires dense IoT deployments with high interference risk, and BLE-based solutions are constrained to short ranges and costly infrastructure. These limitations make

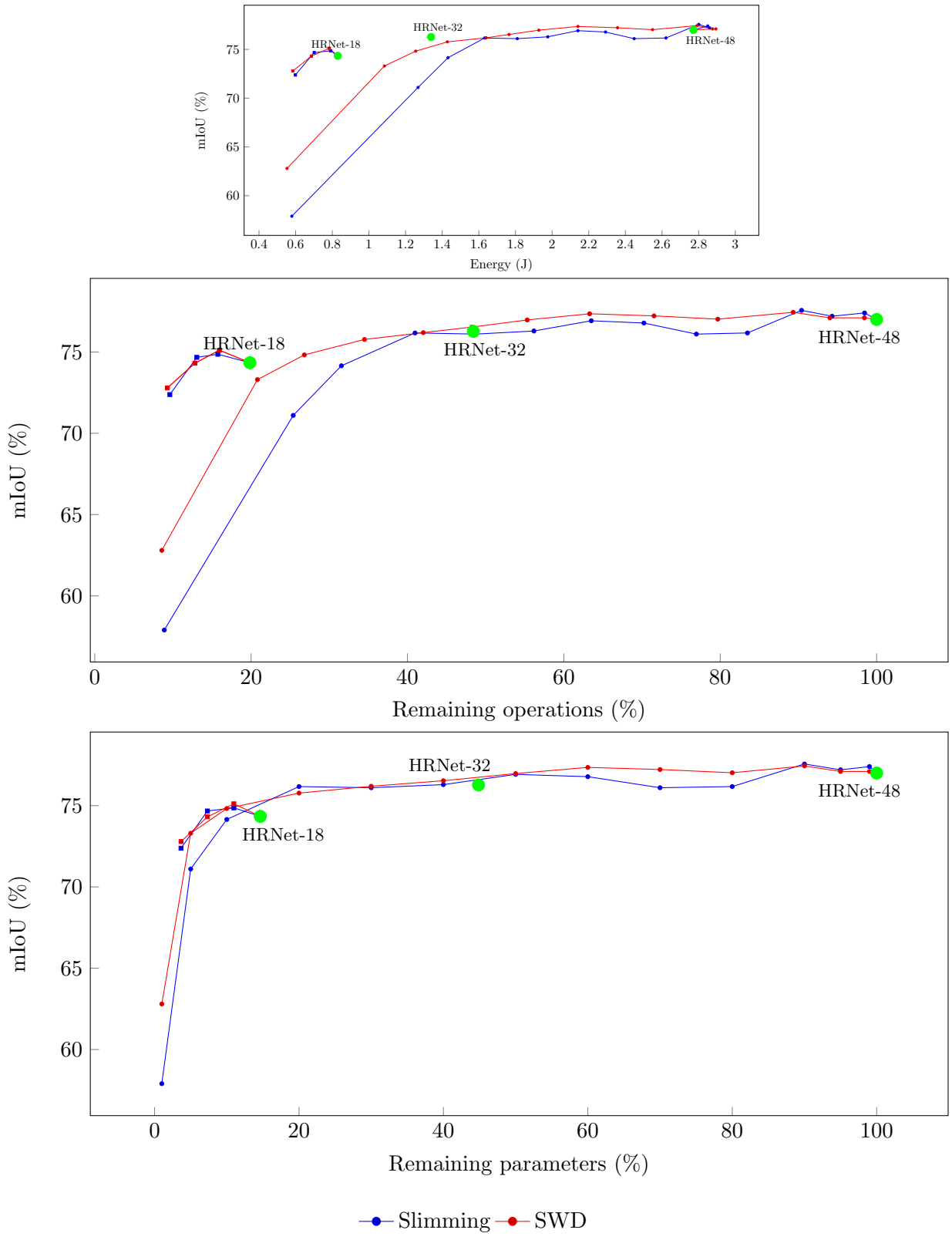


Figure 3.8: Comparison between accuracy (mIoU), energy consumption, number of operations, and number of parameters for pruned and non-pruned HRNet models. The tested pruning methods show very different behaviors depending on whether the comparison is made in terms of parameters or operations.

continuous, reliable monitoring difficult, particularly when packages move across multiple sites or logistics partners. Human-based monitoring is an alternative but is both time-consuming and expensive. These challenges highlight the need for IoT systems capable of detecting critical events in a package's lifecycle rather than continuously tracking its exact location.

To address this need, our study leverages accelerometer data combined with deep learning techniques to detect key events, such as a package being handled by a forklift or transported in a truck. By identifying state changes from sensor data, the system can selectively trigger communication or processing, optimizing energy consumption and extending the lifetime of battery-powered IoT devices. Deploying such models directly on constrained hardware introduces additional challenges due to limited memory and computational capabilities. Tiny Machine Learning (TinyML) provides a solution by enabling low-power, low-latency, and secure inference on embedded devices, eliminating the need for continuous data transfer to servers. TinyML has already shown promise in various domains, including agriculture, healthcare, and industrial automation, but its application to package monitoring remains largely unexplored.

The contributions of this work are twofold. First, we demonstrate that embedded deep learning can accurately detect package handling events in an industrial context using accelerometer signals. Second, we provide a methodology that balances high prediction precision and recall with low energy consumption, ensuring IoT devices can remain in sleep mode for extended periods while responding efficiently to relevant events. This methodology is not only applicable to the GoodFloow use case but can also be generalized to other embedded systems dealing with imbalanced time-series data.

Specifically, our approach includes three main aspects: data augmentation adapted to time-coherent, imbalanced datasets; the design of a convolutional neural network (CNN) with a threshold-based prediction mechanism to enhance robustness and reduce unnecessary processing; and efficient model compression to deploy the CNN on ESP32 microcontrollers commonly used in IoT devices. Experimental validation demonstrates that our system can reliably differentiate between key package events while maintaining low energy usage, illustrating the practical feasibility of embedded deep learning for industrial monitoring applications.

Thresholding soft decisions Battery lifespan is a critical concern in embedded devices, which have inherently limited energy capacity. In this context, defining a classification threshold for neural network predictions is a key strategy to optimize both performance and energy consumption. By selecting a threshold, the system can prioritize precision over recall, ensuring that predictions are highly reliable even in the presence of imbalanced datasets. Low-confidence predictions that do not surpass the threshold are automatically assigned to a "Dummy" class, which can subsequently trigger energy-efficient verification or correction methods.

The choice of the threshold is guided by the trade-off between precision and recall for each class, as shown in Figure 3.9. A higher threshold increases prediction precision, minimizing misclassifications that would otherwise require additional computational operations and thus consume more energy. Conversely, a lower threshold would classify more samples directly but at the cost of reduced confidence. For the use case considered here, it is crucial that predictions are correct when made, even if this allows some instances to fall into the "Dummy" class, as errors would otherwise lead to expensive reprocessing.

Threshold values are therefore determined based on model performance metrics, balancing the need for high precision with acceptable recall levels for each class. These thresholds can also be adapted depending on the configuration being tested or the class-specific characteristics of the dataset. By applying this thresholding approach, the embedded system is able to maintain high predictive reliability while reducing unnecessary computation, thereby extending the operational

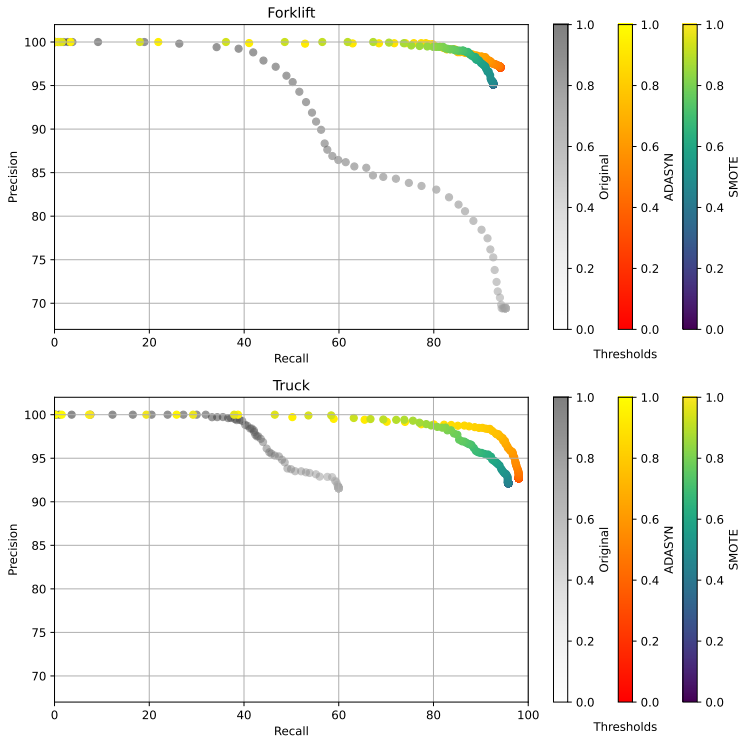


Figure 3.9: Precision-recall curves with data augmentation techniques. Different curves for each technique are generated by varying the classification thresholds.

lifetime of the IoT device.

Data augmentation for time series As mentioned earlier, we faced in this project the issue of imbalanced datasets that we tackled with data augmentation.

Data Augmentation for Imbalanced Time Series

Imbalanced datasets are common in time-series applications, as data collection often faces practical constraints. These imbalances can lead to misclassifications because the model has fewer examples from minority classes to learn from. A common solution is dataset re-sampling, which increases the representation of minority classes. In this study, we test two interpolation-based augmentation techniques: the Synthetic Minority Oversampling Technique (SMOTE) [99] and the ADAptive SYNthetic sampling method (ADASYN) [100].

For the SMOTE algorithm, we adapt it to preserve temporal continuity in accelerometer data. A new synthetic vector x_{new} is generated by randomly selecting a sample from the minority class, $x_{selected}$, identifying its nearest neighbors using Euclidean distance, and linearly interpolating with a randomly chosen neighbor $x_{neighbor}$ using a coefficient $\alpha \in [0, 1]$:

$$x_{new} = x_{selected} + \alpha \cdot (x_{neighbor} - x_{selected})$$

This process is repeated to generate the desired number of synthetic samples, ensuring that the augmented data maintain realistic temporal structure.

ADASYN differs from SMOTE by adaptively determining how many samples to generate for each minority instance, focusing on examples that are close to the decision boundary and harder

to classify. For each minority sample, a ratio r_i is computed based on the number of majority-class neighbors among the K nearest neighbors:

$$r_i = \frac{\Delta}{K}$$

This ratio is normalized to form a density distribution $\hat{r_i}$, and the number of synthetic samples g_i generated for each instance is given by:

$$g_i = \hat{r_i} \cdot G \quad \text{with} \quad G = (m_l - m_s) \cdot \beta$$

where m_l and m_s are the sizes of the majority and minority classes, respectively, and $\beta \in [0, 1]$ is a coefficient controlling the balance level. In our experiments, we use $K = 5$ nearest neighbors and set $\beta = 0.4$ for ADASYN. For fairness, the number of synthetic samples generated by SMOTE was matched to the total generated by ADASYN.

Comparing the two approaches, ADASYN demonstrates superior performance. This is because it concentrates synthetic samples near class boundaries, enhancing the model's ability to distinguish challenging cases, whereas SMOTE distributes samples uniformly across the feature space. When using the selected classification threshold, the model trained with ADASYN achieves a precision of 98.05% for Forklift and 95.75% for Truck events. Consequently, ADASYN was selected for subsequent implementations.

Figure 3.9 illustrates the effect of varying classification thresholds on precision and recall for each data augmentation technique. The curves clearly show that thresholding combined with ADASYN yields high precision while maintaining adequate recall, supporting both robust classification and energy-efficient embedded deployment.

Model compression and embedded deployment To deploy the CNN on resource-constrained IoT devices, we apply deep compression techniques combining weight pruning, quantization, and Huffman coding. Using the weight rewinding technique, the model is first trained with all its weights, and then after pruning, unpruned weights are fine-tuned to recover performance. A pruning ratio of 50% was selected as an optimal trade-off between model size reduction and accuracy [19]. Pruning is applied to convolutional and linear layers to reduce the non-volatile memory footprint, which is often a limiting factor in embedded environments. Huffman coding is then applied to further compress the TFLite model, while dynamic quantization reduces the model size to 14 kB, achieving a compression factor of over four compared to the original model. Importantly, this compression preserves high precision and recall for both Forklift and Truck events, as summarized in Table 3.5.

Configuration	**Compressed model size**	**Forklift**		**Truck**	
		Precision	Recall	Precision	Recall
Original	51 kB	98.05	91.17	95.75	96.62
Pruning	33 kB	93.61	96.12	95.72	93.85
Pruning + Quantization	14 kB	94.54	92.59	95.83	88.66

Table 3.5: Effect of deep compression on model size, precision, and recall for embedded deployment.

For deployment, the compressed and quantized TFLite model is converted into a C++ source file, which is then flashed onto the ESP32 microcontroller using the Espressif IoT Development Framework (ESP-IDF). Although the ESP32 module has 16 MB of flash memory, it is typically

shared with other modules such as WiFi and Bluetooth. The small size of the compressed model allows it to be fully loaded into SRAM, enabling faster access and minimizing memory latency during inference.

Reducing energy consumption is critical to extend the operational lifespan of the IoT device. During typical usage, the device wakes only when motion is detected and performs an inference to determine a state change. The pruned and quantized model consumes approximately 316 mW during inference, with an execution time of 27 ms, while the baseline power consumption of the device is 300 mW when idle. This incremental energy cost is minimal, ensuring that frequent inferences have a limited impact on battery life. The combination of efficient model compression and fast, selective inference allows the IoT device to operate sustainably over extended periods.

3.2.3 CNN acceleration with SIMD on ARM and x86 architectures

I worked through a research contract in 2021-22 with the canadian startup DeepLite, which specializes in optimizing deep learning models for edge devices. My role involved developing techniques to accelerate convolutional neural networks (CNNs) using Single Instruction Multiple Data (SIMD) instructions on ARM and x86 architectures. The core idea was to use SIMD registers as look-up tables (LUTs) to replace computationally intensive operations with simple memory accesses, thereby speeding up inference. This was targetting low-bit quantized models (e.g., INT2, INT4) where the number of possible values is small enough to fit in SIMD registers.

Deep learning models achieve high accuracy but often require a lot of memory and computing power, which makes it difficult to run them efficiently on regular CPUs, especially on small devices. One way to reduce memory and computation is to use ultra low-bit numbers, such as 2-bit values, for storing weights and activations. However, most CPUs are optimized for 8-bit or larger operations, so directly computing with sub-byte numbers is inefficient.

To overcome this limitation, we propose DeepGEMM, which replaces expensive multiplications with a lookup table (LUT) approach. All possible products of the 2-bit weights and activations are precomputed and stored in a small table. At inference time, the CPU simply retrieves the result from the LUT instead of performing a multiplication. This process is illustrated in Figure 3.10. The algorithm first packs multiple low-bit values into larger data types, then unpacks them to form indices for the table, and finally sums the retrieved values to compute the layer outputs.

Using this method, DeepGEMM achieves both faster computation and lower memory usage. On multiple convolutional neural networks, end-to-end inference shows speedups of up to 1.68× compared to optimized 8-bit kernels, as shown in Figure 3.11. While the unpacking step is currently the main bottleneck, future improvements such as better packing schemes and operator fusion are expected to further enhance performance. DeepGEMM is also flexible, supporting different number representations, including signed and non-uniform quantization.

3.2.4 Emerging technologies for deep learning

Concurrently, research focused on the use of emerging memristor-based technologies for the development of binary neural networks [3]. These "in-memory computing" approaches aim to overcome the data bottleneck by performing calculations directly within memory, offering significant gains in speed and energy efficiency for binary neural networks, which are particularly well-suited for these technologies.

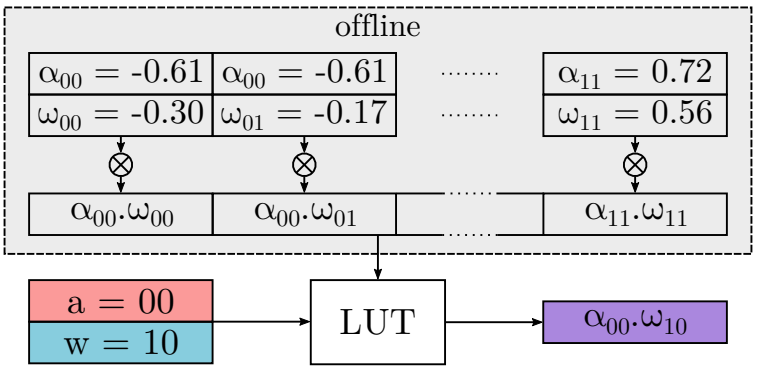


Figure 3.10: Lookup table accessed to retrieve precomputed products of ultra low-bit activation and weight values that are represented by the corresponding indices.

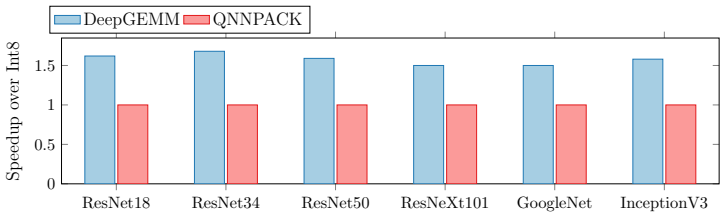


Figure 3.11: End-to-end speedups over QNNPACK INT8.

3.3 Deep Learning on FPGA Targets

3.3.1 Semantic Segmentation on FPGA for Autonomous Vehicles

Hugo Le Blevec's doctoral work focused on the co-design of deep neural networks and pipelined FPGA architectures for semantic segmentation in autonomous vehicles [15, 26]. These systems require low latency and reduced consumption, making classic GPUs less suitable for embedded contexts. FPGAs offer a promising alternative due to their parallel processing and energy efficiency, although they pose challenges related to limited resources. This work is not currently published but is under submission to ACM TRETS.

Over the past decade, the rapid progress of deep learning has been driven not only by algorithmic advances but also by the availability of large datasets and the computing platforms capable of handling them. Our work fits into this trajectory by focusing on the question of how to bring high-performance inference to environments where efficiency, latency, and energy consumption are critical. In this context, FPGAs emerged as a natural candidate: they offer a fine balance between flexibility and efficiency, while enabling custom hardware designs that can be adapted to the specific demands of neural network workloads.

Our research has been anchored in the study of representative datasets that reflect real-world application domains, ranging from image classification to more complex tasks requiring low-latency inference. The choice of these datasets was not incidental: they provided a benchmark against which we could measure the relevance of our contributions, both in terms of raw accuracy and in the broader perspective of system-level efficiency.

The central objective has been to establish new state-of-the-art performance for neural network inference on FPGA platforms. This ambition was made possible thanks to a close collaboration with the FINN team at AMD Dublin, whose toolchain offered both a foundation and a playground for innovative design strategies. Our long-term investment in the FINN ecosystem not only consol-

idated our methodological framework but also opened doors to industrial collaborations, such as with the company ARKANE, and to participation in larger initiatives like the VORTEX project. These partnerships have illustrated the dual impact of our work: academic recognition through the advancement of methods, and industrial relevance through real-world deployments.

Task and dataset The task addressed in this work is *semantic segmentation*, which consists in assigning a class label to every pixel of an image. Compared to classical image classification, where a single label is predicted for the entire image, semantic segmentation provides a fine-grained understanding of the scene and is therefore particularly relevant for autonomous driving, where distinguishing between road, vehicles, pedestrians, and other objects is essential for safety.

Our interest in this task originates from the work of Hugo Le Blevec during his PhD thesis at Stellantis, which first introduced us to the challenges of deploying deep learning for road scene understanding. While image classification was a natural entry point, semantic segmentation appeared both more challenging and more rewarding. It poses a significant implementation difficulty on FPGA devices, as models for semantic segmentation typically rely on non-standard network components (e.g., dilated convolutions, encoder-decoder structures, multi-scale features), and are considerably heavier than classification networks. Achieving competitive accuracy while still fitting the stringent resource and latency constraints of embedded FPGAs thus represents a major research challenge.

The performance of semantic segmentation models is generally measured with the mean Intersection over Union (mIoU). This metric evaluates, for each class, the ratio between the correctly predicted pixels (true positives) and the union of predicted and ground-truth pixels (true positives, false positives, and false negatives). Averaging this score across classes provides a robust measure that accounts for class imbalance and is widely adopted as the main benchmark in the field.

In this work, we focus on two widely used datasets for road scene segmentation:

- **CamVid** (Cambridge-driving Labeled Video Database) contains 701 images extracted from driving video sequences, at an original resolution of 960×720 pixels. The dataset defines 11 semantic classes. For efficiency, especially during inference, the images are often downsampled to 480×360 pixels, which is the resolution adopted in our experiments. While relatively small, CamVid provides a useful benchmark for comparing approaches and validating embedded implementations.

- **Cityscapes** is a much larger dataset with 5000 finely annotated urban street images, originally at 2048×1024 resolution. It defines 30 semantic classes, of which 19 are commonly used for evaluation. Due to its higher resolution and greater class diversity, Cityscapes represents a far more challenging benchmark than CamVid and has become the de facto standard for autonomous driving research. In practice, lower resolutions (e.g., 1024×512 or 512×256) are often used during inference to reduce memory and computation costs. In this work, we primarily focus on Cityscapes due to its difficulty and relevance, while still reporting results on CamVid for broader comparison.

In summary, semantic segmentation of road scenes is both a crucial and demanding task. It provides the right level of difficulty to study FPGA implementations of neural networks, forcing a careful balance between accuracy and hardware efficiency, and motivating the exploration carried out in this work.

Choosing and training a deep learning model for embedded semantic segmentation Several lightweight encoder–decoder architectures have been proposed for real-time semantic segmentation. Table 3.7 lists some of the most representative models and their reported results on Cityscapes, along with the improved results we obtained after retraining. A brief description of each model follows:

- **ENet** [67] (2016): one of the earliest real-time segmentation networks, with only 0.37M parameters. It employs a compact encoder–decoder design with bottleneck modules and dilated convolutions to balance accuracy and efficiency.
- **ESPNet** [68] (2018): designed for efficiency through a spatial pyramid of dilated convolutions, which allows capturing multi-scale context while keeping the parameter count low (0.4M).
- **DABNet** [69] (2019): uses depthwise separable and asymmetric convolutions inside bottleneck blocks, yielding a lightweight (0.76M parameters) yet accurate design.
- **EDANet** [70] (2019): an efficient dense architecture that combines dense connections with asymmetric convolutions, reaching good trade-offs at 0.68M parameters.
- **Fast-SCNN** [71] (2019): explicitly designed for embedded systems, it combines a shallow network for downsampling and a lightweight context path with depthwise separable convolutions (1.1M parameters).
- **ESNet** [72] (2019): builds on an efficient symmetric encoder–decoder structure with channel shuffle operations, leading to competitive results at 1.66M parameters.

Despite being the oldest of the group, ENet is by far the most compact model and was long considered obsolete due to its lower accuracy. However, as highlighted in the image classification domain by the "ResNet Strikes Back" paper [73], performance comparisons are often biased because training routines differ substantially between studies. When properly retrained with modern techniques, ENet can achieve competitive performance without increasing complexity.

We evaluated this hypothesis through an ablation study of training settings, summarized in Table 3.6. Starting from the original ENet (58.3% mIoU), we progressively introduced a longer training schedule, optimized hyperparameters, stronger data augmentation, and revised cropping and resizing strategies. Each factor contributes, but their combination is crucial: only with the full setup do we reach 70.5% mIoU on Cityscapes. This demonstrates that the training pipeline can have a larger impact on performance than the choice of architecture itself.

Table 3.6: Influence of the number of epochs (NE), training parameters (TP), data augmentation (AUG) and cropping/resizing parameters (CR) of the new training procedure on the performance of ENet on Cityscapes.

Procedure	**Cityscapes mIoU**
ENet	58.3%
ENet + NE + TP + AUG + CR	**70.5%**
- NE	65.1%
- TP	59.1%
- AUG	64.6%
- CR	51.2%

We then retrained a set of alternative lightweight encoder–decoder models with the same methodology, as reported in Table 3.7. All models improved, confirming the central role of training. However, ENet remains the best compromise in terms of parameter count and accuracy: its retrained performance (70.5% mIoU on Cityscapes, 77.6% on CamVid) makes it competitive with more recent models, at a fraction of their size.

Table 3.7: Original mIoU on Cityscapes as reported by the authors of several real-time encoder–decoder semantic segmentation networks and the obtained mIoU after retraining, noted mIoU*.

Model	**Year**	**Param. (M)**	**mIoU**	**mIoU***
ENet [67]	2016	0.37	58.3%	70.5%
ESPNet [68]	2018	0.4	60.3%	69.6%
DABNet [69]	2019	0.76	70.1%	72.9%
EDANet [70]	2019	0.68	67.3%	69.8%
Fast-SCNN [71]	2019	1.1	69.2%	70.3%
ESNet [72]	2019	1.66	70.7%	73.8%

In conclusion, while the literature tends to emphasize architectural innovations, our experiments show that training routines and data processing are often far more impactful, especially from an embedded perspective. This suggests a pragmatic strategy: rather than adopting increasingly complex networks, one can select a simple and hardware-friendly architecture such as ENet, and invest in carefully optimized training to achieve competitive accuracy while ensuring efficient FPGA implementation.

Hardware architecture and design space exploration We implement ENet as a fully pipelined dataflow architecture, where every layer of the network is mapped to a dedicated hardware block on the FPGA and processed in a streaming manner. This approach, unlike DPU-based sequential execution, avoids external memory transfers and allows much higher throughput, which is crucial under real-time constraints. The design is carried out with the FINN framework [105, 106], which requires adapting the network to comply with HLS limitations. Several modifications were therefore introduced to make ENet compatible with this flow, in particular replacing PReLU activations with ReLUs, reshaping transposed convolutions to avoid output padding, replacing MaxUnpool operations with equivalent ConvTranspose layers, substituting MaxPool+Concat in downsampling branches by strided convolutions, simplifying the InitialBlock, and adjusting the placement of ReLU activations around skip connections. These changes ensure compatibility with FINN and quantization while limiting accuracy degradation.

Quantization-aware training was performed using Brevitas [107], with INT4 weights and activations and INT8 in the first and last layers. This choice provides a good trade-off between accuracy and hardware efficiency, significantly reducing resource usage and power consumption while keeping segmentation performance close to the floating-point baseline. Interestingly, some structural modifications even slightly improved accuracy, while the most detrimental was the change in ReLU placement within skip connections. Overall, the adapted network shows less than 1% drop in mIoU compared to the original model, at the cost of an increase in parameters from 354K to 513K, which remains very small compared to other real-time models.

We deployed the modified ENet on an AMD ZU19EG FPGA, clocked at 300 MHz. Two configurations were implemented: a smaller, power-efficient version (P-ENet$_S$) and a more parallelized version targeting maximum throughput (P-ENet$_M$). In addition, a CamVid-trained version

Table 3.8: Resources utilization of the different versions of the design on AMD ZU19EG.

Model	Dataset	Inp. Size	LUT	BRAM (18K)	FF
P-ENet$_S$	Cityscapes	512x256	232,930 (44.56%)	929 (47.21%)	186,646 (17.85%)
P-ENet$_M$	Cityscapes	512x256	256,992 (49.16%)	914 (46.44%)	205,756 (19.68%)
P-ENet$_C$	CamVid	480x360	254,486 (48.68%)	1290 (65.55%)	201,328 (19.26%)

(P-ENet$_C$) was produced. Resource utilization of these designs is summarized in Table 3.8. None of the designs required DSP blocks, as INT4 arithmetic could be efficiently implemented using LUTs.

Beyond these adaptations, we explored variations of the architecture to optimize the trade-off between accuracy and resource usage. Increasing the projection ratio in bottleneck modules reduces computations but severely harms accuracy, while reintroducing shortcuts between encoder and decoder only marginally improves accuracy at a high resource cost. In contrast, replacing transposed convolutions with nearest-neighbor upsampling proved the most effective modification, reducing computations from 1.47G to 1.11G MACs on Cityscapes while preserving accuracy. After fine-tuning, this model achieved 70.3% mIoU, less than 0.2% below the floating-point baseline.

The final implementations demonstrate that lightweight models such as ENet, when properly adapted and quantized, can reach state-of-the-art efficiency on embedded FPGAs. On Cityscapes, P-ENet$_S$ achieves 126 FPS with 8 ms latency per frame, while P-ENet$_M$ reaches 226 FPS with 4.4 ms latency. On CamVid, P-ENet$_C$ achieves 76.3% mIoU with 171 FPS and 5.8 ms latency. These results highlight that a careful combination of architectural simplification, quantization, and pipelined hardware design allows real-time semantic segmentation within tight resource constraints.

Comparison with previous works We compare our FPGA implementations with previous works in Table 3.9. On Cityscapes, our designs reach 70.33% mIoU, which is at least 5% higher than all previously reported FPGA implementations. The gain is particularly significant compared to the compressed ENetHQ [74], where our models improve accuracy by more than 34% while keeping latency in the same range. In terms of throughput, P-ENet$_M$ achieves the highest performance at 226 FPS with a latency of 4.2 ms, while P-ENet$_S$ offers 126 FPS with 8 ms latency and lower resource usage. Both designs avoid the use of DSP blocks thanks to INT4 quantization, relying entirely on LUT-based arithmetic.

On CamVid, P-ENet$_C$ achieves 76.26% mIoU and 171 FPS with 5.8 ms latency, clearly outperforming previous works in both accuracy and throughput by a wide margin. Compared to the UNet implementation of [79], our design improves accuracy by over 8% and throughput by nearly 40%, while consuming fewer DSPs. These results show that careful retraining, architectural adaptations for compatibility with a pipelined dataflow framework, and quantization-aware design enable ENet to achieve state-of-the-art accuracy and efficiency on FPGAs, highlighting the importance of hardware–algorithm co-design.

3.3.2 PEFSL: Few-Shot Learning on FPGA-SoC

In parallel, the PEFSL (Pipeline for Embedded Few-Shot Learning) project, originating from a student project I supervised with Lucas Grativol, winner of the 2023 OpenHardware AMD Xilinx

Table 3.9: Comparison between different FPGA implementations of semantic segmentation models trained on Cityscapes and CamVid.

	Model	Arch. HW	mIoU (%)	Target	Res.	Lat. (ms)	FPS	Clock Freq. (MHz)	LUT (K)	BRAMs 18K	FFs (K)	DSPs
Cityscapes	**P-ENet$_S$ [Ours]**	Pip.	**70.33**	ZU19EG	512x256	8	126	300	232.9	929	186.6	0
	P-ENet$_M$ [Ours]	Pip.	**70.33**	ZU19EG	512x256	4.2	**226**	300	257	914	205.8	0
	EnetHQ [74]	Pip.	36.8	ZU9EG	240x152	**3**	-	140	76.7	499	87.1	450
	ENet [75]	DPU	63.1	Zynq 7035	1024x512	720	32.9	-	62.6	514	192.2	689
	DeepLabV3+ [76]	DPU	65.29	Arria 10 GX 1150	960x960	670	-	200	-	-	-	1,362
	FPN [77]	DPU	57.6	ZU9EG	512x256	-	128	333	150	1,120	292	-
	ResNet18-UNet [15]	Pip.	62.9	Alveo U250	256x256	44	22.65	152	490.3	4,260	569.5	1,043
	MobileNet-PSPNet [78]	Pip.	64.2	ZU9EG	1024x512	9.1	139	300	182	1,831	137	1,831
CamVid	**P-ENet$_C$ [Ours]**	Pip.	**76.26**	ZU19EG	480x360	**5.8**	**171**	300	254.6	1290	201.3	0
	UNet [79]	DPU	67.8	Alveo U200	480x352	-	123	300	453.8	1,206	685.6	882
	AlexNet-SFCN [80]	Pip.	44.90	ZU9EG	480x360	-	165	100	105.1	561.5	100.5	390
	SegNet [81]	DPU	41	Arria 10 GX 1150	480x360	2891	-	-	-	-	-	1,451
	BEDN [82]	Pip.	59.9	ZU7EV	480x360	-	25.89	-	160.1	312	124.6	92

competition[2], tackled few-shot learning on FPGA-SoC. The developed flow, based on Tensil, covers the entire training, design chain, and deployment on a PYNQ-Z1 board. The demonstrator achieves a latency of 30 ms for a consumption of 6.2 W on MiniImageNet [13].

Embedded Few Shot Learning Conventional object classification relies on training deep neural networks (DNNs) with large labeled datasets, which are often costly to obtain [83]. Few-shot learning (FSL) addresses this limitation by enabling robust classification with only a few labeled examples per class, leveraging knowledge from pre-trained networks through transfer learning or fine-tuning [84, 85].

Implementing FSL on embedded systems remains challenging due to the computational and memory demands of DNNs. FPGA SoCs have emerged as promising candidates for low-power and low-latency deployment [87, 86, 88], but literature examples are scarce. Key challenges include adapting training routines, designing lightweight backbones, and integrating deployment frameworks for real-time inference on embedded hardware.

In this paper, we address these challenges with the following contributions:

- One of the first platforms for real-time few-shot object classification on FPGA SoCs.
- An open-source pipeline[3] based on the Tensil framework[4] for backbone design, training, evaluation, and deployment on FPGA.
- Demonstration on the MiniImageNet dataset of a low-power, low-latency few-shot model deployed on FPGA hardware.

These contributions enable applications in robotics, drones, and autonomous vehicles where responsiveness and energy efficiency are critical.

[2]https://www.openhw.eu/2023-results-gallery

[3]`https://github.com/brain-bzh/PEFSL`

[4]`https://www.tensil.ai/`

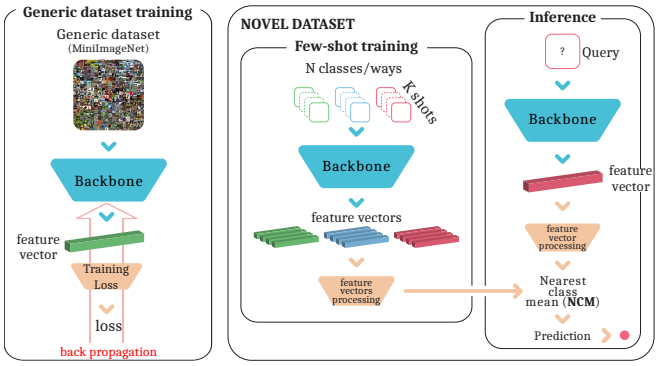


Figure 3.12: Overview of our few-shot learning method.

Few-shot learning aims to classify examples of unseen classes with only a few labeled samples. State-of-the-art methods rely on DNN backbones [85], trained first on a large base dataset with a pretext loss [89] to extract feature embeddings.

Evaluation occurs on a novel dataset split into episodes. Each episode has a number of *ways* (classes), *shots* (labeled examples per class), and *queries* (unlabeled examples). Model performance is measured by accuracy on queries averaged over episodes [90]. Our work focuses on the inductive setting [91], where queries are not known beforehand.

Designing efficient backbones is essential for embedded deployment, enabling rapid adaptation to new tasks with limited computational resources, particularly for real-time object recognition in drones and robots.

Design space exploration of the backbone Our experiments rely on small ResNet architectures [92], using residual blocks with skip connections to enable deeper and more accurate networks than traditional CNNs [93]. We focus on ResNet-9 and ResNet-12, which offer a good trade-off between performance, number of parameters, and computational complexity (Fig. 3.13). Key hyperparameters include network depth, training and test image size, downsampling strategy (strided convolutions or max pooling), and number of feature maps, which together balance accuracy and latency.

We train these networks on MiniImageNet [108], consisting of 64 base, 16 validation, and 20 novel classes with 84×84 images, focusing on a 5-way, 1-shot setup. Data augmentation (multiple crops) improves feature generalization for few-shot learning.

A design space exploration was conducted to identify configurations that optimize both accuracy and real-time performance. Networks were compiled with the Tensil framework to estimate inference cycles. For 32×32 images, ResNet-9 outperforms ResNet-12, likely because deeper networks produce last-layer feature maps that are too small for effective few-shot classification. Matching training and test image sizes (32×32) yields better results than training on larger images. Strided convolutions reduce latency without impacting accuracy, and adjusting the number of feature maps allows tuning the trade-off between speed and performance (Fig. 3.14).

For our embedded application on the PYNQ-Z1, prioritizing real-time inference, the optimal configuration is a strided ResNet-9 with 16 feature maps, trained and tested on 32×32 images, providing an excellent compromise between latency and accuracy.

Compraison with other hardware implementations We set the array size of our systolic array to 12, which corresponds to the maximum possible array size for our setup. The FPGA

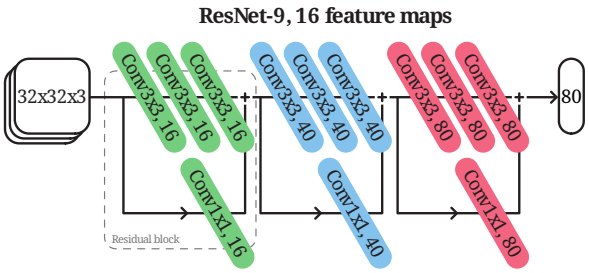


Figure 3.13: ResNet-9 structure: initial layers have 16 output feature maps, and subsequent layers scale accordingly.

Table 3.10: CIFAR-10 inference on Zynq-7020 FPGA

Work	Prec. [bits]	LUT	BRAM [36 kb]	FF	DSP	Latency [ms]	Acc. [%]
[96] hls4ml	8-12	28544	42	49215	4	27.3	87
[96] FINN	1	24502	100	34354	0	1.5	87
[97] (custom)	1-2	23436	135	-	53	1.1	86
[98] (custom)	16	15200	523	41	167	109	-
Ours	16	15667	59	9819	159	35.9	92

frequency has been set to 125 MHz. Under this configuration, the latency of the backbone inference is 30 ms. Few articles have specifically addressed few-shot learning on FPGAs or in embedded systems. An example of few-shot pest recognition on an FPGA has been proposed in [94], reaching 2 frames per second on a PYNQ-Z1.

To demonstrate that the hardware resources and latency obtained using Tensil's framework, based on the computational complexity of our backbone, are within the standards of the literature, we conducted a benchmark and present the results in Table 3.10. We decided to compare with implementations of DNNs proposed for classification on the CIFAR-10 dataset [95], as these images are 32×32 pixels—well-suited to our ResNet-9 backbone with 16 feature maps, supplemented by a downstream linear layer. We restricted our comparison to implementations on the same chip, the Zynq-7020 (z7020). Table 3.10 shows that Tensil's implementation offers comparable latency and accuracy for equivalent resources, validating our backbone choice. It is important to notice that each work implements a different DNN. For this benchmark, we used array size 12 at 50 MHz.

Collaboration with Tohoku University This work led to a collaboration with Tohoku University and Professor Naoya Onizawa, materialized by reciprocal visits and two publications [14, 12]. These articles aim to ensure precision consistency throughout the design flow by applying fixed-point quantization via Brevitas, with QAT and PTQ methods. They show that near floating-point precision can be achieved with only 5 to 6 bits for both integer and fractional parts. These models were then integrated into the FINN suite within PEFSL.

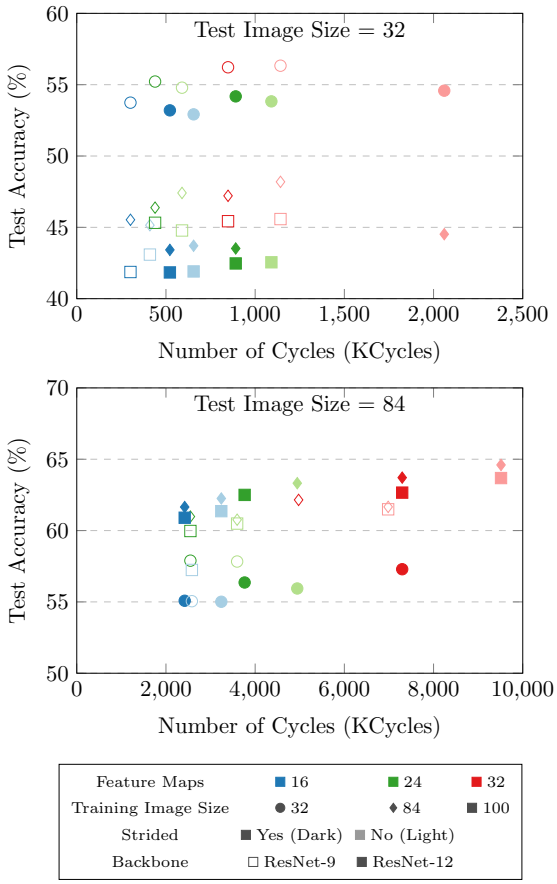


Figure 3.14: Accuracy and latency trade-off for different ResNet configurations, image sizes, and downsampling strategies. Empty markers: ResNet-9, filled: ResNet-12; circles: 32×32, squares: 84×84 images.

3.4 Teaching, Supervision, and Research

The work presented in the previous sections on deep learning compression and hardware implementation relies heavily on a structured supervising strategy. This section details how my research activities are sustained by diverse funding sources and how they are related to my teaching responsibilities as an Associate Professor within the BRAIN team at IMT Atlantique.

3.4.1 Supervision and Funding Strategies

Over the past five years, a lesson I have integrated into my research approach is the critical importance of establishing a strong working relationship with a candidate prior to the official start of a PhD. In France, the standard doctoral timeframe is strictly limited to three years. Once the initial onboarding, literature review, and final manuscript drafting are accounted for, the effective window for producing novel research is quite narrow. If results are delayed or initial hypotheses prove unfruitful, this condensed timeline can place significant pressure on the student.

To mitigate this structural risk, my approach increasingly relies on a preliminary "seed" phase, typically taking the form of a Master's internship or a several-month engineering contract. This onboarding period allows both the student and the supervisory team to build mutual trust and validate the research direction. Furthermore, it frequently enables the publication of a first scientific

contribution before the PhD even begins, thereby significantly reducing the pressure on the student and creating a confident research dynamic.

This framework is supported by a deliberately diversified funding strategy, which guarantees both scientific independence and industrial relevance. Out of the ten doctoral projects I have supervised or am currently supervising, the financial ecosystem is balanced as follows: two are supported by direct industrial research contracts, two by CIFRE agreements bridging academic research with real-world corporate challenges, two by internal funding from IMT and IMT Atlantique, and four by public grants (including three national ANR/PEPR projects and one European COFUND). This diversity demonstrates a capacity to fund ambitious projects autonomously, maintains a strong connection to practical applications through industrial partnerships, and confirms the alignment of our research trajectory with national and European public investment strategies.

3.4.2 Building a Local Talent Pool

A direct consequence of this strategy is a strong local attractiveness for our research projects. Out of the ten aforementioned PhD students, five (Lucas, Hugo Le Blevec, Timotée, Karl, and Jérémy) are alumni of IMT Atlantique. This successful local recruitment pipeline is not accidental; it is the direct result of an active effort to expose engineering students to research challenges early in their curriculum.

3.4.3 Spotlight on Two Courses: PARPING and EFFDL

My teaching activities are closely bound with my research interests, serving as a direct bridge between the classroom and the laboratory. I firmly believe that exposing engineering students to state-of-the-art research challenges is the most effective way to develop their curiosity for research. This philosophy is embodied in two specific courses I have developed and coordinated: PARPING and EFFDL. These courses not only align with the core themes of this manuscript (parallel computing and deep learning compression) but also had the tendency to attract students to pursue research in these areas, either in our team or in other research groups.

PARPING (Calcul Parallèle pour l'Ingénieur) is an intensive eight-week, 40-hour course designed to teach students how to master parallel computing across all levels. The course directly addresses fundamental industry needs, notably the hardware acceleration required for modern generative AI and the optimization of algorithmic energy efficiency, which aligns tightly with the ecological transition goals of the IMT engineering competency framework. Co-taught with Adrien Merlini, Matthieu Arzel, and Serge Guelton from Mozilla (who brings a highly valued industrial software engineering perspective) the curriculum bridges the gap between hardware architecture and practical software implementation. Due to strong student interest, particularly from the software engineering and embedded systems tracks, the course is currently expanding its capacity from 24 to 48 students per year. Students dive deep into instruction, data, thread, and node-level parallelism, gaining hands-on experience with C/C++, OpenMP, OpenMPI, and CUDA.

A important aspect of this class is the flipped classroom approach (*classe renversée*) used to teach Instruction-Level Parallelism (ILP). Instead of lectures, students independently research real-world CPU microarchitectures from major vendors and present their architectural analysis. This approach directly targets core engineering skills, including complex problem solving, system design, and technical communication. The maturity and success of this course were recently recognized when our teaching team was awarded the 2025 IMT "Prix Engagement Pédagogie et Enseignement".

Directly echoing the research presented in this chapter, the EFFDL (Efficient Deep Learning) course introduces students to neural network compression techniques. The syllabus progresses from

deep learning essentials and PyTorch fundamentals to advanced techniques.

Once these solid foundations are established, the core of the curriculum is dedicated to solving the complex trade-off between model accuracy, memory footprint, and computational complexity. Students systematically study and apply state-of-the-art compression techniques that are directly aligned with current research trends:

- **Quantization**: Ranging from reduced precision weights and activations (e.g., INT8, INT4) to dynamic quantization and Quantization-Aware Training (QAT), helping students understand the direct impact of numerical precision on hardware efficiency.
- **Pruning**: Students experiment with both non-structured (magnitude pruning) and structured pruning strategies. This echoes the algorithmic challenges and structural dependencies addressed in my own research, such as Dimensional Clear-Out (DCO).
- **Knowledge Distillation and Factorization**: Exploring how to successfully transfer the generalization capabilities of a heavy, complex teacher model to a much lighter student architecture.

Through this curriculum, students gain a profound understanding of how to tailor software architectures (CNNs) to fully exploit underlying hardware constraints and optimizations.

A strong, shared pedagogical philosophy connects both PARPING and EFFDL, relying heavily on project-based learning and healthy competition. Instead of traditional theoretical exams, students are evaluated through comprehensive, long-term challenges. In PARPING, the course culminates in the "Galax Project," an open-source graphical simulation of galaxy collisions. Students compete to maximize the Frames Per Second (FPS) and optimize energy efficiency by measuring the Joules consumed per generated frame.

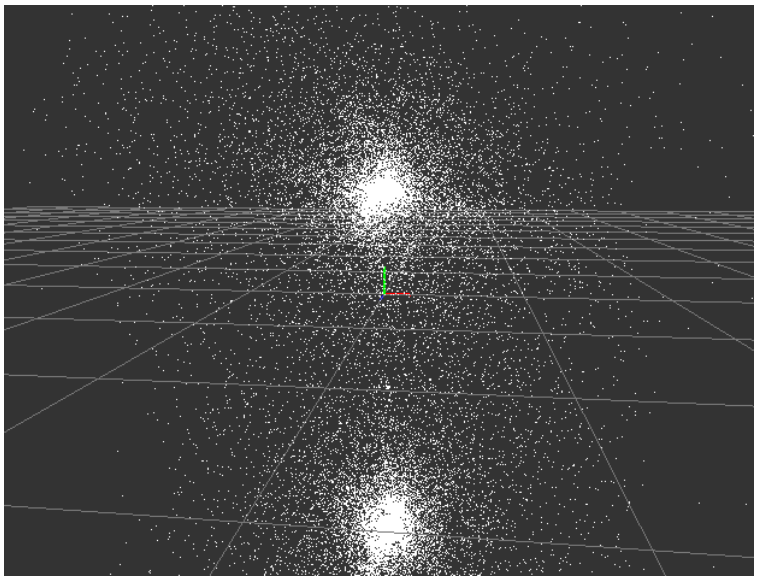


Figure 3.15: Galax Project: Galaxy collision simulation used in the PARPING course for performance optimization challenges.

Similarly, in EFFDL, students engage in a long-term, Kaggle-style challenge to compress a Convolutional Neural Network for the CIFAR-10 dataset. The objective is not simply to achieve the highest accuracy, but to find the optimal ratio between accuracy and the final compression rate of the model.

To avoid the pitfalls of raw competition and rather build a collaborative environment, our evaluation criteria deliberately emphasize critical thinking, the deep understanding of the implemented optimizations, and the sharing of knowledge among peers, rather than just brute-force performance. This innovative methodology has been extensively shared and valorized within the academic community. Through the IMT MEDIANE pedagogy school, we have organized multiple peer-to-peer workshops to disseminate our feedback and strategies, including sessions specifically focused on flipped classrooms ("Motivés en classe renversée") and constructive competition ("Rendre la compétition pédagogique saine").

Finally, a pillar of this teaching approach is continuous, data-driven improvement. Over the past five years, we have systematically collected student feedback through a detailed 13-point questionnaire administered during a dedicated 30-minute oral and written debriefing session. Linear regression analysis on these 120 participants demonstrates highly significant positive trends in student satisfaction, particularly regarding the structure of pedagogical materials, the clarity of lectures, and the transparency of grading expectations. These improvements are the direct result of formalized annual teaching staff debriefings.

Chapter 4

Research Perspectives

4.1 New Methods and Architectures for Digital Communications

4.1.1 AI agents to assist the development of scientific software

The development and maintenance of large-scale open-source scientific software, such as AFF3CT, successfully aggregated the knowledge of a community but faced a structural bottleneck inherent to academic research: the scarce and discontinuous involvement of researchers. Unlike industrial companies, academic labs operate with a limited number of permanent staff and a turnover of PhD students. Maintaining a highly optimized, vectorized, and multi-threaded C++ codebase required a level of expertise and time investment that often competed with the primary goal of publishing new research. This resource constraint limited the pace of innovation and the ability to maintain a "production-grade" infrastructure over time. But one can argue: is this a problem? After all, the primary mission of academic research is to produce novel scientific contributions, not to maintain software.

However, from a higher-level research strategy perspective, the exponential growth in the number of published scientific papers is inevitably diluting the impact of individual publications, even when accepted in top-tier venues. My intuition is that the definition and measurement of scientific impact will substantially evolve in the coming years. As the scientific community increasingly recognizes software as a primary research artifact [109], contributions to major open-source projects will likely become a metric as important as the traditional quantity and citation count of articles. We already observe this shift in the machine learning domain: the integration and adoption of a novel technique into mainstream libraries, such as PyTorch, often has a vastly more profound impact on the broader research community than the original standalone publication. Sevral open source projects constitute major impactful scientific contributions, such as HuggingFace Transformers [125], LLVM [126], or TensorFlow [127]. The number of citations of the original paper may even become a secondary metric compared to the number of downloads, forks, or stars on GitHub. In this context, the strategic importance of maintaining and evolving a widely adopted scientific library like AFF3CT cannot be overstated. It is not just about producing new research; it is about ensuring that this research is accessible, usable, and integrated into the tools that the community relies on for their own innovations.

In the specific field of physical layer communications simulation, AFF3CT has already established a recognized reputation. The strategic necessity now is to capitalize on this traction by expanding the toolbox to be as exhaustive as possible in terms of supported communication standards and channel codes. Ultimately, this comprehensive open-source framework will not only serve as a reference implementation but will also facilitate seamless and robust verification for any

researcher in the field.

The recent emergence of Large Language Models (LLMs) and agents offers a pragmatic solution to the discontinuous nature of academic lab research. The goal is not to replace the researcher with an autonomous system, but to establish a robust Human-AI teaming model. In this paradigm, agents act as force multipliers, handling the tedious, combinatorial "heavy lifting" of software engineering and our CI/CD pipelines. This delegation frees human researchers to focus on what they do best: defining the mathematical architecture, setting rigorous validation criteria, and directing the overarching research strategy.

Implementing and maintaining rigorous test cases in AFF3CT requires high domain expertise. It is not sufficient to simply "run simulations"; one must ensure the scientific relevance of the parameters (e.g., adhering to 3GPP or DVB-S2 standards) and maintain strict consistency across architectures. We envision an AI-assisted workflow where these critical, time-consuming maintenance tasks are delegated to supervised agents:

- **Automated Test Curation and Strategy**: A strength of AFF3CT's ecosystem is the database of simulations that are run on each modification of the codebase. Agents could expand and improve this database by continuously scanning the state of the art and standardization documents to define the most relevant test scenarios. They can identify "gold standard" configurations. Sometimes some specific SNR points are standard in the litterature. Some others may be chosen by working groups like 3GPP as "reference points". Agents could automatically extract these from the literature and standards documents, ensuring that the test suite remains scientifically relevant and up-to-date. They can also identify and prioritize test cases that reveal known weaknesses, such as error floors or specific trapping sets. This moves the test design from a manual, ad-hoc process to a systematic, literature-driven exploration.

- **Robust Reference Generation**: Generating reference files requires covering a massive combinatorial space of simulation parameters (channel model, modulation, code family, rate, blocked length, decoding algorithm, etc.). Agents can do more than passively maintain this database; they can make rational decisions on which simulations to execute via the Command Line Interface (CLI). By prioritizing scenarios that maximize information gain or regression coverage within a fixed computational budget, they effectively orchestrate the Continuous Integration pipeline.

- **Verification as the Bottleneck**: With agents capable of generating C++ implementations of new algorithms (like the GRM-SCF [30]), the role of the human researcher shifts to defining the validation criteria. The AFF3CT infrastructure becomes the automated "Oracle" in which agent's proposals could be tested.

Therefore, a highly relevant research direction is the formal definition and validation of agentic workflows applied to physical layer simulations. The objective is to design the methodological frameworks that allow an AI agent to safely and effectively interact with a complex scientific codebase. This involves establishing structured prompting methodologies for hardware-aware C++ generation, designing automated verification loops, and defining the strict architectural constraints the agent must respect.

For example, in the use case of exploring new implementations of algorithms, in practice, the workflow must define how an agent drafts module structures or suggests SIMD vectorization strategies, and then leverages the AFF3CT harness to automatically verify its proposals against thousands of test vectors. If a proposed implementation fails a compilation check, causes a memory leak, or

degrades the Frame Error Rate (FER) or Bit Error Rate (BER), the protocol must strictly dictate how the agent ingests this execution feedback to iteratively correct its own code. The researcher's focus shifts toward defining the exact latency and accuracy requirements, designing the execution safeguards, and validating the integration logic.

Developing these automated, self-correcting workflows represents a high-TRL research project. It aligns perfectly with strategic funding programs aimed at securing sovereign, open-source technological infrastructures. A project structuring these AI-assisted workflows around the AFF3CT ecosystem would be a strong candidate for support from European open science frameworks, such as the HORIZON-CL4-2026-04-DATA-02 call (Open Internet Stack Sovereign Solutions). By formally establishing these methodologies, we can accelerate the prototyping and integration of novel physical-layer algorithms while maintaining the strict reliability required for next-generation 6G communication standards. These could also lead to well established open source projects such as OAI (Open Air Interface) [113] which is widely used in the research community for 5G and LTE implementations, to integrate AFF3CT as a reference physical layer module, further amplifying its impact and adoption.

4.1.2 Hardware Exploration Facilitated by Evolutionary Algorithms

State of the Art and Limitations The customization of RISC-V architectures for domain-specific applications is currently a thriving field. While commercial solutions like Andes *ACE* [102] facilitate the integration of user-defined instructions, academic frameworks such as *MARVEL* [0] have pushed the boundary by automating the generation of RISC-V ISA extensions specifically targeted at Edge-AI. These tools generally operate by profiling existing C code to identify computational hotspots, subsequently generating the corresponding RTL and toolchain support.

However, their fundamental limitation lies in their strict reliance on human intuition: they optimize only what the engineer has explicitly formulated in the software algorithm. They lack the capacity to invent new algorithmic primitives or to discover non-intuitive, bit-level manipulations that could reduce a decoder's critical path. For instance, in the context of Polar code decoding, while a human engineer might focus on optimizing the SC node update function, an AI agent could potentially discover a novel instruction that combines multiple steps of the decoding process into a single, highly efficient operation. Exploration of this search space is a tedious, combinatorial problem that is not well-suited to manual design. This is where evolutionary algorithms, which have been successfully applied in other domains for architecture search, could play a role in the design of domain-specific architectures for channel coding.

Decoupled Evolutionary Exploration Loop Inspired by DeepMind's *AlphaEvolve* [103], we propose an exploration methodology applied to the design of Domain-Specific Architectures (DSA) for channel coding. Historically, the bottleneck for applying evolutionary algorithms to hardware design has been the prohibitive time cost of evaluating the fitness of each generation. To solve this, our approach enforces a strict separation of concerns: we use our existing, highly optimized software (AFF3CT) and High-Level Synthesis tools as fast, decoupled oracles.

With these rapid evaluation pipelines in place, exploring the combinatorial search space becomes computationally viable. As illustrated in Figure 4.1, the exploration process is structured around a dual-path evolutionary loop:

- **Genotype (Proposal):** An LLM acts as the mutation operator. Taking a baseline decoding kernel (e.g., a Polar SC node update), the agent proposes architectural variations expressed as HLS-ready C++ snippets.

- **Functional Oracle (AFF3CT):** The proposed logic is integrated into the AFF3CT simulator. AFF3CT acts purely as an algorithmic gatekeeper, evaluating the decoding performance (BER/FER) on Monte Carlo simulations. This ensures that any proposed modification maintains or improves the error-correction performance, effectively filtering out functionally invalid candidates.

- **Hardware Estimator (HLS):** In parallel, the generated C++ snippet is pushed through a High-Level Synthesis (HLS) toolchain. This provides reliable, cycle-accurate estimates of hardware metrics, including logic area (LUTs, FFs, DSPs), operational latency, and maximum achievable frequency.

- **Evolution (Feedback):** A fitness function aggregates the error-correction penalty from AFF3CT with the hardware costs from the HLS tool. This composite score, along with specific compilation or behavioral errors, is fed back to the LLM. The agent iterates on the prompt, navigating the complex trade-off space between algorithmic accuracy and silicon efficiency.

I would add that genetic algorithms are unsatisfactory for exploring the design space of domain-specific architectures, as they are not designed to handle the complex, multi-objective optimization problem that arises when trying to balance algorithmic performance with hardware efficiency. Therefore this approach is not intended to be a pure genetic algorithm, but rather a hybrid method that leverages the strengths of both evolutionary strategies and LLM-based code generation. The scheme proposed in Figure 4.1 is a first naive step to initiate this research direction, but our goal is to develop other methodologies that will involve at different degrees LLM decision-making.

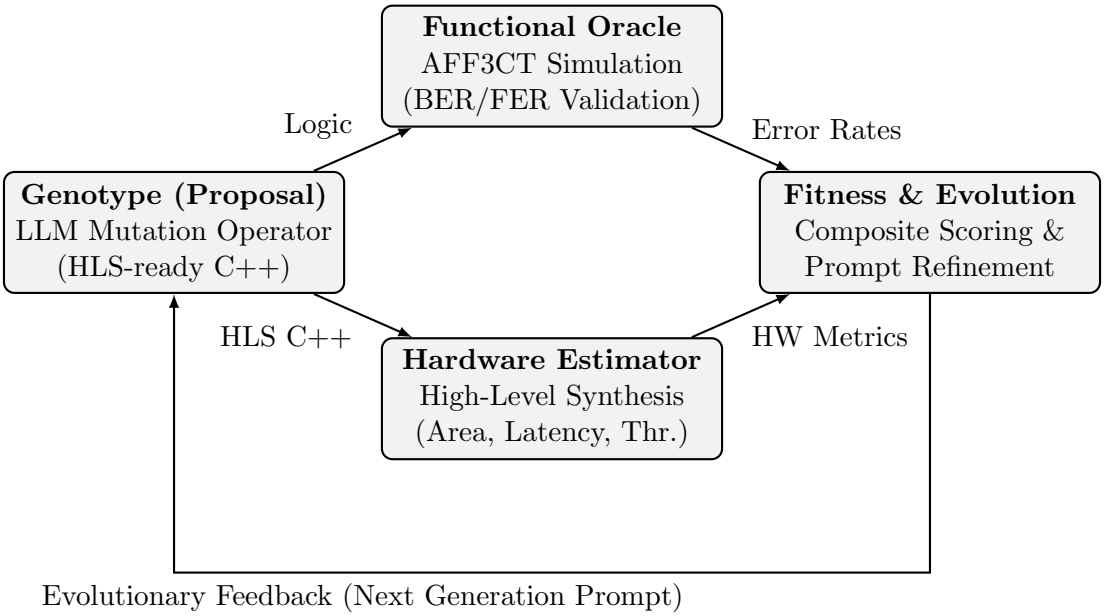


Figure 4.1: The Decoupled Evolutionary Exploration Loop. AFF3CT ensures functional correctness and algorithmic performance, while HLS tools estimate physical hardware costs. The LLM agent optimizes the composite feedback.

4.2 Algorithm-Architecture Co-design

Through the work of Hugo Le Blevec, that now joined the company ARKANE, Jérémy, and Timotée, we have contributed to the vast domain of algorithm-architecture co-design in Deep Learning[15, 8, 25, 10]. In the next years, we will continue to explore this domain, with a particular focus on the application of Deep Learning for GNSS denoising in a spatial context, and on split-processing and event-based vision for Edge AI.

4.2.1 Application of Deep Learning for GNSS Denoising in a Spatial Context

4.2.1.1 Context: The Anti-Jamming Challenge and ARKANE Tech Collaboration

The persistent vulnerability of Global Navigation Satellite Systems (GNSS) to interference and intentional jamming is a critical issue for autonomous systems. This vulnerability is especially severe in space environments, such as for launchers and satellites, where signal integrity is the ultimate guarantor of mission success. To address these challenges, we initiated a CIFRE thesis in collaboration with the company ARKANE, a company specializing in advanced GNSS security, anti-jamming, and robust positioning solutions. Elise Lagarde started her PhD in 2025. The objective of this research is to explore the integration of Deep Learning to replace or augment classical processing chains for interference mitigation, focusing on hardware-aware implementations for constrained avionics FPGAs. Indeed, FPGAs are often the only viable solution for real-time processing in space applications due to their radiation tolerance and deterministic performance, but they impose strict limitations on computational resources and power consumption.

4.2.1.2 The Complexity of Matrix Inversion in STAP

The project focuses on Controlled Reception Pattern Antennas (CRPA), which utilize an antenna array to spatially filter incoming signals. The classical gold standard for such processing is Space-Time Adaptive Processing (STAP). Algorithmic implementations of STAP require estimating the covariance matrix of the incoming noise and computing its inverse to derive optimal beamforming weights. However, direct computation of the inverse using generic or definition-based formulas (e.g., Gauss-Jordan elimination) is computationally expensive, highly sensitive to matrix conditioning, and poorly suited to efficient hardware implementation as the matrix dimension grows [115].

To circumvent these issues, hardware-oriented architectures rely on structured numerical procedures. Decomposition-based methods such as QR [116, 117] and Cholesky (or LDL^H) [118, 119] factorizations have emerged as the most mature solutions on FPGA platforms. For Hermitian positive-definite matrices—such as covariance matrices—Cholesky and LDL^H factorizations are particularly attractive [120]. They exploit the symmetric structure to reduce the global computational complexity to approximately $\frac{1}{3}n^3$, compared to $2n^3$ for QR decomposition, while avoiding resource-heavy square-root operations in the case of LDL^H.

Despite these algorithmic optimizations, deploying them on FPGAs requires a delicate balance of hardware resources. State-of-the-art implementations typically rely on highly optimized systolic arrays to maximize throughput. However, as the antenna array size N grows, maintaining the high data rates necessary for highly dynamic environments (e.g., spinning launchers) becomes increasingly challenging.

4.2.1.3 Deep Learning as an Alternative Paradigm

To explore potential solutions to these scaling limitations, we investigate replacing the classical inversion steps with a quantized Convolutional Neural Network (CNN). Recent literature has demonstrated the potential of machine learning for interference mitigation and beamforming, with models such as LcxNet and LSTM-based approaches successfully detecting and classifying jamming signatures.

Our core research hypothesis is that a neural network might learn the underlying mapping between antenna inputs and optimal weights more efficiently than an explicit matrix inversion for large N. While the number of parameters and operations in the network inherently scales with the input size, the CNN executes a highly regular, deterministic sequence of Multiply-Accumulate (MAC) operations. A useful byproduct of this deterministic execution is a strictly fixed latency, which contrasts with the variable execution times sometimes observed in classical iterative solvers.

However, whether this neural paradigm can consistently outperform highly optimized classical systolic arrays remains an open research question. Recent theoretical analyses have shown that purely data-driven inversion models are fundamentally limited by matrix conditioning and exhibit poor generalization near singularities [121]. To ensure numerical robustness, we are experimenting with hybrid training methodologies, echoing recent trends in input-aware Cholesky approximations [122], that guide the network to learn the underlying matrix factorization rather than just the final inverse. Specifically, we train the network using a compound loss function that balances the accuracy of the intermediate triangular matrix representation and the final reconstructed matrix:

$$L = \alpha \|R - R_n\|^2 + (1 - \alpha)\|A - R_n^{\mathsf{H}} R_n\|^2$$

In this formulation, A represents the input covariance matrix, R is the ground-truth upper triangular factor derived from a classical Cholesky decomposition, and R_n is the network's prediction of this triangular factor. By adjusting the hyperparameter α, we aim to force the CNN to respect the structured mathematical properties of the factorization, bridging the gap between data-driven estimation and algebraic rigor.

4.2.1.4 Hardware-Aware Co-design and Future Goals

Unlike generic software-based deep learning models, our approach is linked with Hardware-Aware Training and Algorithm-Architecture Adequacy. We constrain the neural network design using power-of-two quantization and structured pruning tailored specifically to fit the stringent DSP and memory limits of radiation-hardened FPGAs.

This exploration entails significant scientific risks. A fundamental challenge is ensuring generalization to unknown jammers: unlike analytical STAP, which relies strictly on physical properties, a CNN might hallucinate an incorrect solution when facing a jamming signature widely outside its training distribution. Furthermore, because real-world jammed GNSS data is rare and often classified, we must rely heavily on high-fidelity simulators, thereby introducing a "sim-to-real" gap that must be carefully bridged.

The ultimate objective of this research is to ascertain whether neural networks can provide a viable, hardened, and autonomous anti-jamming IP block for space launchers. If successful, such a module could self-reconfigure to adapt to evolving threat landscapes, directly contributing to the technological sovereignty of future navigation solutions.

4.2.2 Split-Processing and Event-Based Vision for Edge AI

Building upon our recent advancements in embedded perception for unmanned aerial vehicles (UAVs), this project explores the next generation of ultra-low-latency architectures. In our recent work, we introduced LoLa-YOLO [7], a low-latency object detection model specifically designed for search-and-rescue (SAR) missions. Deployed on resource-constrained FPGAs using the FINN framework, LoLa-YOLO efficiently processes data from both RGB and event cameras. By optimizing the network architecture for the specific needs of SAR UAVs, we achieved an inference latency of approximately 6ms for 640x480 resolution inputs. This demonstrated that accurate, real-time computer vision can be successfully embedded onboard to assist human operators, even when operating under strict Size, Weight, and Power (SWaP) constraints.

Moving forward, we want to investigate the split-processing framework (or "learning to sense" paradigm) tailored for event-based vision. Event cameras, which generate a continuous and highly sparse stream of asynchronous data, are inherently well-suited for distributed processing architectures. In a SAR drone context, constantly processing the entire event stream for complex object detection can still be resource-intensive, particularly when dealing with sensor noise or ego-motion from the drone's flight.

The core idea of this project is to split the computational load: a tiny, ultra-low-power neural network is placed as close to the sensor as possible (at the edge), while a larger, more capable network resides on a heavier embedded target or GPU. The edge network would act as an intelligent, localized filter, effectively a "wake-up" mechanism that suppresses background noise and ego-motion. It would only transmit data or trigger the heavier processing layers when a relevant anomaly or object of interest is detected in the scene. This approach promises to optimize power consumption and computational efficiency on the silicon directly adjacent to the sensor.

To develop this architecture, we are establishing a collaboration with Jovita Lukasik from the University of Siegen. Her expertise in Neural Architecture Search (NAS) will be instrumental in exploring the highly constrained search spaces required to design these tiny edge networks, ensuring they do not consume more power or silicon area than the event sensors themselves. To formalize and accelerate this research, Jovita visited our team this year and we are currently applying for joint funding to support a post-doctoral researcher.

On a side note, I want to mention that I organized a seminar in 2022 named "Efficient Hardware for Deep Neural Network Processing" and we are organizing with Nicolas Gac from Université Paris Saclay, this year, 2026, a new one named "Adéquation Algorithme Architecture et approches NAS pour une IA Efficace". Both were organized as seminars of the GDR IASIS (Information, Apprentissage, Signal, Image et ViSion), which is a major research network in France for AI and signal processing. These seminars have the great advantage of bringing together researchers from both the algorithmic and hardware communities, facilitating cross-disciplinary discussions and collaborations that are essential for the co-design approaches we are pursuing.

4.3 Future Hardware for Deep Learning - Modeling and Exploration

4.3.1 Automated Hardware Search

4.3.1.1 The Hardware Design Bottleneck

While our previous exploration of RISC-V architectures focused on accelerating error-correcting codes via specific instructions, designing custom hardware for deep learning represents a different

and broader challenge. Today, the design of AI hardware accelerators, whether commercial GPUs or TPUs, relies heavily on human expertise. The space of possible architectures is very large, yet the points actually explored and instantiated by the industry remain extremely sparse. In practice, designers make a multitude of arbitrary or heuristic choices: the size and hierarchy of memories (register files, on-chip buffers), the number of SIMD units or compute lanes allocated to each processing element (PE), or the bandwidths of the interconnect buses.

These architectures, although optimized for generic workloads, remain intrinsically limited by the engineer's intuition. To overcome these human limitations and discover truly optimal and potentially counter-intuitive architectures, it is necessary to systematically and jointly explore the hardware design space and the neural network topology space. The major challenge is thus to have tools capable of providing fine-grained estimations of latency, energy consumption, and silicon area footprint, without paying the prohibitive cost of a full RTL synthesis or cycle-accurate simulation for every point in the explored space.

4.3.1.2 Modeling the Hardware/Software Interface: Stream and MONET

To make this exploration possible, our team relies on the development of fast, high-level evaluation frameworks. The first building block is the *Stream* framework [124], an experimentally validated tool that models neural network inference on Heterogeneous Dataflow Accelerators (HDAs). Stream models the accelerator as a heterogeneous graph of compute cores, each possessing its own dataflow and communication links. Starting from a software model described in the standard ONNX format, Stream analyzes data dependencies across the different network layers and uses constraint optimization solvers to determine the best spatial and temporal partitioning. It excels particularly in modeling fine-grained layer fusion (layer-fused scheduling), a crucial technique that exploits inter-layer dependencies to minimize memory transfers and significantly reduce latency.

However, modeling inference is only part of the problem. We therefore extended these foundations by developing *MONET* (Modeling and Optimization of neural NEtwork Training from Edge to Data Centers) [123], a framework specifically designed to address the unique challenges of neural network training on these heterogeneous architectures. Unlike inference, training imposes radically different constraints, notably a massive memory footprint due to the need to save activations during the forward pass, as well as increased computational complexity related to gradient backpropagation (backward pass) and optimizer weight updates.

MONET bridges this gap by fully decoupling the software model from the hardware model (flexibly described via JSON files). It introduces a series of transformations on the initial ONNX graph to generate a complete training graph, integrating gradient-specific primitives (such as transposed convolutions) and optimizer states (such as Adam or SGD moments). Furthermore, MONET enables the precise evaluation of complex deployment strategies, such as *activation checkpointing*, which offers a non-linear trade-off between memory savings and the cost of recomputing activations. Through this end-to-end modeling, MONET provides a fast and accurate oracle to estimate the energy, latency, and memory required for a complete training iteration, thereby revealing architectural trade-offs that would remain invisible with tools limited to inference.

4.3.1.3 From Manual Design to Automated Exploration

By leveraging MONET and Stream as fast, reliable fitness functions, it becomes possible to close the loop on automated architecture search. Rather than manually adjusting the size of multiply-accumulate (MAC) arrays or local memory capacities, we can integrate these estimators at the core of a global AI-driven optimization loop.

It is important to emphasize that navigating this highly non-convex design space requires advanced search operators. While traditional genetic algorithms struggle with the complex, multi-objective nature of domain-specific architectures, we propose utilizing LLMs to act as intelligent mutation operators within the search loop.

In this automated paradigm, the AI agent generates a candidate "genotype" which MONET evaluates. This rapid feedback allows the LLM to refine its proposals, uncovering non-intuitive, bit-level manipulations or architectural layouts that standard human heuristic design would miss. Consequently, the engineering effort fundamentally shifts: the researcher no longer draws the architecture by hand, but focuses entirely on defining the search space boundaries and the strict optimization constraints the agent must respect.

4.3.2 Hardware Architectures for AI (PEPR CAMELIA)

4.3.2.1 Context: The Shift to Transformers

This automated co-design methodology is the cornerstone of our involvement in the PEPR CAMELIA program (Composants pour l'Accélération Matérielle Et Logicielle de l'IA). Within this framework, our research focus is expanding to address the most pressing workload in modern artificial intelligence: Transformer architectures. While Convolutional Neural Networks (CNNs) have long dominated embedded vision tasks, Transformers are rapidly becoming the universal standard across natural language processing, vision, and audio domains.

4.3.2.2 Objective: Co-designing Next-Generation Mixed-Precision Accelerators

The primary challenge of deploying Transformers on constrained hardware lies in the quadratic complexity of their Self-Attention mechanism and their massive memory bandwidth requirements. To tackle this "memory wall," we must look beyond classical scheduling and aggressively reduce the data footprint. Utilizing low precision formats (where weights and activations are strictly under 8 bits) represents a major lever to reduce costs and accelerate large AI models during both training and inference.

To this end, I am establishing a new research axis targeting the holistic hardware/software co-design of Transformer accelerators. This effort will be made through a PhD thesis of Arthur Gaudard in the PEPR CAMELIA program, and also by an incoming Ph.D. student (funded by the CEA, co-supervised with Maria Lepecq and Erwan Piriou), focusing specifically on efficient mixed-precision hardware architectures for low-precision quantized transformers.

Recently, the Microscaling FP4 (MXFP4) format was introduced by the Open Compute Project (OCP). This format utilizes fine-grained scaling factors based on shared microexponents, grouping 32 4-bit elements (E2M1) that share a single 8-bit scaling factor (E8M0). While major industry players are adopting these formats—such as Nvidia's NVFP4 in their Blackwell GPUs—and open-source models are releasing weights in these formats, their application is currently heavily skewed towards Large Language Models (LLMs). Furthermore, there is a distinct gap regarding dedicated, low-power embedded hardware architectures, with very few proposals existing beyond basic MAC units.

The Ph.D. research projects will structure our approach around three core scientific questions:

- **Impact on Vision Transformers (ViTs):** We will evaluate the benefit of MXFP4 formats specifically on ViT models, which have been largely overlooked in current Quantization-Aware Training (QAT) and Post-Training Quantization (PTQ) literature in favor of LLMs. This involves identifying which specific layers within the ViT can tolerate MXFP4 quantization,

evaluating the efficacy of these formats on non-linear layers, and defining robust mixed-precision schemes combining FP4, FP8, INT4, and INT8.

- **Format Selection for Embedded Inference:** We will analyze the optimal microscaling format for designing a dedicated embedded hardware architecture. The project will navigate the trade-off between hardware-friendly formats like MXFP4-OCP (which relies on power-of-two scaling) and accuracy-preserving formats like NVFP4. This may include exploring alternative formats like AMXFP4 or researching novel quantization paradigms, such as Quantization-Aware Distillation (QAD), to convert NVFP4 models to MXFP4-OCP.
- **Hardware Architectural Optimization:** Finally, relying on our automated search methodologies, we will design custom, scalable processing elements and memory hierarchies optimized to efficiently support these specific MXFP4 data types in terms of energy and latency.

By coupling these low-precision hardware innovations with advanced Dataflow and Scheduling strategies, we aim to improve energy efficiency. Ultimately, this work will produce sovereign, high-performance hardware IPs capable of supporting the next generation of foundational models in embedded and edge environments.

4.3.3 Edge AI for Health: From Microcontrollers to Wearable ASICs

4.3.3.1 Context: A Strategic Synergy for BCI and EEG

The decoding of Electroencephalographic (EEG) signals is rapidly migrating from highly controlled clinical environments to portable, wearable devices. Beyond the obvious scientific and medical challenges, focusing on EEG represents a highly strategic positioning for our research team. This positioning is fundamentally rooted in a strong, complementary collaboration: on one side, domain experts in Brain-Computer Interfaces (BCI) and EEG, notably Giulia Lioi (IMT Atlantique) who is the member of my team, BRAIN, and on the other, our team specializing in the efficient implementation of deep learning algorithms on constrained hardware.

This synergy provides us with a unique relevance and capacity to contribute to the field. First, it has already established a strong dynamic within the team, highlighted by recent high-impact publications such as the introduction of the REVE foundation model [110]. Second, EEG analysis is an ideal target because the underlying neural models are significantly lighter compared to those required for high-resolution computer vision or complex natural language processing. This characteristic makes it a perfect match for institutions with constrained computing clusters, allowing us to remain highly competitive on the international stage. Finally, the transition of EEG processing to embedded systems is a mature and highly relevant field, as demonstrated by recent literature introducing transformer-based architectures for wearable monitoring [111]. These devices operate under extreme energy constraints, yet they must continuously process multichannel time-series data with high temporal resolution.

4.3.3.2 Preliminary Work: Simple Attention on Microcontrollers

Currently, Convolutional Neural Networks (CNNs), such as EEGNet [104], represent the state of the art for embedded EEG classification due to their compactness. However, CNNs inherently struggle to capture the long-range dependencies essential for identifying complex neurological biomarkers. Transformer architectures theoretically solve this by modeling global relationships, but their Self-Attention mechanism imposes a quadratic memory and computational bottleneck, rapidly exceeding the SRAM limits of standard Microcontroller Units (MCUs).

As a preliminary step to bridge this gap, we are currently working on submitting a work focusing on the direct implementation of highly optimized, lightweight attention variants specifically designed for MCUs. This initial methodology investigates Simple Attention (SimA) [112], which completely removes the computationally expensive Softmax operation, replacing it with a column-wise L_1 normalization. Furthermore, SimA dynamically reorders its matrix multiplications based on the relationship between sequence length and embedding dimension, ensuring the smallest possible intermediate representations. Alongside techniques like Fused Attention we integrated these custom operators directly into the CMSIS-NN framework for high-performance ARM Cortex-M MCUs. While this validates the feasibility of transformer-like mechanisms on off-the-shelf microcontrollers, it represents only a starting point in our broader research agenda.

4.3.3.3 Long-Term Goal: Real-World Wearables and Custom ASICs

To go on towards real-world applications, we must look beyond the limitations of general-purpose microcontrollers and target Application-Specific Integrated Circuits (ASICs). Real-world BCI applications, such as chronic seizure detection or continuous sleep staging, impose strict power budgets often in the sub-milliwatt range, necessitating custom silicon solutions. The literature underscores this inevitable shift; early pioneering works demonstrated the necessity of micro-power EEG acquisition Systems-on-Chip (SoCs) for seizure detection [114].

Our long-term vision is to design a complete, end-to-end system for real-time EEG analysis on wearable ASICs. This involves co-designing the neural network architecture and the underlying hardware to achieve optimal performance within the stringent energy constraints of wearable devices. We will explore advanced quantization techniques, such as mixed-precision formats, to further reduce the computational load while maintaining accuracy.

Appendix A

Curriculum Vitae

A.1 Summary of Professional Career and Academic Activities

MATHIEU LEONARDON

Associate Professor

Algorithm-Architecture Adequation
Deep Learning Compression

MEE Department
IMT Atlantique
born March 18th, 1987, French
(+33) 229001384
mathieu.leonardon@imt-atlantique.fr
My Webpage
Github *Linkedin*

Education

2015–2018 **PhD, Electrical Engineering**, *Polytechnique Montréal and Université de Bordeaux*, codirec, Polar Decoding on programmable architectures, *defended on 2018-12-13*.
Forward Error Correction, Polar Codes, Software Implementations, Hardware Implementations, ASIP

President Mohamad Sawan

Reviewers Amer Baghdadi, Emmanuel Casseau

Directors Christophe Jégo, Yvon Savaria

Examiners Camille Leroux, Olivier Muller, Charly Poulliat

2012–2015 **Master of Engineering, Embedded Electronics**, *Bordeaux INP, ENSEIRB-Matmeca*, Bordeaux.

Publications

Journal Articles

2022 Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, Thomas Hannagan, and David Bertrand. Rethinking Weight Decay for Efficient Neural Network Pruning. *Journal of Imaging*, volume 8, page 64. MDPI, March 2022.

2022 Khaled Alhaj Ali, Amer Baghdadi, Elsa Dupraz, **Mathieu Léonardon**, Mostafa Rizk, and Jean-Philippe Diguet. MOL-based In-Memory Computing of Binary Neural Networks. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, volume 30. IEEE, July 2022.

2019 **Mathieu Léonardon**, Adrien Cassagne, Camille Leroux, Christophe Jego, Louis-Philippe Hamelin, and Yvon Savaria. Fast and Flexible Software Polar List Decoders. *Journal of Signal Processing Systems*. Springer, January 2019.

2019 Alireza Ghaffari, **Mathieu Léonardon**, Adrien Cassagne, Camille Leroux, and Yvon Savaria. Toward High-Performance Implementation of 5G SCMA Algorithms. *IEEE Access*, volume 7, pages 10402–10414. IEEE, January 2019.

2019 Adrien Cassagne, Olivier Hartmann, **Mathieu Léonardon**, Kun He, Camille Leroux, Romain Tajan, Olivier Aumage, Denis Barthou, Thibaud Tonnellier, Vincent Pignoly, Bertrand Le Gal, and Christophe Jego. AFF3CT: A Fast Forward Error Correction Toolbox! *SoftwareX*, volume 10, page 100345. Elsevier, July 2019.

In Conference Proceedings

2025 Manon Renault, Hamoud Younes, Hugo Tessier, Ronan Le Roy, Bastien Pasdeloup, and **Mathieu Léonardon**. Event Classification of Accelerometer Data for Industrial Package Monitoring with Embedded Deep Learning. In *COMPSAC 2025: IEEE 49th Annual Computers, Software, and Applications Conference*, Toronto, Canada, July 2025.

2025 Jérémy Morlier, **Mathieu Léonardon**, and Vincent Gripon. Réduction de la Résolution comme Technique de Compression pour les Systèmes d'Apprentissage Profond en Vision. In *GRETSI*, Strasbourg, France, August 2025. GRETSI.

2025 Jérémy Morlier, **Mathieu Léonardon**, and Vincent Gripon. Input Resolution Downsizing as a Compression Technique for Vision Deep Learning Systems. In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, Rome, Italy, June 2025. IEEE.

2025 Karl La Grassa, André Béliveau, **Mathieu Léonardon**, Jean-Pierre David, Matthieu Arzel, and Yvon Savaria. A Prototyping Framework for P4-Programmable Traffic Managers. In *RSP '25: Proceedings of the 36th International Workshop on Rapid System Prototyping*, pages 1–7, Taipei, Taiwan, September 2025. ACM.

2025 R. Kanda, Hugo Le Blevec, N. Onizawa, **Mathieu Léonardon**, Vincent Gripon, and T. Hanyu. Bit-Width-Aware Design Environment for Few-Shot Learning on Edge AI Hardware. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025 IEEE International Symposium on Circuits and Systems (ISCAS), pages 1–5, Londres, United Kingdom, May 2025. IEEE.

2025 Ismail Amessegher, Arthur Gaudard, Kojo Nyamekye Anyinam-Boateng, Hugo Le Blevec, Lionel Génevé, Florian Pouthier, **Mathieu Léonardon**, Hajer Fradi, Lucia Bergantin, Panagiotis Papadakis, Isabelle Fantoni, Jean-Philippe Diguet, and Matthieu Arzel. Towards Low-Latency Object Detection on Board Reactive Search-and-Rescue Drones. In *Proceedings of the 2025 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, Galway, Ireland, October 2025.

2024 Timotée Ly-Manson, **Mathieu Léonardon**, Abdeldjalil Aissa El Bey, Ghouti Boukli Hacene, and Lukas Mauch. Analyzing Few-Shot Neural Architecture Search in a Metric-Driven Framework. In *Proceedings of the Third International Conference on Automated Machine Learning, PMLR*, volume 256, pages 1–33, Paris, France, September 2024.

2024 R. Kanda, N. Onizawa, **Mathieu Léonardon**, Vincent Gripon, and T. Hanyu. Design Environment of Quantization-Aware Edge AI Hardware for Few-Shot Learning. In *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 928–931, Springfield, France, August 2024. IEEE.

2024 Lucas Grativol, **Mathieu Léonardon**, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. FLoCoRA: Federated Learning Compression With Low-Rank Adaptation. In *European Signal Processing Conference (EUSIPCO), 2024 32nd*, Lyon, France, August 2024.

2024 Lucas Grativol, Lubin Gauthier, **Mathieu Léonardon**, Jérémy Morlier, Antoine Lavrard-Meyer, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. PEFSL: A deployment Pipeline for Embedded Few-Shot Learning on a FPGA SoC. In *ISCAS 2024: IEEE International Symposium on Circuits and Systems*, Singapore, Singapore, May 2024.

2023 Timotée Ly-Manson, **Mathieu Léonardon**, and Abdeldjalil Aïssa-El-Bey. Understanding Few-Shot Neural Architecture Search with Zero-Cost Proxies. In *29ème Colloque Francophone de Traitement du Signal et des Images (GRETSI 2023)*, Grenoble, France, August 2023.

2023 Hugo Le Blevec, **Mathieu Léonardon**, Hugo Tessier, and Matthieu Arzel. Pipelined Architecture for a Semantic Segmentation Neural Network on FPGA. In *ICECS 2023: IEEE 30th International Conference on Electronics, Circuits and Systems*, Istanbul, Turkey, December 2023.

2023 Hugo Le Blevec, **Mathieu Léonardon**, Hugo Tessier, and Matthieu Arzel. Co-conception d'un réseau de neurones convolutif de segmentation sémantique et de son architecture matérielle. In *29° Colloque sur le traitement du signal et des images*, Grenoble, France, August 2023. GRETSI - Groupe de Recherche en Traitement du Signal et des Images.

2023 Lucas Grativol, **Mathieu Léonardon**, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. Federated learning compression designed for lightweight communications. In *ICECS 2023: IEEE 30th International Conference on Electronics, Circuits and Systems*, Istanbul, Turkey, December 2023.

2023 Lucas Grativol, **Mathieu Léonardon**, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. Compression de réseaux de neurones pour l'apprentissage fédéré. In *GRETSI 2023 : XXIXème Colloque Francophone de Traitement du Signal et des Images*, Grenoble, France, August 2023.

2023 Darshan C Ganji, Saad Ashfaq, Ehsan Saaboori, Sudhakar Sah, Saptarshi Mitra, Mohammadhossein Askarihemmat, Alexander Hoffmann, Ahmed Hassanien, and **Mathieu Léonardon**. DeepGEMM: Accelerated Ultra Low-Precision Inference on CPU Architectures using Lookup Tables. In *CVPRW 2023: IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, Vancouver, Canada, June 2023.

2022 Hamoud Younes, Hugo Le Blevec, **Mathieu Léonardon**, and Vincent Gripon. Inter-Operability of Compression Techniques for Efficient Deployment of CNNs on Microcontrollers. In *Advances in System-Integrated Intelligence*, volume 546 of *Lecture Notes in Networks and Systems book series (LNNS)*, pages 543–552, Genova, Italy, September 2022. Springer International Publishing.

2022 Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, Thomas Hannagan, and David Bertrand. Élagage de réseaux profond de neurones par dégradation sélective des pondérations. In *GRETSI 2022 : 28ème colloque du Groupement de Recherche en Traitement du Signal et des Images*, Nancy, France, September 2022.

2022 Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, David Bertrand, and Thomas Hannagan. Leveraging Structured Pruning of Convolutional Neural Networks. In *2022 IEEE Workshop on Signal Processing Systems (SiPS)*, pages 1–6, Rennes, France, November 2022.

2022 Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, David Bertrand, and Thomas Hannagan. Investigating the Not-So-Obvious Effects of Structured Pruning. In *ICML 2022 - Hardware-aware efficient training (HAET)*, Baltimore, United States, July 2022.

2022 Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, David Bertrand, and Thomas Hannagan. Energy Consumption Analysis of pruned Semantic Segmentation Networks on an Embedded GPU. In *International Conference on System-Integrated Intelligence*, volume 546 of *International Conference on System-Integrated Intelligence*, pages 553–563, Genova, Italy, September 2022. Springer.

2021 **Mathieu Léonardon** and Vincent Gripon. Using Deep Neural Networks to Predict and Improve the Performance of Polar Codes. In *ISTC 2021: 11th IEEE International Symposium on Topics in Coding*, Montréal, Canada, August 2021.

2021 Adrien Cassagne, **Mathieu Léonardon**, Romain Tajan, Camille Leroux, Christophe Jégo, Olivier Aumage, and Denis Barthou. A Flexible and Portable Real-time DVB-S2 Transceiver using Multicore and SIMD CPUs. In *The 11th IEEE International Symposium on Topics in Coding (ISTC 2021)*, Montréal, Canada, August 2021.

2018 **Mathieu Léonardon**, Camille Leroux, Pekka Jaaskelainen, Christophe Jego, and Yvon Savaria. Transport Triggered Polar Decoders. In *2018 IEEE 10th International Symposium on Turbo Codes & Iterative Information Processing (ISTC)*, pages 1–5, Hong Kong, Hong Kong SAR China, December 2018. IEEE.

2018 **Mathieu Léonardon**, Camille Leroux, David Binet, J. M Pierre Langlois, Christophe Jego, and Yvon Savaria. Custom Low Power Processor for Polar Decoding. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2018 IEEE International Symposium on Circuits and Systems (ISCAS), Florence, Italy, May 2018.

2017 Alireza Ghaffari, **Mathieu Léonardon**, Yvon Savaria, Christophe Jego, and Camille Leroux. Improving performance of SCMA MPA decoders using estimation of conditional probabilities. In *2017 15th IEEE International New Circuits and Systems Conference (NEWCAS)*, pages 21–24, Strasbourg, France, June 2017. IEEE.

2017 Adrien Cassagne, Olivier Hartmann, **Mathieu Léonardon**, Thibaud Tonnellier, Guillaume Delbergue, Camille Leroux, Romain Tajan, Bertrand Le Gal, Christophe Jego, Olivier Aumage, and Denis Barthou. Fast Simulation and Prototyping with AFF3CT. In *The 20th International Workshop on Signal Processing Systems (SiPS 2017)*, Lorient, France, October 2017.

Work Experience

2020 – now **Associate Professor**, *IMT Atlantique, Lab-STICC, UMR CNRS 6285*.
2018 – 2019 **ATER**, *Bordeaux INP, ENSEIRB-Matmeca France*.
2015 – 2018 **PhD Student**, *Bordeaux INP, ENSEIRB-Matmeca France*, and *Polytechnique Montréal - Canada*.
2012 – 2015 **Engineer Apprentice**, *Worldcast Systems, France*.

Reviewer

IEEE SIPS, IEEE ISTC, GRETSI, IEEE NEWCAS, IEEE SysInt, MDPI Entropy, Neurips

Computer skills

Programming Languages: C, C++, Python, PyTorch
HDL: VHDL, Vivado HLS
Software: Git, Gitlab CI, Linux, Inkscape

Research Supervising

Ph.D. students

2020-2023 **Hugo Tessier**, IMT Atlantique, Stellantis.
2021-2024 **Hugo Le Blevec**, IMT Atlantique.
2021-2024 **Lucas Grativol**, IMT Atlantique, Telecom Saint Etienne.
2022-present **Timotée Ly-Manson**, IMT Atlantique.
2022-present **Karl La Grassa**, IMT Atlantique, Polytechnique Montréal.
2022-present **Ismail Amessegher**, IMT Atlantique, University of Adelaide.
2025-present **Elise Lagarde**, IMT Atlantique, Télécom Paris.
Arkane

Post-doctoral researchers

2022-2023 **Hamoud Younes**, IMT Atlantique, GoodFloow.
2023-present **Hugo Tessier**, IMT Atlantique.
2024-Present **Hugo Le Blevec**, IMT Atlantique.

Research Engineer

2024-2025 **Manon Renault**, IMT Atlantique.

Teaching

2018-2019 **EN112: Digital Electronics Design**, ENSEIRB-Matmeca.
2018-2019 **EN102: Combinatorial and Sequential Logic**, ENSEIRB-Matmeca.
2018-2019 **EN103: Micro-controller project**, ENSEIRB-Matmeca.
2018-2019 **EN114: Computer Architecture**, ENSEIRB-Matmeca.
2018-2019 **MI202: Micro-controller project**, ENSEIRB-Matmeca.
2018-2019 **PG208: Object-Oriented Programmation with C++**, ENSEIRB-Matmeca.
2020-present **EFFDL: Efficient Deep Learning**, IMT Atlantique, Lead.
2020-present **SEIML: Embedded Systems - Software Hardware Interaction**, IMT Atlantique.
2022-2025 **SECTI: From Sensor to Intelligent Processing**, IMT Atlantique.
2020-present **ParPIng: Parallel Computing for Engineers**, IMT Atlantique, Lead.
2020-2022 **CHLS: High-Level Circuit Design**, IMT Atlantique.

2020-2022 **Electronics**, IMT Atlantique.

2020-2025 **Computer Architecture**, IMT Atlantique.

2020-2025 **Logical and Analog Electronic Functions**, IMT Atlantique.

2020-2025 **Diverse Student Projects**, IMT Atlantique.

Public Fundings Obtained

2024-2026 **ADEME Logistique 4.0**, *260k€*, Autonomous Pack : Decarbonize the supply chain, PI.
ADEME

2024-2026 **PCR**, *140k€*, Spatio-Temporal Processing Optimization for GNSS Anti-Jamming with CRPA Antennas Using Lightweight CNN and TCN Beamformers, Member.
Région Bretagne

2023-2026 **ANR JCJC**, *250k€*, ProPruNN: Profitable Pruning of Neural Networks, Project Lead.
ANR

2023-2026 **ANR TSIA**, *200k€*, Vortex: VisiOn-based Reconfigurable drone swarms for fasT EXploration, Member.
ANR

2024-2025 **R&T CNES**, *5k€*, Deep-Learning-FPGA-Eco-conception-challenge, Project Lead.
CNES

2022-2024 **Labex CominLabs**, *325k€*, Leasard: Low Energy deep neural networks for Autonomous Search-And-Rescue Drones, Member.
Labex

2022-2024 **GDR ISIS**, *7k€*, Furnitures.
CNRS

2021-2024 **Futur et Ruptures**, *120k€*, FLCNNFPGA: Towards an efficient and privacy-protecting IoT through the use of federated learning and FPGA technologies, Member.
IMT Atlantique

Industry Fundings Obtained

2025-2028 **CIFRE Thesis**, Optimisation du traitement spatio-temporel pour l'antibrouillage GNSS des antennes CRPA via des beamformers CNN et TCN légers, Member.
Arkane

2023-2025 **Research Contract**, Analyzing Few-Shot Neural Architecture Search in a Metric-Driven Framework, Member.
Sony

2024-2025 **Research Contract**, Embedded Deep Learning for Semantic Segmentation and LLMs, Project Lead.
Schneider Electric

2020-2023 **CIFRE Thesis**, Convolutional neural networks pruning and its application to embedded vision systems, Member.
Stellantis

2022 **Research Contract**, DeepGEMM: Accelerated Ultra Low-Precision Inference on CPU Architectures using Lookup Tables, Project Lead.
Deeplite

Visiting Researcher

As Guest

2023 **Tohoku University**, *Japan*, 1 month.

2022 **Tampere University**, *Finland*, 2 weeks.

2022 **Sony Stuttgart**, *Germany*, 2 weeks.

A.2 Teaching Activities

Unité d'enseignement	Niveau	Niveau ingénieur	Heures réalisées				Contenu	Contenu produit par M. Léonardon	Responsable	
			CM	TD	TP	Projet				
Conception haut niveau de circuits	M1-M2	2A-3A			46.25		SystemC, HLS		Matthieu Arzel	matthieu.arzel@imt-atlantique.fr
Efficient Deep Learning	M1-M2	2A-3A	75	62.5		162.5	Deep Learning, Compression, Pytorch	https://github.com/brain-bzh/efficient-deep-learning	Mathieu Léonardon	mathieu.leonardon@imt-atlantique.fr
Electronique	L3	1A	2.5	35	51.25		Electronique, Sequentiel et Combinatoire	https://docs.google.com/presentation/d/1wMEa0etI5Z8lxkZleYJvFai2tdVx2doLeIjslf-Sy2EE/edit?usp=sharing	Matthieu Arzel	matthieu.arzel@imt-atlantique.fr
Architecture des ordinateurs	L3	1A	12.5	6.25	12.5		Architecture des ordinateurs, RiscV	Cours & TP & TD (Utilisation Ripes)	Mathieu Léonardon	mathieu.leonardon@imt-atlantique.fr
Fonctions electroniques logiques et analogiques	L3	1A		13.75			Fonctions élémentaires elec		Charlotte Langlais	charlotte.langlais@imt-atlantique.fr
Calcul parallèle pour l'ingénieur	M1-M2	2A-3A	37.5		18.75	81.25	Instruction, Thread, Data Level Parallelism, GPU	Cours, TPs et projet long : https://github.com/bonben/galax_eleves	Mathieu Léonardon	mathieu.leonardon@imt-atlantique.fr
Projets divers	L3-M2	1A-3A				246.25	Suivi Projets 1A, 2A, 3A, entreprise, transverses...			
Méthodologies de conception	M1-M2	2A-3A			11.25		Conception numérique		Amer Baghdadi	amer.baghdadi@imt-atlantique.fr
Systèmes embarqués - interaction hw / sw	M1-M2	2A-3A	12.5		88.75	31.25	Systèmes embarqués, OS, logiciel embarqué	Cours, TPs	Amer Baghdadi	amer.baghdadi@imt-atlantique.fr
Systèmes embarqués : du capteur au traitement intelligent	M1-M2	2A-3A				45	Systèmes embarqués, OS, logiciel embarqué		Stefan Weithoffer	stefan.weithoffer@imt-atlantique.fr

Tableau 1: Enseignements effectués par Mathieu Léonardon à IMT Atlantique entre le 1er février 2020 et le 13 mai 2025

A.3 Publications

A.3.1 PhD Thesis

[1] **Mathieu Léonardon**. "Décodage de codes polaires sur des architectures programmables". Theses. Université de Bordeaux (France) ; Polytechnique Montréal (Québec, Canada), Dec. 2018. URL: https://theses.hal.science/tel-01997553.

A.3.2 Publications in International Journals

[2] Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, Thomas Hannagan, and David Bertrand. "Rethinking Weight Decay for Efficient Neural Network Pruning". In: *Journal of Imaging* 8.3 (Mar. 2022), p. 64. DOI: 10.3390/jimaging8030064. URL: https://hal.science/hal-03675138.

[3] Khaled Alhaj Ali, Amer Baghdadi, Elsa Dupraz, **Mathieu Léonardon**, Mostafa Rizk, and Jean-Philippe Diguet. "MOL-based In-Memory Computing of Binary Neural Networks". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 30.7 (July 2022). DOI: 10.1109/TVLSI.2022.3163233. URL: https://imt-atlantique.hal.science/hal-03659297.

[4] **Mathieu Léonardon**, Adrien Cassagne, Camille Leroux, Christophe Jego, Louis-Philippe Hamelin, and Yvon Savaria. "Fast and Flexible Software Polar List Decoders". In: *Journal of Signal Processing Systems* (Jan. 2019). DOI: 10.1007/s11265-018-1430-3. URL: https://inria.hal.science/hal-01987848.

[5] Alireza Ghaffari, **Mathieu Léonardon**, Adrien Cassagne, Camille Leroux, and Yvon Savaria. "Toward High-Performance Implementation of 5G SCMA Algorithms". In: *IEEE Access* 7 (Jan. 2019), pp. 10402–10414. DOI: 10.1109/ACCESS.2019.2891597. URL: https://hal.science/hal-01977885.

[6] Adrien Cassagne, Olivier Hartmann, **Mathieu Léonardon**, et al. "AFF3CT: A Fast Forward Error Correction Toolbox!" In: *SoftwareX* 10 (July 2019), p. 100345. DOI: 10.1016/j.softx.2019.100345. URL: https://inria.hal.science/hal-02358306.

A.3.3 Publications in International Conferences with Proceedings

[7] Ismail Amessegher, Arthur Gaudard, Kojo Nyamekye Anyinam-Boateng, et al. "Towards Low-Latency Object Detection on Board Reactive Search-and-Rescue Drones". In: *IEEE International Symposium on Safety, Security, and Rescue Robotics*. 2025.

[8] Jérémy Morlier, **Mathieu Léonardon**, and Vincent Gripon. "Input Resolution Downsizing as a Compression Technique for Vision Deep Learning Systems". In: *2025 International Joint Conference on Neural Networks (IJCNN)*. Rome, Italy: IEEE, June 2025, pp. 1–8. DOI: 10.1109/IJCNN64981.2025.11229265. URL: https://imt-atlantique.hal.science/hal-05097338.

[9] Karl La Grassa, André Béliveau, **Mathieu Léonardon**, Jean-Pierre David, Matthieu Arzel, and Yvon Savaria. "A Prototyping Framework for P4-Programmable Traffic Managers". In: *RSP '25: Proceedings of the 36th International Workshop on Rapid System Prototyping*. Taipei, Taiwan: ACM, Sept. 2025, pp. 1–7. DOI: 10.1145/3768617.3770728. URL: https://hal.science/hal-05525884.

[10] Timotée Ly-Manson, **Mathieu Léonardon**, Abdeldjalil Aissa El Bey, Ghouti Boukli Hacene, and Lukas Mauch. "Analyzing Few-Shot Neural Architecture Search in a Metric-Driven Framework". In: *Proceedings of the Third International Conference on Automated Machine Learning, PMLR*. Vol. 256. Paris, France, Sept. 2024, pp. 1–33. URL: https://imt-atlantique.hal.science/hal-04576139.

[11] Manon Renault, Hamoud Younes, Hugo Tessier, Ronan Le Roy, Bastien Pasdeloup, and **Mathieu Léonardon**. "Event Classification of Accelerometer Data for Industrial Package Monitoring with Embedded Deep Learning". In: *COMPSAC 2025: IEEE 49th Annual Computers, Software, and Applications Conference*. Toronto, Canada, July 2025. DOI: 10.1109/COMPSAC65507.2025.00225. URL: https://imt-atlantique.hal.science/hal-05097386.

[12] Ryosuke Kanda, Naoya Onizawa, Hugo Le Blevec, **Mathieu Léonardon**, Vincent Gripon, and Tetsuya Hanyu. "Bit-Width-Aware Design Environment for Few-Shot Learning on Edge Ai Hardware". In: *ISCAS 2025: IEEE International Symposium on Circuits and Systems*. London, United Kingdom, 2025.

[13] Lucas Grativol, Lubin Gauthier, **Mathieu Léonardon**, et al. "PEFSL: A deployment Pipeline for Embedded Few-Shot Learning on a FPGA SoC". In: *ISCAS 2024: IEEE International Symposium on Circuits and Systems*. Singapore, Singapore, May 2024. DOI: 10.1109/ISCAS58744.2024.10557995. URL: https://hal.science/hal-04559365.

[14] R. Kanda, N. Onizawa, **Mathieu Léonardon**, Vincent Gripon, and T. Hanyu. "Design Environment of Quantization-Aware Edge AI Hardware for Few-Shot Learning". In: *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*. Springfield, France: IEEE, Aug. 2024, pp. 928–931. DOI: 10.1109/MWSCAS60917.2024.10658789. URL: https://hal.science/hal-04984380.

[15] Hugo Le Blevec, **Mathieu Léonardon**, Hugo Tessier, and Matthieu Arzel. "Pipelined Architecture for a Semantic Segmentation Neural Network on FPGA". In: *ICECS 2023: IEEE 30th International Conference on Electronics, Circuits and Systems*. Istanbul, Turkey, Dec. 2023. DOI: 10.1109/ICECS58634.2023.10382715. URL: https://hal.science/hal-04262138.

[16] Lucas Grativol, **Mathieu Léonardon**, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. "Federated learning compression designed for lightweight communications". In: *ICECS 2023: IEEE 30th International Conference on Electronics, Circuits and Systems*. Istanbul, Turkey, Dec. 2023. DOI: 10.1109/ICECS58634.2023.10382717. URL: https://hal.science/hal-04251969.

[17] Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, David Bertrand, and Thomas Hannagan. "Leveraging Structured Pruning of Convolutional Neural Networks". In: *2022 IEEE Workshop on Signal Processing Systems (SiPS)*. Rennes, France, Nov. 2022, pp. 1–6. DOI: 10.1109/SiPS55645.2022.9919253. URL: https://hal.science/hal-03767634.

[18] Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, David Bertrand, and Thomas Hannagan. "Energy Consumption Analysis of pruned Semantic Segmentation Networks on an Embedded GPU". In: *International Conference on System-Integrated Intelligence*. Vol. 546. International Conference on System-Integrated Intelligence. Springer. Genova, Italy, Sept. 2022, pp. 553–563. DOI: 10.1007/978-3-031-16281-7_52. URL: https://hal.science/hal-03767636.

[19] Hamoud Younes, Hugo Le Blevec, **Mathieu Léonardon**, and Vincent Gripon. "Inter-Operability of Compression Techniques for Efficient Deployment of CNNs on Microcontrollers". In: *Advances in System-Integrated Intelligence*. Vol. 546. Lecture Notes in Networks and Systems book series (LNNS). Genova, Italy: Springer International Publishing, Sept. 2022, pp. 543–552. DOI: 10.1007/978-3-031-16281-7_51. URL: https://hal.science/hal-03850858.

[20] Adrien Cassagne, **Mathieu Léonardon**, Romain Tajan, et al. "A Flexible and Portable Real-time DVB-S2 Transceiver using Multicore and SIMD CPUs". In: *The 11th IEEE International Symposium on Topics in Coding (ISTC 2021)*. Montréal, Canada, Aug. 2021. DOI: 10.1109/ISTC49272.2021.9594063. URL: https://hal.science/hal-03336450.

[21] **Mathieu Léonardon**, Camille Leroux, Pekka Jaaskelainen, Christophe Jego, and Yvon Savaria. "Transport Triggered Polar Decoders". In: *2018 IEEE 10th International Symposium on Turbo Codes & Iterative Information Processing (ISTC)*. Hong Kong, Hong Kong SAR China: IEEE, Dec. 2018, pp. 1–5. DOI: 10.1109/ISTC.2018.8625310. URL: https://hal.science/hal-03291077.

[22] **Mathieu Léonardon**, Camille Leroux, David Binet, J. M Pierre Langlois, Christophe Jego, and Yvon Savaria. "Custom Low Power Processor for Polar Decoding". In: *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. 2018 IEEE International Symposium on Circuits and Systems (ISCAS). Florence, Italy, May 2018. URL: https://hal.science/hal-01978830.

[23] Adrien Cassagne, Olivier Hartmann, **Mathieu Léonardon**, et al. "Fast Simulation and Prototyping with AFF3CT". In: *The 20th International Workshop on Signal Processing Systems (SiPS 2017)*. Lorient, France, Oct. 2017. DOI: 10.13140/RG.2.2.10295.42409/1. URL: https://hal.science/hal-01965633.

[24] Alireza Ghaffari, **Mathieu Léonardon**, Yvon Savaria, Christophe Jego, and Camille Leroux. "Improving performance of SCMA MPA decoders using estimation of conditional probabilities". In: *2017 15th IEEE International New Circuits and Systems Conference (NEWCAS)*. Strasbourg, France: IEEE, June 2017, pp. 21–24. DOI: 10.1109/NEWCAS.2017.8010095. URL: https://hal.science/hal-03762203.

A.3.4 Publications in National Conferences with Proceedings

[25] Jérémy Morlier, **Mathieu Léonardon**, and Vincent Gripon. "Réduction de la Résolution comme Technique de Compression pour les Systèmes d'Apprentissage Profond en Vision". In: *GRETSI*. GRETSI. Strasbourg, France, Aug. 2025. URL: https://imt-atlantique.hal.science/hal-05497988.

[26] Hugo Le Blevec, **Mathieu Léonardon**, Hugo Tessier, and Matthieu Arzel. "Co-conception d'un réseau de neurones convolutif de segmentation sémantique et de son architecture matérielle". In: *29° Colloque sur le traitement du signal et des images*. GRETSI - Groupe de Recherche en Traitement du Signal et des Images. Grenoble, France, Aug. 2023. URL: https://hal.science/hal-04705594.

[27] Lucas Grativol, **Mathieu Léonardon**, Guillaume Muller, Virginie Fresse, and Matthieu Arzel. "Compression de réseaux de neurones pour l'apprentissage fédéré". In: *GRETSI 2023 : XXIXème Colloque Francophone de Traitement du Signal et des Images*. Grenoble, France, Aug. 2023. URL: https://hal-emse.ccsd.cnrs.fr/emse-04217775.

[28] Hugo Tessier, Vincent Gripon, **Mathieu Léonardon**, Matthieu Arzel, Thomas Hannagan, and David Bertrand. "Élagage de réseaux profond de neurones par dégradation sélective des pondérations". In: *GRETSI 2022 : 28ème colloque du Groupement de Recherche en Traitement du Signal et des Images*. Nancy, France, Sept. 2022. URL: https://hal.science/hal-03695958.

A.4 Summary of Supervision Activities

A.4.1 Doctoral Student Supervision

Theses defended:

- Hugo Tessier, defended on 09/02/2023
  - 25% - Co-supervision of this CIFRE thesis with Stellantis.Participation in Hugo's thesis covered all research work, with a particular focus on deployment on embedded targets. In collaboration with Thomas Hannagan and David Bertrand from Stellantis, Matthieu Arzel and Vincent Gripon from IMT Atlantique.
- Lucas Grativol Ribeiro, defended on 05/12/2024
  - 25% - Participation in the design of this Futur et Ruptures thesis with colleagues from Telecom and Mines Saint Etienne. Provided expertise on the choice of learning models and the application of compression techniques. In collaboration with Matthieu Arzel from IMT Atlantique and Virginie Fresse from Telecom Saint Etienne, directors, and Guillaume Muller from Mines Saint Etienne, co-supervisor
- Hugo Le Blevec, defended on 06/12/2024
  - 50% - Co-supervision of Hugo Le Blevec's thesis. Participation in all research work, particularly initiating collaborations with the AMD team in Dublin (Thomas Preusser). Matthieu Arzel from IMT Atlantique was the director.
- Timotée Ly-Manson, defensed on 09/03/2025
  - 50% - Co-supervision of Timotée Ly-Manson's thesis, funded by my research contracts and subject designed with him. Collaboration established with the Sony Stuttgart team, particularly Lukas Mauch. Abdeldjalil Aissa El Bey from IMT Atlantique was the director.

Theses in progress:

- Karl La Grassa, defense scheduled for May 2026
  - 25% - Initiated this thesis in codirectorship with Polytechnique Montréal, with a funding of IMT Atlantique.
- Ismail Amessegher, defense scheduled for June 2026
  - 33% - Co-supervision of Ismail Amessegher's thesis, with involvement in the deployment of object detection models on FPGA.
- Jérémy Morlier, defense scheduled for December 2026
  - 50% - Initiated of Jérémy Morlier's thesis, funded by my JCJC ProPruNN project.
- Elise Lagarde, since January 2025
  - 50% - Initiated this CIFRE thesis in collaboration with Arkane and Telecom Paris, following a previous research contract.

- Kaiteng Jiang, since January 2025
  - 50% - Co-supervision of this thesis in collaboration with Sony Stuttgart, through a european COFUND funding ("SEED"). Nicolas Farrugia from IMT Atlantique is the director, Axel Marmoret from IMT Atlantique is a co-supervisor. This thesis is on music generation with deep learning, with a particular focus on efficient strategies for training large generative models on constrained hardware.
- Arthur Gaudard, since January 2026
  - 33% - Co-supervision of this thesis in the PEPR CAMELIA (Composants pour l'Accélération Matérielle Et Logicielle de l'IA) program, funded by the ANR.

A.4.2 Master's Internship Supervision

- Hugo Kermabon, May to August 2020
- Hugo Le Blevec, April to October 2020
- Jérémy Morlier, June to July 2021
- Lubin Gauthier, July to September 2023
- Pedro Figueireido, May to August 2025
- Mohammed El Houcine Ayoubi, April to August 2025

A.4.3 Postdoctoral Supervision

- Hamoud Younes, 2021 to 2022

A.4.4 Research Engineers

- Manon Renault, 2023 to 2025
- Tévchhorpoan Khieu, 2026 (may start a PhD with Erwan Piriou from CEA List and myself as director, late 2026)

A.5 Research Funding

A.5.1 Public Funding

Table A.1: Public Funding

Year	Short Title	Organization	Role	Amount (k€)	Details
2023-2027	Autonomous Pack	ADEME Logistique 4.0	PI	260	Principal investigator of the project. Expertise on embedding neural networks on accelerometer data on microcontrollers
2024-2026	GNSS Anti-Jamming w/ CRPA & CNN/TCN	Région Bretagne	co-PI	140	Participation in the project supervising a post-doctoral researcher on porting embedded learning processing to FPGA
2023-2026	ProPruNN: Pruning Neural Networks	ANR JCJC	PI	250	Principal investigator of my JCJC project on the co-design of deep learning models and hardware architectures
2023-2026	Vortex: Drone Swarms for Exploration	ANR TSIA	co-PI	200	Supervision of a doctoral student, Ismail, on the implementation of object detection algorithms on event data
2024-2025	Deep-Learning-FPGA-Eco-conception	CNES	PI	5	Responsible for a training course on neural network compression for the company Arkane
2021-2025	GoodFloow	ADEME Perfecto	PI	70	Principal investigator of the project for IMT Atlantique, by supervising a post-doctoral researcher, Hamoud Younes
2022-2024	Leasard: Low Energy DNN for Drones	Labex CominLabs	co-PI	120	Supervision of a doctoral student, Ismail, on the implementation of object detection algorithms on event data
2022-2024	FLCNNFPGA: Federated Learning	CNRS GDR ISIS	PI	7	Funding for a master's internship and equipment as part of Lucas Grativol's thesis
2021-2024	FLCNNFPGA: Federated Learning & FPGA	IMT Futur & Ruptures	co-PI	120	Supervision of Lucas Grativol

A.5.2 Private Funding

Table A.2: Private Funding

Year	Short Title	Organization	Role	Amount (k€)	Details
2025-2028	GNSS Anti-Jamming Opt. (CIFRE)	Arkane	co-PI	pvt	Initiator of Elise Lagarde's CIFRE thesis, responsible for the FPGA target implementation part
2023-2025	Few-Shot NAS in Metric-Driven Fmwk	Sony	co-PI	180k€	Project with Sony on many topics including few-shot learning and LLMs. For my part, I participated in the writing of an article on NAS
2024-2025	Embedded DL for Seg. & LLMs	Schneider Electric	PI	40k€	Principal investigator of this project, which focused on the integration of semantic segmentation networks on embedded targets on the one hand, and the implementation of RAG (retrieval-augmented generation) on embedded targets on the other.
2020-2023	CNN Pruning for Embedded Vision (CIFRE)	Stellantis	co-PI	pvt	Participated in the supervision of Hugo Tessier upon my arrival at IMT Atlantique
2022	DeepGEMM: Low-Precision Inference	Deeplite	PI	60k€	Principal investigator and lead developer of a quantized neural network implementation on embedded targets using LUTs and SIMD instructions.

A.6 References

[29] E. Arıkan. "Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels". In: *IEEE Transactions on Information Theory (TIT)* 55.7 (July 2009), pp. 3051–3073. ISSN: 0018-9448. DOI: 10.1109/TIT.2009.2021379.

[30] Ilshat Sagitov, Charles Pillet, Alexios Balatsoukas-Stimming, and Pascal Giard. "Generalized Restart Mechanism for Successive-Cancellation Flip Decoding of Polar Codes". In: *Journal of Signal Processing Systems* (Apr. 2025). ISSN: 1939-8115. DOI: 10.1007/s11265-025-01949-8. URL: https://doi.org/10.1007/s11265-025-01949-8.

[31] I. Tal and A. Vardy. "List Decoding of Polar Codes". In: *IEEE Transactions on Information Theory (TIT)* 61.5 (May 2015), pp. 2213–2226. ISSN: 0018-9448. DOI: 10.1109/TIT.2015.2410251.

[32] B. Li, H. Shen, and D. Tse. "An Adaptive Successive Cancellation List Decoder for Polar Codes with Cyclic Redundancy Check". In: *IEEE Communications Letters (COMML)* 16.12 (Dec. 2012), pp. 2044–2047. ISSN: 1089-7798. DOI: 10.1109/LCOMM.2012.111612.121898.

[33] G. Sarkis, P. Giard, A. Vardy, C. Thibeault, and W. J. Gross. "Fast List Decoders for Polar Codes". In: *IEEE Journal on Selected Areas in Communications (JSAC)* 34.2 (Feb. 2016), pp. 318–328. ISSN: 0733-8716. DOI: 10.1109/JSAC.2015.2504299.

[34] A. Alamdar-Yazdi and F. R. Kschischang. "A Simplified Successive-Cancellation Decoder for Polar Codes". In: *IEEE Communications Letters (COMML)* 15.12 (Dec. 2011), pp. 1378–1380. ISSN: 1089-7798. DOI: 10.1109/LCOMM.2011.101811.111480.

[35] G. Sarkis, P. Giard, A. Vardy, C. Thibeault, and W. J. Gross. "Fast Polar Decoders: Algorithm and Implementation". In: *IEEE Journal on Selected Areas in Communications (JSAC)* 32.5 (May 2014), pp. 946–957. ISSN: 0733-8716. DOI: 10.1109/JSAC.2014.140514.

[36] J. Schreier. "On Tournament Elimination Systems". In: *Mathesis Polska* 7 (1932), pp. 154–160. URL: https://ci.nii.ac.jp/naid/10027928626/en/.

[37] Y. Shen, C. Zhang, J. Yang, S. Zhang, and X. You. "Low-Latency Software Successive Cancellation List Polar Decoder using Stage-Located Copy". In: *International Conference on Digital Signal Processing (DSP)*. IEEE, Oct. 2016, pp. 84–88. DOI: 10.1109/ICDSP.2016.7868521.

[38] P. Trifonov. "Efficient Design and Decoding of Polar Codes". In: *IEEE Transactions on Communications (TCOM)* 60.11 (Nov. 2012), pp. 3221–3227. ISSN: 0090-6778. DOI: 10.1109/TCOMM.2012.081512.110872.

[39] ETSI. *3GPP - TS 38.212 - Multiplexing and Channel Coding (R. 15)*. Aug. 2018. URL: https://www.etsi.org/deliver/etsi_ts/138200_138299/138212/15.02.00_60/ts_138212v150200p.pdf.

[40] T. Tonnellier, C. Leroux, B. Le Gal, C. Jégo, B. Gadat, and N. Van Wambeke. "Hardware Architecture for Lowering the Error Floor of LTE Turbo Codes". In: *Design and Architectures for Signal and Image Processing Conference (DASIP)*. IEEE, Oct. 2016, pp. 107–112. DOI: 10.1109/DASIP.2016.7853805.

[41] T. Tonnellier, C. Leroux, B. Le Gal, C. Jégo, B. Gadat, and N. Van Wambeke. "Lowering the Error Floor of Double-Binary Turbo Codes: The Flip and Check Algorithm". In: *International Symposium on Turbo Codes and Iterative Information Processing (ISTC)*. IEEE, Sept. 2016, pp. 156–160. DOI: 10.1109/ISTC.2016.7593096.

[42] T. Tonnellier, C. Leroux, B. Le Gal, B. Gadat, C. Jégo, and N. Van Wambeke. "Lowering the Error Floor of Turbo Codes With CRC Verification". In: *IEEE Wireless Communications Letters (WCL)* 5.4 (Aug. 2016), pp. 404–407. ISSN: 2162-2337. DOI: 10.1109/LWC.2016.2571283.

[43] B. Le Gal, C. Leroux, and C. Jégo. "Multi-Gb/s Software Decoding of Polar Codes". In: *IEEE Transactions on Signal Processing (TSP)* 63.2 (Jan. 2015), pp. 349–359. ISSN: 1053-587X. DOI: 10.1109/TSP.2014.2371781.

[44] A. Cassagne, B. Le Gal, C. Leroux, O. Aumage, and D. Barthou. "An Efficient, Portable and Generic Library for Successive Cancellation Decoding of Polar Codes". In: *International Workshop on Languages and Compilers for Parallel Computing (LCPC)*. Springer, Nov. 1, 2015. DOI: 10.1007/978-3-319-29778-1_19.

[45] A. Cassagne, O. Aumage, C. Leroux, D. Barthou, and B. Le Gal. "Energy Consumption Analysis of Software Polar Decoders on Low Power Processors". In: *European Signal Processing Conference (EUSIPCO)*. IEEE, Aug. 2016, pp. 642–646. DOI: 10.1109/EUSIPCO.2016.7760327.

[46] A. Cassagne, T. Tonnellier, C. Leroux, B. Le Gal, O. Aumage, and D. Barthou. "Beyond Gbps Turbo decoder on multi-core CPUs". In: *International Symposium on Turbo Codes and Iterative Information Processing (ISTC)*. IEEE, Sept. 2016, pp. 136–140. DOI: 10.1109/ISTC.2016.7593092.

[47] P. Giard, A. Balatsoukas-Stimming, T. C. Müller, et al. "PolarBear: A 28-nm FD-SOI ASIC for Decoding of Polar Codes". In: *Journal on Emerging and Selected Topics in Circuits and Systems* 7.4 (Dec. 2017), pp. 616–629. ISSN: 2156-3357. DOI: 10.1109/JETCAS.2017.2745704.

[48] Henk Corporaal. *Microprocessor Architectures: From VLIW to TTA*. New York, NY, USA: John Wiley & Sons, Inc., 1997. ISBN: 978-0-471-97157-3.

[49] Otto Esko, Pekka Jääskeläinen, Pablo Huerta, Carlos S. de La Lama, Jarmo Takala, and Jose Ignacio Martinez. "Customized Exposed Datapath Soft-Core Design Flow with Compiler Support". In: *IEEE FPL*. 2010.

[50] D. Tabak and G. J. Lipovski. "MOVE Architecture in Digital Controllers". In: *IEEE Journal of Solid-State Circuits* (1980).

[51] Chris Lattner and Vikram Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation". In: *CGO*. 2004. ISBN: 978-0-7695-2102-2.

[52] A. Mishra, A. J. Raymond, L. G. Amaru, et al. "A Successive Cancellation Decoder ASIC for a 1024-bit Polar Code in 180nm CMOS". In: *IEEE A-SSCC*. Nov. 2012. DOI: 10.1109/IPEC.2012.6522661.

[53] Jakub Konečnỳ, Brendan McMahan, and Daniel Ramage. "Federated optimization: Distributed optimization beyond the datacenter". In: *arXiv preprint arXiv:1511.03575* (2015).

[54] Peter Kairouz and et al. "Advances and open problems in federated learning". In: *Foundations and Trends® in Machine Learning* 14.1–2 (2021), pp. 1–210.

[55] Brendan McMahan and et al. "Communication-efficient learning of deep networks from decentralized data". In: *Artificial intelligence and statistics*. PMLR. 2017, pp. 1273–1282.

[56] Xinchi Qiu, Titouan Parcollet, Javier Fernandez-Marques, et al. "A first look into the carbon footprint of federated learning". In: *Journal of Machine Learning Research* 24.129 (2023), pp. 1–23.

[57] Xinchi Qiu, Javier Fernandez-Marques, Pedro PB Gusmao, Yan Gao, Titouan Parcollet, and Nicholas Donald Lane. "Zerofl: Efficient on-device training for federated learning with local sparsity". In: *arXiv preprint arXiv:2208.02507* (2022).

[58] H. Nikopour and H. Baligh. "Sparse Code Multiple Access". In: *International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC)*. IEEE, Sept. 2013, pp. 332–336. DOI: 10.1109/PIMRC.2013.6666156.

[59] T. Aulin and C. Sundberg. "Continuous Phase Modulation - Part I: Full Response Signaling". In: *IEEE Transactions on Communications (TCOM)* 29.3 (Mar. 1981), pp. 196–209. ISSN: 0090-6778. DOI: 10.1109/TCOM.1981.1095001.

[60] T. Aulin, N. Rydbeck, and C. Sundberg. "Continuous Phase Modulation - Part II: Partial Response Signaling". In: *IEEE Transactions on Communications (TCOM)* 29.3 (Mar. 1981), pp. 210–225. ISSN: 0090-6778. DOI: 10.1109/TCOM.1981.1094985.

[61] Walter Murray and Kien-Ming Ng. "An algorithm for nonlinear optimization problems with binary variables". In: *Computational optimization and applications* 47.2 (2010), pp. 257–288.

[62] Hugo Tessier. "Convolutional neural networks pruning and its application to embedded vision systems". Theses. Ecole nationale supérieure Mines-Télécom Atlantique, Feb. 2023. URL: https://theses.hal.science/tel-04064137.

[63] Song Han, Jeff Pool, John Tran, and William Dally. "Learning both weights and connections for efficient neural network". In: *Advances in neural information processing systems* 28 (2015).

[64] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. "Learning efficient convolutional networks through network slimming". In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2736–2744.

[65] Hidenori Tanaka, Daniel Kunin, Daniel L Yamins, and Surya Ganguli. "Pruning neural networks without any data by iteratively conserving synaptic flow". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 6377–6389.

[66] Alex Renda, Jonathan Frankle, and Michael Carbin. "Comparing rewinding and fine-tuning in neural network pruning". In: *arXiv preprint arXiv:2003.02389* (2020).

[67] Adam Paszke, Abhishek Chaurasia, Sangpil Kim, and Eugenio Culurciello. *ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation*. 2016. arXiv: 1606.02147 [cs.CV]. URL: https://arxiv.org/abs/1606.02147.

[68] Sachin Mehta, Mohammad Rastegari, Anat Caspi, Linda Shapiro, and Hannaneh Hajishirzi. *ESPNet: Efficient Spatial Pyramid of Dilated Convolutions for Semantic Segmentation*. 2018. arXiv: 1803.06815 [cs.CV]. URL: https://arxiv.org/abs/1803.06815.

[69] Gen Li, Inyoung Yun, Jonghyun Kim, and Joongkyu Kim. *DABNet: Depth-wise Asymmetric Bottleneck for Real-time Semantic Segmentation*. 2019. arXiv: 1907.11357 [cs.CV]. URL: https://arxiv.org/abs/1907.11357.

[70] Shao-Yuan Lo, Hsueh-Ming Hang, Sheng-Wei Chan, and Jing-Jhih Lin. *Efficient Dense Modules of Asymmetric Convolution for Real-Time Semantic Segmentation*. 2019. arXiv: 1809.06323 [cs.CV]. URL: https://arxiv.org/abs/1809.06323.

[71] Rudra P K Poudel, Stephan Liwicki, and Roberto Cipolla. *Fast-SCNN: Fast Semantic Segmentation Network*. 2019. arXiv: 1902.04502 [cs.CV]. URL: https://arxiv.org/abs/1902.04502.

[72] Yu Wang, Quan Zhou, and Xiaofu Wu. *ESNet: An Efficient Symmetric Network for Real-time Semantic Segmentation*. 2019. arXiv: 1906.09826 [cs.CV]. URL: https://arxiv.org/abs/1906.09826.

[73] Ross Wightman, Hugo Touvron, and Herve Jegou. "ResNet strikes back: An improved training procedure in timm". In: *NeurIPS 2021 Workshop on ImageNet: Past, Present, and Future*. 2021. URL: https://openreview.net/forum?id=NG6MJnVl6M5.

[74] Nicolò Ghielmetti, Vladimir Loncar, Maurizio Pierini, et al. *Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml*. 2022. DOI: 10.1088/2632-2153/ac9cb5. URL: https://dx.doi.org/10.1088/2632-2153/ac9cb5.

[75] Weisheng Jia, Jinling Cui, Xin Zheng, and Qiang Wu. "Design and Implementation of Real-Time Semantic Segmentation Network Based on FPGA". In: *2021 7th International Conference on Computing and Artificial Intelligence*. ICCAI 2021. 2021, pp. 321–325. ISBN: 9781450389501. DOI: 10.1145/3467707.3467756.

[76] Pierpaolo Morì, Manoj-Rohit Vemparala, Nael Fasfous, et al. "Accelerating and pruning CNNs for semantic segmentation on FPGA". In: *Proceedings of the 59th ACM/IEEE Design Automation Conference*. DAC '22. San Francisco, California: Association for Computing Machinery, 2022, pp. 145–150. ISBN: 9781450391429. DOI: 10.1145/3489517.3530424. URL: https://doi.org/10.1145/3489517.3530424.

[77] Shaoxia Fang, Lu Tian, Junbin Wang, et al. "Real-Time Object Detection and Semantic Segmentation Hardware System with Deep Learning Networks". In: *2018 International Conference on Field-Programmable Technology (FPT)*. 2018, pp. 389–392. DOI: 10.1109/FPT.2018.00081.

[78] Youki Sada, Masayuki Shimoda, Akira Jinguji, and Hiroki Nakahara. "A Dataflow Pipelining Architecture for Tile Segmentation with a Sparse MobileNet on an FPGA". In: *2019 International Conference on Field-Programmable Technology (ICFPT)*. Dec. 2019, pp. 267–270. DOI: 10.1109/ICFPT47387.2019.00044.

[79] Masayuki Miyama. "FPGA Implementation of 3-bit Quantized CNN for Semantic Segmentation". In: *Journal of Physics: Conference Series* 1729.1 (2021), p. 012004. DOI: 10.1088/1742-6596/1729/1/012004. URL: https://dx.doi.org/10.1088/1742-6596/1729/1/012004.

[80] Masayuki Shimoda, Youki Sada, and Hiroki Nakahara. "Filter-Wise Pruning Approach to FPGA Implementation of Fully Convolutional Network for Semantic Segmentation". In: *Applied Reconfigurable Computing*. Ed. by Christian Hochberger, Brent Nelson, Andreas Koch, Roger Woods, and Pedro Diniz. Cham: Springer International Publishing, 2019, pp. 371–386. ISBN: 978-3-030-17227-5.

[81] Mengqi Yu, Hongzhi Huang, Hong Liu, et al. "Optimizing FPGA-based Convolutional Encoder-Decoder Architecture for Semantic Segmentation". In: *2019 IEEE 9th Annual International Conference on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER)*. 2019, pp. 1436–1440. DOI: 10.1109/CYBER46603.2019.9066759.

[82] Hyunwoo Kim, Jeonghoon Kim, Jungwook Choi, Jungkeol Lee, and Yong Ho Song. "Binarized Encoder-Decoder Network and Binarized Deconvolution Engine for Semantic Segmentation". In: *IEEE Access* 9 (2021), pp. 8006–8027. DOI: 10.1109/ACCESS.2020.3048375.

[83] Teodor Fredriksson, David Issa Mattos, Jan Bosch, and Helena Holmström Olsson. "Data labeling: An empirical investigation into industrial challenges and mitigation strategies". In: *International Conference on Product-Focused Software Process Improvement*. Springer. 2020, pp. 202–216.

[84] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. "A survey of transfer learning". In: *Journal of Big data* 3.1 (2016), pp. 1–40.

[85] Yassir Bendou, Yuqing Hu, Raphael Lafargue, et al. "Easy—ensemble augmented-shot-y-shaped learning: State-of-the-art few-shot classification with simple components". In: *Journal of Imaging* 8.7 (2022), p. 179.

[86] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. "Survey and benchmarking of machine learning accelerators". In: *2019 IEEE high performance extreme computing conference (HPEC)*. IEEE. 2019, pp. 1–9.

[87] Xiaowei Xu, Xinyi Zhang, Bei Yu, et al. "DAC-SDC Low Power Object Detection Challenge for UAV Applications". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.2 (2021), pp. 392–403. DOI: 10.1109/TPAMI.2019.2932429.

[88] Jinming Zhang, Lifu Cheng, Cen Li, et al. "A low-latency FPGA implementation for real-time object detection". In: *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE. 2021, pp. 1–5.

[89] Spyros Gidaris, Andrei Bursuc, Nikos Komodakis, Patrick Pérez, and Matthieu Cord. "Boosting few-shot visual learning with self-supervision". In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 8059–8068.

[90] Steinar Laenen and Luca Bertinetto. "On episodes, prototypical networks, and few-shot learning". In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 24581–24592.

[91] Tyler Scott, Karl Ridgeway, and Michael C Mozer. "Adapted deep embeddings: A synthesis of methods for k-shot inductive transfer learning". In: *Advances in Neural Information Processing Systems* 31 (2018).

[92] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[93] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. "A survey of convolutional neural networks: analysis, applications, and prospects". In: *IEEE transactions on neural networks and learning systems* (2021).

[94] Yang Li and Jiachen Yang. "Few-shot cotton pest recognition and terminal realization". In: *Computers and Electronics in Agriculture* 169 (2020), p. 105240.

[95] Alex Krizhevsky, Geoffrey Hinton, et al. "Learning multiple layers of features from tiny images". In: (2009).

[96] Hendrik Borras, Giuseppe Di Guglielmo, Javier Duarte, et al. "Open-source FPGA-ML codesign for the MLPerf Tiny Benchmark". In: *arXiv preprint arXiv:2206.11791* (2022).

[97] Li Yang, Zhezhi He, and Deliang Fan. "A fully onchip binarized convolutional neural network fpga impelmentation with accurate inference". In: *Proceedings of the International Symposium on Low Power Electronics and Design*. 2018, pp. 1–6.

[98] Heekyung Kim and Kyuwon Ken Choi. "A Reconfigurable CNN-based Accelerator Design for Fast and Energy-Efficient Object Detection System on Mobile FPGA". In: *IEEE Access* (2023).

[99] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. "SMOTE: Synthetic Minority Over-sampling Technique". In: *Journal of Artificial Intelligence Research* 16 (June 1, 2002), pp. 321–357. ISSN: 1076-9757. DOI: 10.1613/jair.953. URL: https://www.jair.org/index.php/jair/article/view/10302 (visited on 09/30/2024).

[100] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. "ADASYN: Adaptive synthetic sampling approach for imbalanced learning". In: *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*. Ieee. 2008, pp. 1322–1328.

[101] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. "What is the state of neural network pruning?" In: *Proceedings of machine learning and systems* 2 (2020), pp. 129–146.

[102] Andes Technology Corporation. *Andes Custom Extension (ACE)*. http://www.andestech.com/en/products-solutions/andes-custom-extension-ace/. Accessed: 2025-01-15. 2024.

[103] DeepMind. *AlphaEvolve: Automated Discovery of Learning Algorithms and Architectures*. Google DeepMind Blog. 2025. URL: https://deepmind.google/technologies/alphaevolve/.

[104] Vernon J Lawhern, Amelia J Solon, Nicholas R Waytowich, Stephen M Gordon, Chou P Hung, and Brent J Lance. "EEGNet: a compact convolutional neural network for EEG-based brain–computer interfaces". In: *Journal of Neural Engineering* 15.5 (2018), p. 056013.

[105] Michaela Blott, Thomas B Preusser, Nicholas J Fraser, et al. "FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks". In: *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*. Vol. 11. 4. ACM New York, NY, USA, 2018, pp. 1–23.

[106] Yaman Umuroglu, Nicholas J Fraser, Giulio Gambardella, et al. "FINN: A framework for fast, scalable binarized neural network inference". In: *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 2017, pp. 65–74.

[107] Alessandro Pappalardo. *Xilinx/brevitas*. 2021. URL: https://github.com/Xilinx/brevitas.

[108] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Daan Wierstra, et al. "Matching networks for one shot learning". In: *Advances in neural information processing systems*. Vol. 29. 2016.

[109] Arfon M Smith, Daniel S Katz, Kyle E Niemeyer, FORCE11 Software Citation Working Group, et al. "Software citation principles". In: *PeerJ Computer Science* 2 (2016), e86.

[110] Yassine El Ouahidi, Jonathan Lys, Philipp Thölke, et al. "REVE: A Foundation Model for EEG: Adapting to Any Setup with Large-Scale Pretraining on 25,000 Subjects". In: *Advances in Neural Information Processing Systems* (2025). URL: https://brain-bzh.github.io/reve/.

[111] Luca Benini et al. "EEGformer: Transformer-Based Epilepsy Detection on Raw EEG Traces for Low-Channel-Count Wearable Continuous Monitoring Devices". In: *IEEE Transactions on Biomedical Circuits and Systems* (2023).

[112] Soroush Abbasi Koohpayegani and Hamed Pirsiavash. "Sima: Simple softmax-free attention for vision transformers". In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2024, pp. 2607–2617.

[113] Navid Nikaein, Mahesh K Marina, Saravana Manickam, Alex Dawson, Raymond Knopp, and Christian Bonnet. "OpenAirInterface: A flexible platform for 5G research". In: *ACM SIGCOMM Computer Communication Review* 44.5 (2014), pp. 33–38.

[114] Naveen Verma, Ali Shoeb, Jose Bohorquez, Joel Dawson, John Guttag, and Anantha P. Chandrakasan. "A Micro-Power EEG Acquisition SoC With Integrated Feature Extraction Processor for a Chronic Seizure Detection System". In: *IEEE Journal of Solid-State Circuits* 45.4 (2010), pp. 804–816.

[115] Sherif Moussa, Ahmed M. Abdel Razik, Adel Omar Dahmane, and Habib Hamam. "FPGA Implementation of floating-point complex matrix inversion based on Gauss-Jordan elimination". In: *IEEE Canadian Conference Of Electrical And Computer Engineering*. 2013.

[116] Parth Desai, Semih Aslan, and Jafar Saniie. "FPGA implementation of Gram-Schmidt QR decomposition using high level synthesis". In: *2017 IEEE International Conference on Electro Information Technology (EIT)*. IEEE. 2017, pp. 482–487.

[117] Marjan Karkooti, Joseph Cavallaro, and Chris Dick. "FPGA implementation of matrix inversion using QRD-RLS algorithm". In: *Circuits, Systems and Computers, 1977. Conference Record. 1977 11th Asilomar Conference on* 2005 (Jan. 2005). DOI: 10.1109/ACSSC.2005.1600043.

[118] Jun Luo, Qijun Huang, Sheng Chang, Xiaoying Song, and Yun Shang. "High throughput Cholesky decomposition based on FPGA". In: *2013 6th International Congress on Image and Signal Processing (CISP)*. Vol. 3. IEEE. 2013, pp. 1649–1653.

[119] Yi Yang, Wen-mei Jiang, and Xiaohui Chen. "Compressed Sensing and Cholesky Decomposition on FPGAs and GPUs". In: *Journal of Signal Processing Systems* (2012).

[120] Wei Li, Zhiyong Lei, Gang Wu, and Zhongping Huang. "A Matrix Inversion Method Based on LDL^T Decomposition and its Application in STAP". In: *2022 7th International Conference on Signal and Image Processing (ICSIP)*. IEEE. 2022, pp. 756–762.

[121] Yuliang Ji, Jian Wu, and Yuanzhe Xi. "Rethinking Neural-based Matrix Inversion: Why can't, and Where can". In: *arXiv preprint arXiv:2506.00642* (2025).

[122] Jixiao Deng, Qinglin Wang, Lin Chen, et al. "IA-Chol: Input-Aware Cholesky Decomposition on CPU and GPU". In: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*. 2025.

[123] Jérémy Morlier, Robin Geens, Stef Cuyckens, et al. *MONET: Modeling and Optimization of neural NEtwork Training from Edge to Data Centers*. 2026. arXiv: 2603.15002 [cs.LG]. URL: https://arxiv.org/abs/2603.15002.

[124] Arne Symons, Linyan Mei, Steven Colleman, Pouya Houshmand, Sebastian Karl, and Marian Verhelst. "Stream: Design space exploration of layer-fused DNNs on heterogeneous dataflow accelerators". In: *IEEE Transactions on Computers* 74.1 (2024), pp. 237–249.

[125] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. "Transformers: State-of-the-art natural language processing". In: *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 2020, pp. 38–45.

[126] Chris Lattner and Vikram Adve. “LLVM: A compilation framework for lifelong program analysis & transformation”. In: *International symposium on code generation and optimization, 2004. CGO 2004.* IEEE. 2004, pp. 75–86.

[127] Martín Abadi, Paul Barham, Jianmin Chen, et al. “{TensorFlow}: a system for {Large-Scale} machine learning”. In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, pp. 265–283.